

THE UNIVERSITY OF HONG KONG

COMP3259: PRINCIPLES OF PROGRAMMING LANGUAGES

Tutorial 9

Tutor

Wan Qianrong `qywan@cs.hku.hk`

Instructor

Bruno Oliveira `bruno@cs.hku.hk`

29 April 2025

Table of Contents

1	Introduction	3
2	An Imperative Language	4
2.1	A Monadic Interpreter for an Imperative Language	5
2.2	A Monadic Interpreter with Error Handling	6

1 Introduction

This tutorial aims at providing students with experience in writing monadic interpreters, especially interpreters with imperative features.

2 An Imperative Language

Now that we already have experience with monads in the last tutorial, we can come back to our SNH language. Previous tutorials have been focusing on functional programming (i.e. no mutation is allowed.) In this tutorial, we are going to add some imperative features to SNH. However, unlike JavaScript, we use a model where mutable variables have to be explicitly declared using the `Mutable` constructor. Furthermore, accessing a mutable variable should be prefixed with `@`, and `:=` is used to update a mutable variable. Here are two examples:

```
var mb = Mutable(true);
var mi = Mutable(6);
if (@mb) {
  mi := @mi + 1
} else {
  mi := @mi - 1
}

var mx = Mutable(4);
var my = Mutable(8);
mx := @mx + @my;
my := @mx - @my;
mx := @mx - @my
```

Note that semicolons (`;`) now have two different meanings, although the two usages are quite similar. On the first two lines, the semicolons are used to separate initial values and declaration bodies. The other usage is called sequential composition, which is a standard construct in imperative programs. It sequentializes two expressions and keeps the result of the latter, discarding the former's result while still allowing it to perform side effects like mutating the state.

The abstract syntax of the changed SNH is as follows:

```
data Value = IntV Int
           | BoolV Bool
           | ClosureV String Exp Env
           | AddressV Int      -- new
deriving Eq
```

An `AddressV` is generated by the `Mutable` constructor, which contains an address in a virtual memory.

```
data Exp = Lit Value
        | Unary UnaryOp Exp
        | Bin BinaryOp Exp Exp
        | If Exp Exp Exp
        | Var String
        | Decl String Exp Exp
        | Call Exp Exp
        | Fun String Exp
        | Mutable Exp           -- new
        | Access Exp           -- new
        | Assign Exp Exp       -- new
        | Seq Exp Exp          -- new
    deriving (Eq, Show)
```

Sequential composition is modeled by the `Seq` constructor, which consists of two expressions. The `Access` and `Assign` constructors, as the names suggest, are used for accessing and updating mutable variables.

In an imperative language, besides environments, we also need a virtual memory and a notion of *stateful* values. They are modeled as follows:

```
type Memory = [Value]

type Stateful t = Memory -> (t, Memory)
```

`Stateful.hs` includes an incomplete evaluator for SNH. You can start with the code there and try to understand how the environment and the virtual memory interact for the case of `Mutable`. Your job is to finish the remaining cases, following the same idea.

Question 1. Complete the cases for `Access`, `Assign`, and `Seq` in the file `Stateful.hs`.

Hint: For `Seq`, evaluate the first expression, throw away the result, and then evaluate the second expression. Note that the two helper functions `access` and `update` may come in handy when interacting with the virtual memory.

2.1 A Monadic Interpreter for an Imperative Language

We are now going to implement the same language using a *monadic* interpreter. The code presented here is available in the file `StatefulMonad.hs`.

The interpreter can be monadic because `Stateful` is actually a monad, which can be defined as follows:

```
newtype Stateful t = ST (Memory -> (t, Memory))

instance Monad Stateful where
  return val = ST $ \m -> (val, m)
  ST c >>= f = ST $ \m ->
    let (val, m') = c m
    in ST c' = f val
    in c' m'
```

The monad allows us to rewrite the evaluator such that the details of threading the state through the evaluation is encapsulated by the monadic operation `>>=`. The rewritten code is much simpler and cleaner.

With the `Stateful` monad, the type of `evaluate` is as follows:

```
evaluate :: Exp -> Env -> Stateful Value
```

Question 2. Complete the definition of `evaluate` using the `do`-notation in the file `StatefulMonad.hs`.

At the end, we use an empty (initial) memory to extract the value from the `Stateful` monad obtained by evaluation.

2.2 A Monadic Interpreter with Error Handling

Since we have known that error handling is also a monadic operation, what if we combine the two monads into a single one and obtain an interpreter that both checks errors and supports mutable state?

There are at least two ways to compose the two monads `Checked` and `Stateful`. One way is:

```
newtype CheckedStateful1 t = CST1 (Memory -> (Checked t, Memory))
```

Another way is:

```
newtype CheckedStateful2 t = CST2 (Memory -> Checked (t, Memory))
```

Question 3. Fill out the missing monadic operators (`return` and `>>=`) for these two monads in the file `StatefulCheckedMonad.hs`.

THE UNIVERSITY OF HONG KONG

COMP3259: PRINCIPLES OF PROGRAMMING LANGUAGES

Tutorial 8

Tutor

Wan Qianrong `qywan@cs.hku.hk`

Instructor

Bruno Oliveira `bruno@cs.hku.hk`

15 April 2025

Table of Contents

1	Introduction	3
2	A Language with Recursive Functions	4
3	Monads	6
3.1	A Navigation Language	6
3.2	Introducing Monads	7
3.3	Parametrization of Monads	9

1 Introduction

This tutorial aims at providing students with experience in writing interpreters and type checkers with first-class functions and *recursion*. In addition, we will introduce writing interpreters in a *monadic* way.

2 A Language with Recursive Functions

We now continue to improve SNH by supporting recursive definitions. For example, you may want to write a Fibonacci function as follows:

```
var fib = function(x: Int) {  
  if (x == 0) 0;  
  else if (x == 1) 1;  
  else fib(x-1) + fib(x-2)  
};  
fib(10)
```

Recall that the code for evaluating non-recursive variable declarations is:

```
evaluate (Decl x e body) env =  
  let newEnv = (x, evaluate e env) : env  
  in evaluate body newEnv
```

The declared variable `x` itself is not in the environment so it cannot be used in `e`. We can add support for recursion in SNH with the help of recursion in Haskell. By using a *circular program*, the change to variable declarations is minimal:

```
evaluate (Decl x e body) env =  
  -- when evaluating `e`, use `newEnv` instead of `env`  
  let newEnv = (x, evaluate e newEnv) : env  
  in evaluate body newEnv
```

Similarly, when type checking a variable declaration, we don't know the type of `x` that may be recursively used in `e`:

```
tcheck (Decl x e body) env = do  
  t <- tcheck e env  
  tcheck body ((x, t) : env)
```

Unfortunately, the same trick doesn't apply here because it leads to an endless loop. To remedy this, one simple way is to give extra information to the type checker. Therefore, we slightly alter the syntax of variable declarations, that is, we require every variable to be explicitly typed. For example, `fib` now becomes:

```
var fib: Int -> Int = function(x: Int) {  
  if (x == 0) 0;  
  else if (x == 1) 1;  
  else fib(x-1) + fib(x-2)  
};  
fib(10)
```

Here we specify the type of `fib` as `Int -> Int`. It may seem tedious to write types for non-recursive declarations, but it makes type checking recursive functions much easier.

How are variable declarations type checked then? The answer is that we assume the variable have the specified type and then check whether the definition is consistent with this assumption. In the case of `fib`, we assume it have type `Int -> Int` in the typing environment, and then call `tcheck e`. Since the result is indeed `Int -> Int`, it type checks.

Question 1. Fix type checking `Decl` in the file `TypeCheck.hs`.

Now you can recompile `SNH` and test if `demo.txt` can type check.

Question 2. In Tutorial 7, we built a stack machine called `K`. Does the `K` machine support recursive definitions? If so, show the evaluation derivation of `fib(2)`; if not, modify the implementation to support recursion.

3 Monads

Let's take a closer look at monads and understand why they can be useful. In functional programming (Haskell in particular), monads are used as a *design pattern*. This means that monads offer a reusable solution to a commonly occurring problem. The abstraction of monads reduces boilerplate code in Haskell programming. We are going to see several monads and how they can be used to model errors.

3.1 A Navigation Language

Suppose that we have a very small language which simulates movements in a plane. We may define the current location as a pair of coordinates:

```
type Loc = (Int, Int) -- (x, y)
```

We want to define three functions (**up**, **down**, and **left**), which represent safe movements within the plane. By “safe” we mean that only non-negative coordinates are allowed after movements. It should be clear that, in functional programming, we would like to have total functions. Thus, we want potential errors to be reflected in the type system, rather than take the risk of blowing up the program at run time.

Let's first consider the following (possibly unsafe) definitions:

```
up    :: Loc -> Loc  -- increases y by 1
down  :: Loc -> Loc  -- decreases y by 1
left  :: Loc -> Loc  -- decreases x by 1
```

Judged by their names and type signatures, the definition of **up** should be correct: no matter what valid location is passed, it will always return another valid location. However, **down** and **left** are error-prone: for some valid locations taken as input, they may produce invalid locations. For instance, **down** (3, 0) will produce the invalid location (3, -1). Therefore, we want **down** and **left** to return errors in those cases. Here are three possible options for **down**:

```
down1 :: Loc -> Maybe Loc
down2 :: Loc -> List Loc
down3 :: Loc -> Either String Loc
```

All the three implementations are trying to convey the same information. In case the movement is invalid, `down1` returns `Nothing`; `down2` returns an empty list; `down3` returns `Left err` (where `err` is the error message). Otherwise, `down1` returns `Just v`; `down2` returns `[v]` (a singleton list); `down3` returns `Right v`.

Then we can write a sample program that composes `left`, `down`, and an extra `update`. Note that we have to constantly check errors:

```
update :: (Int -> Int) -> (Int -> Int) -> Loc -> Loc
update f g (x, y) | f x >= 0 && g y >= 0 = (f x, g y)
                  | otherwise           = (x, y)

ex1 :: Loc -> Maybe Loc
ex1 loc = case left1 loc of
  Nothing -> Nothing
  Just loc' -> case down1 loc' of
    Nothing -> Nothing
    Just loc'' -> Just $ update (+1) (+1) loc''

ex2 :: Loc -> List Loc
ex2 loc = case left2 loc of
  Nil -> Nil
  Cons loc' _ -> case down2 loc' of
    Nil -> Nil
    Cons loc'' _ -> singleton $ update (+1) (+1) loc''

ex3 :: Loc -> Either String Loc
ex3 loc = case left3 loc of
  Left err -> Left err
  Right loc' -> case down3 loc' of
    Left err' -> Left err'
    Right loc'' -> Right $ update (+1) (+1) loc''
```

As you can witness, this kind of code is extremely tedious to write. Furthermore, no matter which option we choose, the code looks remarkably similar. This motivates the need for a new design pattern. The most common solution amongst functional programmers is probably monads.

3.2 Introducing Monads

Monads are data structures that support two primitive operators: `return` (`pure`) and `bind`. Haskell models them as a type class, which can be thought as an interface that requires every member of it to implement the two operators:

```
class Monad m where
  return :: a -> m a
  (>>=)  :: m a -> (a -> m b) -> m b
```

The latter (`bind` is denoted as `>>=` in Haskell) serves the purpose of sequencing; while the former injects a value into a monad. The benefit of using a monad is that constant checking for corner cases is no longer visible to the programmer.

We can declare all three types (i.e. `Maybe`, `List`, and `Either String`) as monads by define the two primitive operators. You should have tried to implement `bind` for `Either` in Assignment 1. It is important for you to understand what is happening under the hood for the other two cases. For example, the `Maybe` monad is implemented as:

```
instance Monad Maybe where
  return a      = Just a
  ma >>= a2mb = case ma of Nothing -> Nothing
                  Just a   -> a2mb a
```

We define two functions: `return a` and `ma >>= a2mb`. The first function, given a value of any type, injects it into the monad by applying the `Just` constructor to it. The second function, given a computation which is already inside the monad (i.e. a term of type `Maybe a`) and a continuation in case the evaluation is successful, performs pattern matching and applies the continuation only when the first computation succeeds. Therefore, error checking is hidden behind the `>>=` operator. In fact, our previous example `ex1` was no more than a sequence of these checks, followed by an injection of an application of the `update` function. It means that we can write less verbose code to express equivalent logic:

```
ex1monad :: Loc -> Maybe Loc
ex1monad loc = left1 loc  >>= \loc'  ->
                down1 loc' >>= \loc'' ->
                return $ update (+1) (+1) loc''
```

This pattern is so common in Haskell that there is a special syntax for it. The same program can be written with the *do-notation*:

```
ex1do :: Loc -> Maybe Loc
ex1do loc = do loc'  <- left1 loc
               loc'' <- down1 loc'
               return $ update (+1) (+1) loc''
```

Here, `do` is a keyword indicating that the inner block is running inside a monad. Each occurrence of `<-` is translated to a use of `>>=`. Finally, `return` remains unchanged. This notation is reminiscent of some form of imperative programming.

Question 3. Complete the definitions of `ex2do` and `ex3do` using the `do`-notation.

3.3 Parametrization of Monads

Though we have already written neat monadic code, we can still add a final degree of abstraction. Since we went through the trouble of defining safe operators for all the three monads, we can now easily offer programmers the opportunity to choose their preferred ones!

The idea is that we define a new function that runs our language inside any monad, as long as it provides the basic constructs. Thus, the example can be parametrized by the type of monads:

```
exM :: Monad m => (Loc -> m Loc) -> (Loc -> m Loc) -> (Loc -> m Loc)
               -> Loc -> m Loc
exM up down left loc = undefined
```

The first three parameters are the base constructs: `up`, `down`, and `left`. The forth parameter is the input location. Notice that even though we are free to choose a `Monad` and its implementation for the basic operators, we can not mix different monads together. We can choose to call `exM` with `up1`, `down1`, and `left1`, but not with nonsensical combinations like `up1`, `down2`, and `left3`. For example, the following program instantiates `exM` with the `Either String` monad:

```
exEither :: Loc -> Either String Loc
exEither = exM up3 down3 left3
```

Question 4. Complete the definition of `exM` using the `do`-notation, whose logic is equivalent to `ex1do`, `ex2do`, or `ex3do`.

The evaluators we wrote in previous lectures and tutorials can also be simplified by monads and `do`-notation. Try to find some of them and refactor by yourself!

THE UNIVERSITY OF HONG KONG

COMP3259: PRINCIPLES OF PROGRAMMING LANGUAGES

Tutorial 7

Tutor

Wan Qianrong `qywan@cs.hku.hk`

Instructor

Bruno Oliveira `bruno@cs.hku.hk`

1 April 2025

Table of Contents

1	Introduction	3
2	A Stack Machine	4

1 Introduction

In this tutorial, we will introduce a *stack machine* for evaluating expressions. This material is for your self-study and *will not be covered* in the exam.

2 A Stack Machine

One disadvantage of our previous implementation of evaluation is that we make the control flow *implicit*. In other words, we are utilizing the semantics of the meta-language (Haskell) to help define the semantics of our language (SNH). Take the evaluation of addition for example:

```
evaluate (Add a b) = evaluate a + evaluate b
```

From the above code, we don't know which operand (*a* or *b*) is evaluated first. This is all controlled by Haskell. Of course, the order doesn't matter to addition (we get the same result either from left to right or from right to left.) But for *If* expressions:

```
evaluate (If a b c) =  
  let BoolV test = evaluate a  
  in if test then evaluate b else evaluate c
```

The semantics of *If* in SNH tells us that either *b* or *c* is evaluated but not both. Here we are utilizing the semantics of *if* in Haskell to help guarantee that. But what if the Haskell compiler has a bug and evaluates both branches of *if* no matter what the predicate is? Then we are in trouble.

That's our motivation to use a stack machine to make the control flow *explicit*. Java is one example of the real-world programming languages that use a stack machine for execution. The standard model is that we compile Java source code to Java bytecode, which is then interpreted by the Java virtual machine (JVM). One benefit of bytecode is that it is portable to any architecture as long as the JVM is available.

Here we introduce a different stack machine from Java called **K**. A state *s* of the K machine consists of a control stack *k*, which records the work remaining to be done after an instruction is executed, and a closed expression *e*. States are in either of the two forms:

1. An *evaluation* state of form $k \succ e$, which corresponds to the evaluation of a closed expression *e* on a control stack *k*;
2. A *return* state of form $k \prec v$ (we use *v* to imply that *v* must be a value), which returns a value *v* that has been computed.

The control stack represents the evaluation context. It records the “current location” of evaluation, where the value of the current expression is returned to. A control stack k may contain control frames f as well as evaluation environments Δ :

$$k ::= \epsilon \mid k \triangleright f \mid k \blacktriangleright \Delta$$

Formally speaking, a stack is either empty (ϵ), or a stack k plus a control frame f ($k \triangleright f$), or a stack k plus an evaluation environment Δ ($k \blacktriangleright \Delta$).

A frames f is defined as follows:

$$\begin{array}{ll} f ::= u(-) & \text{unary operations} \\ \mid o(-, e_2) \mid o(v_1, -) & \text{binary operations} \\ \mid \text{call}(-, e_2) \mid \text{call}(v_1, -) & \text{function calls} \\ \mid \text{decl}(x, -, e) & \text{variable declarations} \\ \mid \text{if}(-, e_1, e_2) & \text{conditionals} \end{array}$$

A hole (denoted as $-$) in the top frame is intended to hold the value returned by evaluating the current expression. It corresponds to the location in an expression where evaluation are taking place. By placing a hole in an expression, we effectively specify which part to evaluate first.

We often need to get the most recent (outermost) evaluation environment of a stack. This operation is denoted by $\Delta(k)$ and is recursively defined by:

$$\begin{aligned} \Delta(\epsilon) &= \cdot \\ \Delta(k \triangleright f) &= \Delta(k) \\ \Delta(k \blacktriangleright \Delta') &= \Delta' \end{aligned}$$

The main judgment defining the stack machine is:

$$s \longmapsto s'$$

It means that state s makes a transition to state s' in one step. An initial state of the stack machine has the form $\epsilon \succ e$, and a final state has the form $\epsilon \prec v$. Transitions of the stack machine are in a *small-step* style, and to establish a relation with big-step operational semantics taught in the lectures and tutorials, we have: if $\cdot \vdash e \longrightarrow v$, then

$$\epsilon \succ e \longmapsto \cdots \longmapsto \epsilon \prec v$$

We begin with the rule for literals. To evaluate a literal n , we simply return it:

$$k \succ n \longmapsto k \prec n$$

Next, we consider the rules for unary expressions:

$$k \succ u(e) \longmapsto k \triangleright u(-) \succ e$$

$$k \triangleright u(-) \prec v \longmapsto k \prec u(v)$$

To evaluate $u(e)$, we push the frame $u(-)$ onto the stack to record the pending unary operation and evaluate e ; when a value v is returned, we calculate the result of $u(v)$.

The rules are similar for binary expressions:

$$k \succ o(e_1, e_2) \longmapsto k \triangleright o(-, e_2) \succ e_1$$

$$k \triangleright o(-, e_2) \prec v_1 \longmapsto k \triangleright o(v_1, -) \succ e_2$$

$$k \triangleright o(v_1, -) \prec v_2 \longmapsto k \prec o(v_1, v_2)$$

To evaluate $o(e_1, e_2)$, we first push the frame $o(-, e_2)$ to the stack and evaluate e_1 . When v_1 is returned, we pop the top frame and push a new frame $o(v_1, -)$ to the stack. After e_2 is evaluated, we return the result of $o(v_1, v_2)$. This accounts for the left-to-right evaluation order of binary expressions.

Then it comes to the rules for conditionals:

$$k \succ \text{if}(e_1, e_2, e_3) \longmapsto k \triangleright \text{if}(-, e_2, e_3) \succ e_1$$

$$k \triangleright \text{if}(-, e_2, e_3) \prec \text{true} \longmapsto k \succ e_2$$

$$k \triangleright \text{if}(-, e_2, e_3) \prec \text{false} \longmapsto k \succ e_3$$

To evaluate $\text{if}(e_1, e_2, e_3)$, we first push the frame $\text{if}(-, e_2, e_3)$ to the stack and evaluate e_1 . If **true** is returned, we evaluate e_2 ; otherwise, we evaluate e_3 . Note that we don't need the help of Haskell's **if** expressions, we gain the control by ourselves!

The rule for functions is as simple as that for literals:

$$k \succ \text{function}(x)\{e\} \mapsto k \prec (\text{function}(x)\{e\}, \Delta(k))$$

We pack the most recent environment $\Delta(k)$ together with the function definition to form a closure.

Now let's consider the rules for function calls:

$$k \succ \text{call}(e_1, e_2) \mapsto k \triangleright \text{call}(-, e_2) \succ e_1$$

$$k \triangleright \text{call}(-, e_2) \prec v_1 \mapsto k \triangleright \text{call}(v_1, -) \succ e_2$$

$$k \triangleright \text{call}((\text{function}(x)\{e\}, \Delta), -) \prec v_2 \mapsto k \blacktriangleright (\Delta, x = v_2) \succ e$$

Just like our previous implementation of function calls, we first evaluate e_1 and then e_2 . The result of evaluating e_1 must be a closure. We then push the environment $(\Delta, x = v_2)$ to the stack and continue to evaluate the body e .

The rules for variable declarations are as follows:

$$k \succ \text{decl}(x, e_1, e_2) \mapsto k \triangleright \text{decl}(x, -, e_2) \succ e_1$$

$$k \triangleright \text{decl}(x, -, e_2) \prec v_1 \mapsto k \blacktriangleright (\Delta(k), x = v_1) \succ e_2$$

We first push a new frame $\text{decl}(x, -, e_2)$ to the stack and evaluate the bound expression e_1 . Then we pop the top frame and push the environment $(\Delta(k), x = v_1)$ to the stack, and evaluate e_2 .

Finally, we have the rule for variables:

$$k \succ x \mapsto k \prec \Delta(k)(x)$$

To evaluate x , we get the most recent environment $\Delta(k)$ from the stack and fetch the value from it.

The last rule is just to repeatedly pop environments:

$$k \blacktriangleright \Delta \prec v \mapsto k \prec v$$

It takes quite a lot of time to write down all the rules! These rules will make much more sense when you try to evaluate an expression by yourself. I encourage you to do that with pen and paper before writing any code. As an example, below shows the evaluation derivation of `(function(x){x + 1})(3)`:

ϵ	$\succ$	<code>call(function(x){x + 1}, 3)</code>
$\mapsto \epsilon \triangleright \text{call}(-, 3)$	$\succ$	<code>function(x){x + 1}</code>
$\mapsto \epsilon \triangleright \text{call}(-, 3)$	$\prec$	<code>(function(x){x + 1}, .)</code>
$\mapsto \epsilon \triangleright \text{call}((\text{function}(x)\{x + 1\}, .), -)$	$\succ$	<code>3</code>
$\mapsto \epsilon \triangleright \text{call}((\text{function}(x)\{x + 1\}, .), -)$	$\prec$	<code>3</code>
$\mapsto \epsilon \blacktriangleright x = 3$	$\succ$	<code>x + 1</code>
$\mapsto \epsilon \blacktriangleright x = 3 \triangleright +(-, 1)$	$\succ$	<code>x</code>
$\mapsto \epsilon \blacktriangleright x = 3 \triangleright +(-, 1)$	$\prec$	<code>3</code>
$\mapsto \epsilon \blacktriangleright x = 3 \triangleright +(3, -)$	$\succ$	<code>1</code>
$\mapsto \epsilon \blacktriangleright x = 3 \triangleright +(3, -)$	$\prec$	<code>1</code>
$\mapsto \epsilon \blacktriangleright x = 3$	$\prec$	<code>4</code>
$\mapsto \epsilon$	$\prec$	<code>4</code>

Question 1. Write the evaluation derivation for the following expression:

```
var bar = 4;
var foo = function(x) { x + bar };
var bar = 8;
foo(bar)
```

Now let's turn to the implementation. To start off, we first consider the data structure for frames:

```
data Frame = FUnary UnaryOp
  | FBin BinaryOp (Either Value Exp)
  | FIf Exp Exp
  | FCall1 ()           -- TODO: Give a proper type for both kinds
  | FCall2 ()           --       of call frames
  | FDecl ()           -- TODO: Give a proper type for decl frames
  | FEnv Env
```

Question 2. Replace `()` with proper types for `call` and `decl` frames in `Interp.hs`.

For the sake of convenience, we tactically make environments part of frames. That's why we have the `FEnv` constructor in `Frame`. Then a stack is just a list of frames:

```
type Stack = [Frame]
```

States of our stack machine are represented as follows:

```
data State = Eval Stack Exp
           | Return Stack Value
```

Operation $\Delta(k)$ is just a simple recursive function:

```
getEnv :: Stack -> Env
getEnv [] = []
getEnv (FEnv env : k) = env
getEnv (_ : k) = getEnv k
```

The soul of our stack machine is the `step` function, which specifies how one state makes a transition to another:

```
step :: State -> State
```

Question 3. Implement the `step` function in `Interp.hs` according to the rules we have elaborated above.

To run the stack machine, we just repeat `step` until it reaches the final state:

```
execute' :: Exp -> Value
execute' e = go (Eval [] e)
  where go :: State -> Value
        go (Return [] v) = v
        go s = go (step s)
```


THE UNIVERSITY OF HONG KONG

COMP3259: PRINCIPLES OF PROGRAMMING LANGUAGES

Tutorial 6

Tutor

Wan Qianrong `qywan@cs.hku.hk`

Instructor

Bruno Oliveira `bruno@cs.hku.hk`

25 March 2025

Table of Contents

1	Introduction	3
2	A Language with First-Class Functions	4
2.1	Implementing First-Class Functions	5
2.2	Type Checker, Revisited	6
2.3	REPL	7
3	Call-by-Value and Call-by-Name	8
3.1	Call-by-Value	8
3.2	Call-by-Name	9

1 Introduction

This tutorial aims at providing students with experience in writing interpreters and type checkers with *first-class* functions. We will then discuss some different design choices in terms of evaluation strategies.

2 A Language with First-Class Functions

In this tutorial, we continue to extend SNH with first-class functions. The datatypes for types, values, and expressions are changed accordingly:

```
data Type = TInt
          | TBool
          | TFun Type Type           -- added
          deriving Eq

data Value = IntV Int
           | BoolV Bool
           | ClosureV (String, Type) Exp Env -- added
           deriving Eq

data Exp = Lit Value
         | Unary UnaryOp Exp
         | Bin BinaryOp Exp Exp
         | If Exp Exp Exp
         | Var String
         | Decl String Exp Exp
         | Call Exp Exp             -- changed
         | Fun (String, Type) Exp   -- added
         deriving Eq
```

The datatype `Type` now has a new type representing functions, as denoted by the constructor `TFun`. These types are in the same spirit as Haskell function types so that you can write a function type such as `TFun TInt TInt` (like `Int -> Int` in Haskell) or a higher-order function type such as `TFun (TFun TInt TInt) TInt` (like `(Int -> Int) -> Int` in Haskell).

We also add a new kind of value named `ClosureV`, which can be thought of as the value we get when evaluating a function.

As for the revised datatype `Exp`, the new constructor `Fun` represents first-class functions: functions in the full mathematical sense are supported and can be passed around freely just as any other piece of data. Below is an expression showing how to write first-class functions in the concrete syntax:

```

var foo = function(x: Int) {
  x + 1
};
foo(3)

```

A few things deserve attention. First of all, every function now has only *one* parameter. However, this is not a limitation, as we learned in the class that functions with multiple arguments can be desugared to nested functions with one argument. In the same vein, every function call now takes only one argument. A nice thing about Curried functions is that we can have *partial application*:

```

var x = 5;
var f = function(foo: Int -> Int) {
  foo(x)
};
var add = function(x: Int) {
  function(y: Int) {
    x + y
  }
};
var x = 4;
f(add(x))

```

Here, `add(x)` is a partially applied function, which is then passed as an argument to another function `f`. Before proceeding any further, make sure you understand the code above and know what the result is.

2.1 Implementing First-Class Functions

Let's take a closer look at `ClosureV`. A closure is a combination of a function expression and an environment, which preserves the bindings that existed at the point when the function was defined. So a function evaluates to a closure that bundles the function definition with the environment at the definition site:

$$\frac{}{\Delta \vdash \text{function}(x : T)\{e\} \rightarrow (\text{function}(x : T)\{e\}, \Delta)} \text{EFUN}$$

When evaluating a function call, we first evaluate the callee function and get a closure value. We then evaluate the argument under the same environment. Finally, we evaluate the function body of the callee function in a new environment, which is extracted from

the closure and extended with a binding between the parameter name and the argument value:

$$\frac{\Delta \vdash f \rightarrow (\text{function}(x : T)\{e'\}, \Delta') \quad \Delta \vdash e \rightarrow v \quad \Delta', x = v \vdash e' \rightarrow v'}{\Delta \vdash f e \rightarrow v'} \text{ECALL}$$

Question 1. Complete the function `evaluate` in the file `Interp.hs`.

2.2 Type Checker, Revisited

As before, we need a type environment (denoted by Γ) to record variables and their associated types:

```
type TEnv = [(String, Type)]
```

Recall that we talked about typing rules in Tutorial 4, which are used to guide the type checker. In this tutorial, we have two more typing rules:

- **Type-checking functions:**

$$\frac{\Gamma, x : T \vdash e : T'}{\Gamma \vdash \text{function}(x : T)\{e\} : T \rightarrow T'} \text{TFUN}$$

To type-check a function definition (*abstraction*), we first type-check its body e in the augmented environment $(\Gamma, x : T)$. If type-checking is successful, we will get a type T' , which is the type of the body. Thus the overall type of the function is $T \rightarrow T'$.

- **Type-checking function calls:**

$$\frac{\Gamma \vdash f : T \rightarrow T' \quad \Gamma \vdash e : T}{\Gamma \vdash f e : T'} \text{TCALL}$$

To type-check a function call (*application*), we first type-check f and see if its type is a function type $T \rightarrow T'$. Then we type-check the argument e and see if its type is indeed T . If so, the type of the function call is T' .

Question 2. Complete the function `tcheck` in the file `TypeCheck.hs`.

2.3 REPL

In Assignment 2, we showed how to build an executable for our SNH interpreter. This time we update it to a REPL (Read–Eval–Print Loop). You can easily install it by running:

```
$ stack install
```

The SNH executable can be run in two ways:

```
$ snh                # launch a REPL
$ snh demo.txt        # execute the program in 'demo.txt'
```

Or, if you don't want to install the executable to your local bin path, you can use the following commands:

```
$ stack build          # compile SNH
$ stack exec snh       # launch a REPL
$ stack exec snh demo.txt # execute the program in 'demo.txt'
```

In the REPL, you can type any expression you want to evaluate, for example:

```
SNH> var x = 4; x + 8
12 : Int
```

It outputs the result of evaluation, along with the type of the expression.

There are two built-in commands in the REPL:

Command	Action
:load <file>	Load an SNH program from a file.
:quit	Quit the REPL.

Enjoy yourself!

3 Call-by-Value and Call-by-Name

In languages with declarations and first-class functions, there are a few different design choices in terms of evaluation strategies.

3.1 Call-by-Value

So far, our interpreter employs one particular evaluation strategy, called *call-by-value*. With call-by-value evaluation, the argument are always evaluated to a value eagerly before being added to the environment. The same strategy applies to the evaluation of the assigned value in a variable declaration.

In other words, we evaluate the argument before evaluating the function body in the case of function applications; we evaluate the assigned value before evaluating the body of the declaration in the case of variable declarations.

The call-by-value strategy is the most common design for function calls in programming languages. A majority of programming languages (including C++, Java, and C#) use call-by-value by default. Note that some programming languages also support other evaluation strategies. For example, C++ also supports call-by-reference.

Drawback of call-by-value: Call-by-value can waste resources on useless evaluation in some cases. For example, consider the following expression:

```
var x = <long-computation>; 3
```

Here, `<long-computation>` stands for an expression that takes *really* long time to compute. But the final result is always 3 regardless of the value to which `<long-computation>` evaluates. So in this case, it seems a waste to spend resources on `<long-computation>`.

3.2 Call-by-Name

A different design choice is *call-by-name*. With call-by-name evaluation, an expression is not evaluated until it is used. So in the same example:

```
var x = <long-computation>; 3
```

`<long-computation>` is never evaluated, which makes the evaluation much faster.

Haskell is one of the few languages where (a variant of) call-by-name is the default strategy for evaluating function applications (lazy evaluation). It allows infinite streams whose computation will not terminate (or you need to manually `delay` it by wrapping it in a function) in call-by-value languages:

```
intFrom n = n : intFrom (n + 1)
take 10 $ intFrom 5

sieve (s:sx) = s : sieve (filter (\x -> x `mod` s /= 0) sx)
primes = sieve [2..] -- list contains ALL primes
take 10 primes

inf n = n : inf n
nats = 0 : zipWith (+) (inf 1) nats
take 10 nats

fibs = 0 : 1 : zipWith (+) fibs (tail fibs)
take 10 fibs
```

Drawback of call-by-name: While the program above benefits from call-by-name, there are also programs where call-by-name is worse than call-by-value. For example:

```
var x = <long-computation>; x + x
```

With call-by-name evaluation, `<long-computation>` will be evaluated twice, since evaluation is delayed to use sites of the expression, whereas it will be evaluated only once with call-by-value evaluation.

In languages like Haskell, this drawback is avoided by using an optimization of call-by-name called *call-by-need*. With call-by-need evaluation, when the use of a variable forces the evaluation of an expression, the result of that evaluation is cached (*memoization*). Next time when the variable is used, we can simply return the cached value. This means an expression will be evaluated **at most once** with call-by-need evaluation.

In this section, let's develop a variant of our interpreter using the *call-by-name* evaluation strategy. A significant change is concerning the environment:

```
data ClosureExp = CExp Exp Env
type Env = [(String, ClosureExp)]
```

That is, instead of a `Value`, we bind a variable to a `ClosureExp`, which consists of an expression and its associated environment. A `ClosureExp` is similar to a closure that we need to remember the environment at the point of introducing the expression.

As for the `evaluate` function, the major changes lie in the cases of `Var`, `Decl` and `Call`:

- **Var**: Looking up a variable in the environment now returns `CExp e env`, so we need to continue to evaluate `e` in `env`;
- **Decl**: We evaluate the body of the declaration in a new environment, which is extended with a pair of the variable name and the corresponding `ClosureExp`;
- **Call**: We first evaluate the function name to get a closure value. We then evaluate the function body in a new environment, which is extracted from the closure and extended with a pair of the parameter name and the corresponding `ClosureExp`.

Question 3. Complete the function `evaluate` in the file `CallByName.hs`.

THE UNIVERSITY OF HONG KONG

COMP3259: PRINCIPLES OF PROGRAMMING LANGUAGES

Tutorial 5

Tutor

Wan Qianrong `qywan@cs.hku.hk`

Instructor

Bruno Oliveira `bruno@cs.hku.hk`

18 March 2025

Table of Contents

1	Introduction	3
2	Free Variables and Closed Expressions	4
3	Nameless Representation	6
3.1	Renaming	6
3.2	De Bruijn Indices	7
3.3	Target Syntax	7
3.4	Translation Algorithm	8

1 Introduction

In this tutorial, we continue with the language with Boolean values added in Tutorial 4. We will introduce new notions to reason about free and bound variables. This tutorial will deepen your understanding of lexical scoping.

2 Free Variables and Closed Expressions

Sometimes we are going to check whether an expression is legal. One of these cases is that some variables exist without any declaration in advance. These illegal variables are called *free variables*.

The set of free variables within an expression contains the variables that are not bound by any variable declaration. For example, for the expression:

```
x * x
```

$\{x\}$ is the set of free variables. For the expression:

```
var x = 3; x * y * z
```

$\{y, z\}$ is the set of free variables. Here x is not free because there is a declaration of x . Finally, for the expression:

```
var x = y; var y = 3; x + y
```

$\{y\}$ is the set of free variables. In this case, even though there is a variable y declared inside the expression, the first occurrence of y is not bound anywhere.

Question 1. Now you are asked to define the function `fv` that calculates the set of free variables within an expression. Go to the file `Interp.hs`, you will find the following type signature:

```
fv :: Exp -> [String]
```

Hint: You may use the following two auxiliary Haskell functions:

```
union :: [String] -> [String] -> [String]
delete :: String -> [String] -> [String]
```

The `union` function takes two lists of strings and returns a list that contains all strings in the two lists without repeated elements. In other words, `union` is similar to `++`, except that it removes duplicated elements. For example:

```
> ["a","b"] ++ ["b","c"]  
["a","b","b","c"]
```

```
> union ["a","b"] ["b","c"]  
["a","b","c"]
```

The `delete` function deletes a value from a list. For example:

```
> delete "x" ["a","x","y"]  
["a","y"]
```

If an expression has no free variable, we call it a *closed expression*. In other words, a closed expression implies that all variables within it are bound by variable declarations.

Question 2. Complete the definition of `closed` in `Interp.hs`.

```
closed :: Exp -> Bool
```

3 Nameless Representation

Sometimes, we may have to deal with expressions with free variables. For example:

```
var x = y; var y = 1; x + y
```

If we follow the conventional evaluation rule to substitute x with variable y , the expression will evaluate to:

```
var y = 1; y + y
```

Which is wrong because the y is a free variable in the expression that x is being substituted into, but after the substitution it becomes bound to variable declaration `var y = 1`. In other words, y is *captured*. We want to avoid such variable capture by [Capture-Avoiding Substitutions](#).

3.1 Renaming

Note that we give every variable a name in our language. In fact, every bound variables can be renamed **to a fresh name** without changing the semantics of expressions, which effectively prevents unexpected capture. For example, consider the expression we show above:

```
var x = y; var y = 1; x + y
```

We can replace the inner y by z :

```
var x = y; var z = 1; x + z
```

Although different variable names are used, they are obviously two equivalent expressions under the notion of [\$\alpha\$ -equivalence](#). After substitution of x , we get:

```
var z = 1; y + z
```

As shown above, we can rename a variable to a fresh name every time we find a name conflict. However, in this way, we should maintain a set of used names, which is troublesome.

3.2 De Bruijn Indices

Instead of renaming variables, we suggest a more straightforward (but less readable) idea to rewrite the expression: referring to a variable by the distance to its binder. The distance is defined as the number of occurrences of other binders in-between. This approach is well known as *De Bruijn indices*.

Here, we replace named variables by De Bruijn indices. Each index is a natural number, corresponding to the distance to its binder. For example, the expression:

```
var x = 2; var x = x; x + 3
```

can be translated to

```
var = 2; var = {0}; {0} + 3
```

In this example, both x 's are replaced by $\{0\}$, referring to their respective closest binders (note that they are not the same.) Another example is:

```
var x = 4; var y = 5; var z = 6; x + y + z;
```

This code can be translated to:

```
var = 4; var = 5; var = 6; {2} + {1} + {0};
```

Here, $\{0\}$ represents z because there is no binder between the declaration and the occurrence of z ; $\{1\}$ represents y because there is one binder (z) between the declaration and the occurrence of y ; and $\{2\}$ represents x because there are two binders (y and z) between the declaration and the occurrence of x .

3.3 Target Syntax

Now we have a better representation of expressions, it's time to implement the translation algorithm from the source language (using names) to the target language (using indices).

First of all, we need to define the target language:

```
type Index = Int

data TExp = TLit Value
         | TUnary UnaryOp TExp
         | TBinary BinaryOp TExp TExp
```

```

    | TIf TExp TExp TExp
    | TVar Index
    | TDecl TExp TExp
deriving Show

```

As you see above, we use `Int` as `Index` and add a new datatype `TExp` to represent the syntax of the target language. `TVar` now takes an index instead of a names, and `TDecl` does not include any variable name explicitly anymore.

What we going to do next is to translate the abstract syntax in the source language to that in the target language. For example, concerning the source code:

```
var x = 4; var y = 8; x + y
```

we want to translate the syntax from:

```

source :: Exp
source = Decl "x" (Literal (IntV 4))
        (Decl "y" (Literal (IntV 8)) (Binary Add (Var "x") (Var "y")))

```

to:

```

target :: TExp
target = TDecl (TLit (IntV 4))
        (TDecl (TLit (IntV 8)) (TBinary Add (TVar 1) (TVar 0)))

```

3.4 Translation Algorithm

The algorithm is quite straightforward: we maintain a list of variable names when traversing the source AST and building the target AST. Once we meet a binder, we add the bound name to the list. Once we meet a name reference, we determine its index by searching the list.

Question 3. You are asked to write a function that translates from the source language to the target language.

```

translate :: Exp -> TExp
translate source = convert source [] -- starts with an empty environment
  where
    convert :: Exp -> [String] -> TExp
    convert = error "TODO"

```

Since the AST of the target language is different from that of the source language, we need to write a new evaluation function.

Question 4. You are asked to write a new evaluation function for the target language.

```
tevaluate :: TExp -> Value
tevaluate e = teval e [] -- starts with an empty environment
  where
    teval :: TExp -> [Value] -> Value
    teval = error "TODO"
```

Question 5. Just like what were done in Tutorial 4, you are also asked to write a type checker for target language.

```
tcheck :: TExp -> TEnv -> Maybe Type
tcheck = error "TODO"
```

THE UNIVERSITY OF HONG KONG

COMP3259: PRINCIPLES OF PROGRAMMING LANGUAGES

Supplementary Materials

Tutor

Wan Qianrong `qywan@cs.hku.hk`

Instructor

Bruno Oliveira `bruno@cs.hku.hk`

7 March 2025

Table of Contents

1	Introduction	3
2	Multi-line Definition in GHCi	4
3	Functions with Multiple Parameters, in Two Ways	5
4	Fold	8
4.1	Sidenote: Tail Recursion	10

1 Introduction

Materials on programming in Haskell and common questions for reading & thinking.

2 Multi-line Definition in GHCi

If you want to write multiple line definitions in GHCi, simply doing it line-by-line will not work:

```
ghci> isZero 0 = True
ghci> isZero _ = False
ghci> isZero 0
False
```

The reason is that, in GHCi, each line is treated as a separate definition, and the latter will shadow the previous one.

The correct way is to enclose your definition by `:{` and `:}` (note that it's not `:{ }:`), e.g.:

```
ghci> :{
ghci| isZero 0 = True
ghci| isZero _ = False
ghci| :}
ghci> isZero 0
True
ghci> isZero 1
False
```

But it's still recommended to save your code into a file and then load it into GHCi. Directly coding under GHCi will force you to start over whenever you make a mistake, which is inconvenient.

3 Functions with Multiple Parameters, in Two Ways

When you see the following definition:

```
ghci> add a b = a + b
ghci> :t add
add :: Num a => a -> a -> a
```

You may ask why the type of `add` is, say `Int -> Int -> Int`, rather than `(Int, Int) -> Int` stating the argument is “two integers.” Well, there is a longer story.

In Haskell, a function definition like `f x y z = x + y + z` is actually a “syntactic sugar” for `f = \x y z -> x + y + z`, and the right-hand side of the equal sign can be view as the “literal form” of a function (the backslash here means “ λ ” which refers to the famous λ -calculus). And let’s unfold it to the furthest:

```
\x y z -> x + y + z
-->
\x -> \y z -> x + y + z
-->
\x -> \y -> \z -> x + y + z
```

There are so many dizzy arrows! What does it mean?

First, we need a basic property of the `->` operator: it’s **right-associative**. Thus `\x -> \y -> \z -> x + y + z` actually means `\x -> (\y -> (\z -> x + y + z))`. Back to types (ignore polymorphism), when we say the type of the expression is `Int -> Int -> Int -> Int`, it also means `Int -> (Int -> (Int -> Int))`.

You may still be confused since the return value/type of the function still looks like a... function. And yes, it’s a function. Recall that in Tutorial 1, we say functions in Haskell are *first-class*. By first-class we mean functions can be stored into variables, passed around as arguments, and also returned from other functions (we call such function “higher-order” once it uses functions as arguments or return value). Our multiple arguments function above is indeed a function that takes **one** integer as the argument and returns another function.

And `f 1 2 3` actually means `((f 1) 2) 3`—apply one argument at a time and use the returned function to receive the next argument. Let’s see the reduction steps, using the substitution model:

```
f 1 2 3
-->
((\x -> (\y -> (\z -> x + y + z)) 1) 2) 3
--> x|->1
(\y -> (\z -> 1 + y + z) 2) 3
--> y|->2
(\z -> 1 + 2 + z) 3
--> z|->3
1 + 2 + 3
-->
6
```

The technique of translating a function that takes multiple arguments into a sequence of functions with each taking a single argument is called **Currying**. It looks fancy, but why do we really need it given the complexity? One advantage of Curried functions is that they allow *partial application*—since they take only one argument at a time, we do not need to get all arguments ready simultaneously. We can supply some of the arguments and use the returned function on different remaining arguments multiple times to avoid writing repeated code. Example:

```
ghci> :t map
map :: (a -> b) -> [a] -> [b]
ghci> mapPlus1 = map (+1)
ghci> :t mapPlus1
mapPlus1 :: Num b => [b] -> [b]
ghci> mapPlus1 [0]
[1]
ghci> mapPlus1 [1,2,3,4]
[2,3,4,5]
```

So what about `(Int, Int) -> Int`? An example can be `\(x, y) -> x + y :: Num a => (a, a) -> a`. One difference here is it cannot enjoy partial application, i.e., you should supply arguments altogether.

To facilitate transformation between these two kind of functions, Haskell provides us with `curry` and `uncurry` auxiliary functions:

```
ghci> :t curry
curry :: ((a, b) -> c) -> a -> b -> c
ghci> :t uncurry
```

```
uncurry :: (a -> b -> c) -> (a, b) -> c
ghci> uncPlus = \(a, b) -> a + b
ghci> cPlus = curry uncPlus
ghci> plus1 = cPlus 1
ghci> plus1 5
6
```

Question 1. Implement `curry` and `uncurry`.

Question 2. Using `uncurry`, implement `zipWith :: (a -> b -> c) -> [a] -> [b] -> [c]` in terms of `zip :: [a] -> [b] -> [(a, b)]`.

4 Fold

We have seen a recurrent pattern in our Haskell code, for example, in Tutorial 1:

```
sumList :: [Int] -> Int
sumList [] = 0
sumList (x:xs) = x + sumList xs
```

And we can find more:

```
length :: [a] -> Int
length [] = 0
length (x:xs) = 1 + length xs
```

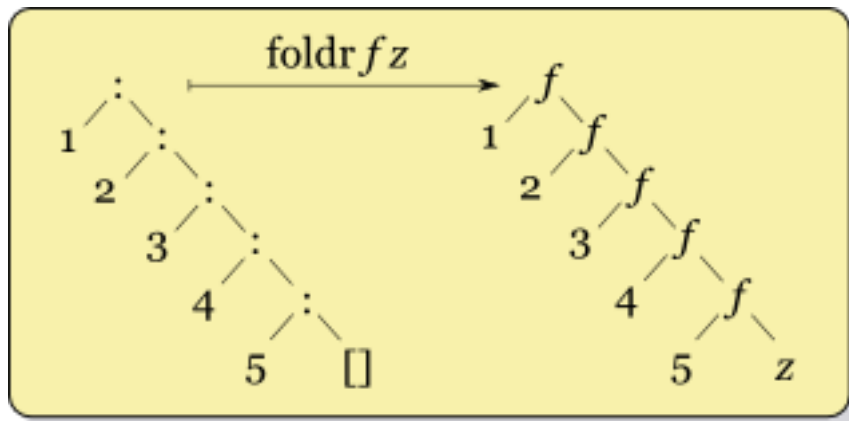
```
or :: [Bool] -> Bool
or [] = False
or (x:xs) = x || or xs
```

Among all functions, we have a “default value” for calling it on an empty list, then for non-empty lists, we take the first element, process it, and combine it with the recursive result on the list tail by some operation.

Well, we can abstract this pattern into a helper function. Here comes `foldr :: Foldable t => (a -> b -> b) -> b -> t a -> b`: the first argument is a function that combines the first argument and the recursive result on list tail, follows by the default value and the list. It is defined as

```
foldr op v [] = v
foldr op v (x:xs) = op x (foldr op v xs)
```

From the definition, we notice `foldr` will process the last (rightmost) element first:



With the help of `foldr`, we can simplify the functions above:

```
sumList = foldr (+) 0
```

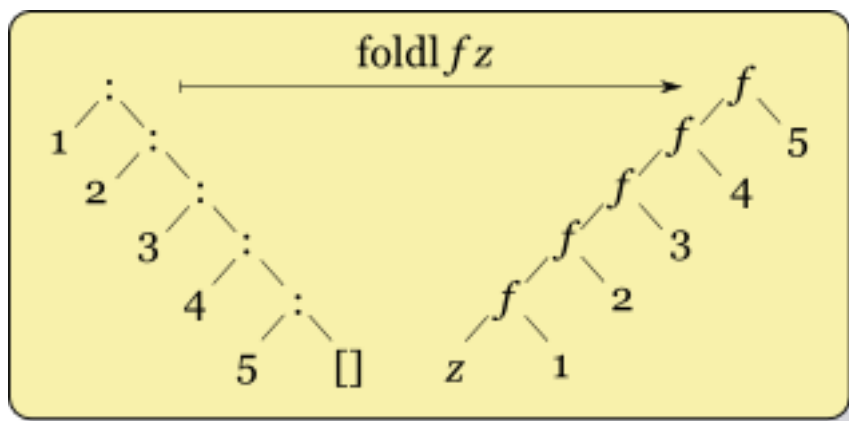
```
length = foldr (const $ \b -> b + 1) 0
```

```
or = foldr (||) False
```

Moreover, does a counterpart of `foldr` exist that processes the list from left to right? Actually we do have `foldl :: Foldable t => (b -> a -> b) -> b -> t a -> b`:

```
foldl op v [] = v
```

```
foldl op v (x:xs) = foldl op (op v x) xs
```



This time, we can view the `v` here as an “accumulator,” storing the processed result of the first few elements in the list (instead of the list tail.) You can read more about these functions in the Haskell document [\[1\]](#)[\[2\]](#).

Question 3. Using `foldr`, implement the following functions:

```

-- test if all elements satisfy a predicate function
all :: (a -> Bool) -> [a] -> Bool

-- get elements that satisfy the predicate
filter :: (a -> Bool) -> [a] -> [a]

-- [(1,2),(3,4),(5,6)] -> ([1,3,5],[2,4,6])
unzip :: [(a, b)] -> ([a], [b])

-- [1,2,3] -> [3,2,1]
reverse :: [a] -> [a]

-- [[1,2],[3,4]] -> [1,2,3,4]
flatten :: [[a]] -> [a]

```

Question 4. (HARD) Implement `foldl` and `zip :: [a] -> [b] -> [(a, b)]` based on `foldr`.

4.1 Sidenote: Tail Recursion

In the first lecture, a student proposed another way to write `sumList`:

```

sumList [] = 0
sumList (x:xs) = x + sumList xs

sumList' acc [] = acc
sumList' acc (x:xs) = sumList' (acc + x) xs

```

Both of them work. But there is one interesting difference: in `sumList'`, the recursive call is the last operation in the function body, while in `sumList` it's not. We call such call a “tail recursion.” In some other languages, a tail call may perform better since it does not need to keep the call stack.

Question 5. For `foldr` and `foldl` implemented as above, which one is in a tail recursion manner?

Note: You may also consider the relation between the two versions of `sumList` and `foldr` & `foldl`, respectively.

THE UNIVERSITY OF HONG KONG

COMP3259: PRINCIPLES OF PROGRAMMING LANGUAGES

Tutorial 4

Tutor

Wan Qianrong `qywan@cs.hku.hk`

Instructor

Bruno Oliveira `bruno@cs.hku.hk`

4 March 2025

Table of Contents

1	Introduction	3
2	Type-Checking and Abstract Interpretation	4
3	A Language with Boolean Values	5
3.1	Types Do Matter	7
3.2	Typing Rules for Expressions	8
3.3	Putting All Together	11

1 Introduction

The goal of this tutorial is to provide students with more experience in writing interpreters and also introduce them to type-checking and abstract interpretation.

2 Type-Checking and Abstract Interpretation

So far, we have been focused on writing interpreters for small languages. An interpreter is a program that works out a given expression to a value in a certain language. The value of an expression is a piece of information that is as precise as it can be by executing that program. In other words, when we evaluate an expression like the following:

```
evaluate (3 + 5) --> 8
```

The result of this particular program cannot be more precise: the expression $3 + 5$ just evaluates to the concrete number 8. The `evaluate` function implements what is called a *concrete interpreter*.

However, it is possible to write interpreters that return abstract values. Those interpreters, called *abstract interpreters*, output some abstraction of the result of executing a program.

The most common example of abstract interpretation is type-checking or type-inference. A type-checker analyzes an expression, checks whether the types of all sub-expressions are compatible, and returns the corresponding type of the expression. For example:

```
tcheck (3 + 5) --> Int
```

A type-checker works in a similar way to a concrete interpreter. The difference is that it returns a *type* instead of returning a concrete value. A type is an abstraction of values. When we see that the type of an expression is `Int`, we do not know exactly to what concrete number that expression evaluates, but we do know that it will evaluate to an Integer but not to a Boolean value, for example.

Type-checking is not the only example of abstract interpretation. In fact, abstract interpretation is a huge area of research in programming languages because various forms of abstract interpretation are useful to prove certain properties about programs.

3 A Language with Boolean Values

Before we embark on writing a type-checker, we first augment our small language again, now with Boolean values (i.e. `true` and `false`). Boolean values alone don't seem much interesting, so we will also add comparison operators (e.g. `>`, `<`, `<=`, `>=`, and `==`), logical operators (e.g. `&&`, `||`, and `!`), and conditional execution (`if-then-else`).

The augmented language we are going to use is:

```
data BinaryOp = Add | Sub | Mult | Div
              | GT  | LT  | LE   | GE   | EQ
              | And | Or

data UnaryOp = Neg | Not

data Value = IntV Int
           | BoolV Bool

data Exp = Literal Value
        | Unary UnaryOp Exp
        | Binary BinaryOp Exp Exp
        | If Exp Exp Exp
        | Var String
        | Decl String Exp Exp
```

You may notice that, in this tutorial, the representation of operators is slightly different from before. We separate unary operators from binary operators and pull them out into different datatypes. These changes are for the benefit of keeping `Exp` data type small and less clustered as more features are added to our language.

The abstract syntax of the language now has 9 new operators in 3 categories:

- Comparison operators: `>` (`GT`), `<` (`LT`), `<=` (`LE`), `>=` (`GE`), and `==` (`EQ`);
- Logical operators: `&&` (`And`), `||` (`Or`), and `!` (`Not`);
- Conditional execution: the `if-then-else` expression.

There are also two kinds of values in the language: Integers (e.g. `IntV 3`) and Boolean values (e.g. `BoolV True`).

With these changes, our language now seems more like what we would expect from a programming language. Now you can write programs like:

```
var x = 3; var y = 4; if (x >= y) x + 1; else y + 1 // returns 5
var x = true; var y = false; (x || y) && (!x || !y) // returns true
```

Note that the concrete syntax for an `if` expression is:

```
if <condition> <exp>; else <exp>
```

As before, whenever we add new features to our language, we will have to rethink how evaluation works in the language. This time, it is slightly more complicated than before because we changed the representation of operators. Moreover, we have Boolean values in the language, so evaluating an expression can return either an Integer or a Boolean value.

To bridge the gap between different operators and values, we will define two auxiliary functions as follows:

```
unary  :: UnaryOp -> Value -> Value
binary :: BinaryOp -> Value -> Value -> Value
```

Given the type signatures, the two functions above should be self-explanatory.

Question 1. Complete the definition of `unary` and `binary` in the file `Interp.hs`.

Note: You should be cautious about exhausting all possible combinations of operators and values. For example, the binary operator `Add` expects its two operands to be Integers. Therefore, you can take for granted that `Add` is undefined for Boolean values.

Now we are ready to implement the evaluation function again. Same as the last tutorial, we will need an *environment* as a collection of *bindings*. Go to the file `Interp.hs`, you will find the following definitions:

```
evaluate :: Exp -> Value
evaluate e = eval e [] -- starts with an empty environment
  where eval :: Exp -> Env -> Value
        eval = error "TODO: Question 2"
```

Question 2. Fill in the missing part of the interpreter function `eval`. Remember to use `unary` and `binary` functions in the cases of `Unary` and `Binary` constructors, respectively.

If you play around with the new evaluation function for a while, you may encounter something as follows when you mistakenly enter some invalid expressions:

```
*Interp> calc "1 + true"
*** Exception: Non-exhaustive patterns in function binary
```

It is unsurprising, isn't it? We are setting ourselves on fire by adding an Integer to a Boolean value. It would be better if our interpreter could spot that and reject such programs without ever running them. This is where type-checking comes onto the stage.

3.1 Types Do Matter

For our small language, types are represented as:

```
data Type = TInt | TBool
```

This datatype accounts for the two possible types (i.e. Integer and Boolean) in the language.

So what do we mean by type-checking? In the tutorial, when we say type-checking, we refer to *static type-checking*. In other words, we verify the type safety of a program based on analysis of a program's source code. If a program passes the static type-checking, then the program is guaranteed to satisfy some set of type-safety properties.

For example, `1 + true` does not type-check because an addition expects two Integers. In the expression above, the second argument is not an Integer but a Boolean value.

In a language with variables, a type-checker needs to track all the types of bound variables. To do this, we can use a *type environment*:

```
type TEnv = [(String, Type)]
```

We are going to use the following type signature for the type-checker:

```
tcheck :: Exp -> TEnv -> Maybe Type
```

You may notice the similarity between this type signature and that of an environment-based interpreter, except for two differences:

1. While we use `Value` in the concrete interpreter, we now use `Type`;
2. The return type is now `Maybe Type`.

This is because 1) if we look on `tcheck` as an abstract interpreter, then types play the role of abstract values; 2) using `Maybe` makes the code more robust when tracking type-checking errors.

To begin with, we also need two auxiliary functions, just like `unary` and `binary`.

```
tunary  :: UnaryOp -> Type -> Maybe Type
tbinary :: BinaryOp -> Type -> Type -> Maybe Type
```

Question 3. Go to the file `TypeCheck.hs` and implement the two functions above.

3.2 Typing Rules for Expressions

To type-check expressions in our language, we need a set of typing rules. We will go over some examples, together with their associated inference rules. The inference rules are composed of typing judgements of the form:

$$\Gamma \vdash e : T$$

where Γ is a type environment, e is an expression, and T is the type being checked. We may read it as *the expression e has type T , under the type environment Γ* . Possible ways to construct an environment are formally described as:

$$\Gamma ::= \cdot \mid \Gamma, x : T$$

It means that an environment can be either the empty set ($\cdot$) or an environment extended with a variable x of type T . In short, an environment is a collection of pairs of variable names and their types.

Now, we are ready to look at the set of typing rules:

- **Typing Arithmetic Expressions:** Type-checking arithmetic expressions is fairly trivial. For example, to type-check an expression of the form:

`e1 + e2`

we proceed as follows:

1. Check whether the type of `e1` is `TInt`.
2. Check whether the type of `e2` is `TInt`.
3. If both types are `TInt` then return `TInt` as the result type (`Just TInt`); otherwise fail to type-check (`Nothing`).

You should be able to handle other cases in a similar way. The inference rule for the description above can be formalized as:

$$\frac{\Gamma \vdash e_1 : \mathbb{Z} \quad \Gamma \vdash e_2 : \mathbb{Z}}{\Gamma \vdash e_1 + e_2 : \mathbb{Z}} \text{ T+}$$

Similar rules can be derived for other binary operations. For unary operations, one can write, for example:

$$\frac{\Gamma \vdash e : \mathbb{Z}}{\Gamma \vdash -e : \mathbb{Z}} \text{ TNEG}$$

- **Typing Declare Expressions:** To type-check a declare expression of the form:

`var x = e; body`

we proceed as follows:

1. Type-check the expression `e`.
2. If `e` has a valid type, then type-check the `body` expression with the type environment extended with a pair `(x, typeof e)`.

For expression with unbound variables, you should return **Nothing**. In other words, the type-checker works only for *closed* expressions (i.e. with no free variables.) Inference rules for declarations and variables can be formalized as:

$$\frac{\Gamma \vdash e_1 : T_1 \quad \Gamma, x : T_1 \vdash e_2 : T_2}{\Gamma \vdash (\text{var } x = e_1; e_2) : T_2} \text{ TDECL}$$

$$\frac{x : T \in \Gamma}{\Gamma \vdash x : T} \text{ TVAR}$$

- **Typing If Expressions:** The only slightly tricky expression for type-checking is an `if` expression. The typing rule for an `if` expression of the form:

`if e1 e2; else e3`

is:

1. Check whether the type of `e1` is `TBool`.
2. Compute the type of `e2`.
3. Compute the type of `e3`.
4. Check whether the types of `e2` and `e3` are the same. If so then return that type; otherwise type-checking fails.

Note that if any sub-expression fails to type-check (`Nothing`), then the whole `if` expression will also fail to type-check.

The inference rule for `if` expressions can be formalized as:

$$\frac{\Gamma \vdash e_1 : \mathbb{B} \quad \Gamma \vdash e_2 : T \quad \Gamma \vdash e_3 : T}{\Gamma \vdash (\text{if } e_1 \text{ } e_2; \text{ else } e_3) : T} \text{ TIf}$$

Question 4. Implement the function `tcheck` according to the typing rules above.

Now consider the following expression:

`var x = 1; 2 + x`

Supposing that we want to derive its type, we may use the set of inference rules previously defined to formally type-check this expression, as in:

$$\frac{\Gamma \vdash 1 : \mathbb{Z} \quad \frac{\Gamma, x : \mathbb{Z} \vdash 2 : \mathbb{Z} \quad \frac{x : \mathbb{Z} \in (\Gamma, x : \mathbb{Z})}{\Gamma, x : \mathbb{Z} \vdash x : \mathbb{Z}} \text{ TVar}}{\Gamma, x : \mathbb{Z} \vdash 2 + x : \mathbb{Z}} \text{ T+}}{\Gamma \vdash \text{var } x = 1; 2 + x : \mathbb{Z}} \text{ TDECL}$$

This is very similar to what you have done previously for evaluation. We can see the correspondence between concrete interpretation and abstract interpretation.

3.3 Putting All Together

Now let's combine the type-checker with the `evaluate` function. Given an expression, we will first type-check it. If it passes, then we evaluate it; otherwise we raise an exception with an error message.

```
tcalc :: String -> Value
```

has the same type with `calc`. However, it has the ability to reject ill-typed programs!

Here are some examples:

```
*TypeCheck> tcalc "3 == 3"  
true
```

```
*TypeCheck> tcalc "if (3 == 4) true; else false"  
false
```

```
*TypeCheck> tcalc "var x = 3; x + true"  
*** Exception: You have a type error in your program!
```

Question 5. Complete the definition of `tcalc`.

With the help of type-checker, our evaluator can now successfully reject these ill-formed programs that get stuck in evaluation. The type system gives us a valuable guarantee: **well-typed programs cannot go wrong!**

THE UNIVERSITY OF HONG KONG

COMP3259: PRINCIPLES OF PROGRAMMING LANGUAGES

Tutorial 3

Tutor

Wan Qianrong `qywan@cs.hku.hk`

Instructor

Bruno Oliveira `bruno@cs.hku.hk`

25 February 2025

Table of Contents

1	Introduction	3
2	A Language with Local Variable Declarations	4
2.1	Variable Renaming	5
3	Implementing Interpreter with Substitution	7
3.1	Multiple Bindings in Variable Declarations	7
3.2	Interpreter, revised	7
4	Implementing Interpreter with Environments	9
4.1	Nested Declarations and Variable Name Conflicts	11
4.2	Multiple Bindings in Variable Declarations	12

1 Introduction

In this tutorial, we will review some of the concepts that we have learned so far in programming languages and add features to the simple interpreter we built last time. We will extend the interpreter to support local variables.

2 A Language with Local Variable Declarations

First, please open `Declare.hs` under the `src` folder.

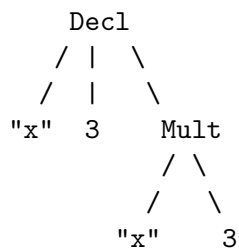
Recall that in the lecture, we were presented with the following definition for the abstract syntax of the arithmetic language we will be using (and later extending), similar to the following:

```
data Exp = Num Int
        | Add Exp Exp
        | Sub Exp Exp
        | Mult Exp Exp
        | Div Exp Exp
        | Var String           -- new
        | Decl String Exp Exp -- new
```

Compared to what we had last time, we add two new constructors `Var` and `Decl` to the datatype `Exp`, which allows our language to have local variables. For example, the expression:

```
e1 :: Exp
e1 = Decl "x" (Num 3) (Mult (Var "x") (Num 3))
```

has the abstract syntax tree as shown below:



Now you may be wondering what is the meaning of the `Decl` constructor, or how can we interpret it. Take `e1` for example, the `Decl` constructor introduces a new variable

named x , to which we assign value 3, and then we multiply it by 3. (Ask yourself what is the result of this expression.)

Another example of our extended language would be:

```
e2 :: Exp
e2 = Decl "x" (Add (Num 3) (Num 4)) (Var "x")
```

In this tutorial, we assume that the concrete syntax for variable declarations is JavaScript-like, of the form:

```
var name = exp; exp
```

So `e2` is effectively equivalent to:

```
var x = 3 + 4; x
```

Question 1. In the last tutorial and assignment, we knew how to build a pretty printer for our language. Now extend it to also support `Var` and `Decl` constructors. Here are some more examples for you to test your pretty printer:

```
e3 :: Exp
e3 = Add (Var "x") (Mult (Num 4) (Num 5))

e4 :: Exp
e4 = Decl "y" e3 (Div (Var "x") (Var "y"))
```

Here is the expected output in GHCi:

```
*Declare> e3
(x + (4 * 5))

*Declare> e4
var y = (x + (4 * 5)); (x / y)
```

2.1 Variable Renaming

In the lecture, we have seen how substitution works for an expression. Sometimes it is also useful to rename a variable in an expression. Here are some examples of renaming at work in GHCi:

```
*Declare> rename "x" "i" e3
(i + (4 * 5))
```

```
*Declare> rename "x" "i" e4
var y = (i + (4 * 5)); (i / y)
```

Note that if a variable is bound within the expression, then renaming should not rename the variable inside that scope (similar to what happens in substitution). For example:

```
*Declare> rename "x" "i" e1
var x = 3; (x * 3)
```

Here the variable x is bound by `var x = 3`. So in the expression $x \times 3$, renaming should not rename the variable x .

Question 2. Now you are asked to define the operation of renaming. Go to the file `Declare.hs` and look for the following definition:

```
rename :: String -> String -> Exp -> Exp
rename = error "TODO: Question 2"
```

The first argument of `rename` is the variable name to be renamed, and the second is the new name to replace with.

3 Implementing Interpreter with Substitution

3.1 Multiple Bindings in Variable Declarations

You have learned in the lectures how to implement an interpreter based on the idea of substitution in order to support local variable declarations. Another extension we can add is to allow multiple bindings in a variable binding expression. For example:

```
var x = 3, y = 9; x * y
```

declares two variables at once.

In order to allow for this change, we will have to again modify the abstract syntax of `Exp`. The new language with multiple bindings can be expressed by adding a `DeclareMulti` constructor to support a list of pairs of strings and expressions:

```
data Exp = ...
    | DeclareMulti [(String, Exp)] Exp
```

Question 3. Modify the definition of `Exp` as stated above. You should also modify all associated functions: `showExp` and `rename`.

Question 4. Please implement a `subst` function that supports substitution of a binding on the extended language with `DeclareMulti`.

3.2 Interpreter, revised

Once again, our interpreter needs to be augmented. This time it should be almost identical to what we have for the case of `Decl`. However, if a `DeclareMulti` expression has duplicate identifiers, your program must signal an error. It is legal for a nested `Decl` to reuse the same name. Two examples:

```
var x = 3, x = x + 2; x * 2      -- illegal
var x = 3; var x = x + 2; x * 2  -- legal
```

The meaning of a `DeclareMulti` expression is that all of the expressions associated with variables are evaluated in the outer environment before the body is evaluated. Then all the variables are simultaneously bound to the values that result from those evaluations. Then these bindings are added to the outer environment, creating a new environment that is used to evaluate the body. This means that the scope of all variables is the body of the `DeclareMulti` expression in which they are defined.

Note that a multiple-variable declaration is not the same as multiple nested declarations. For example:

```
var a = 3; var b = 8; var a = b, b = a; a + b      -- evaluates to 11
var a = 3; var b = 8; var a = b; var b = a; a + b  -- evaluates to 16
```

Question 5. Please implement an `evaluate` function based on the `subst` function.

You can assume that the inputs are valid programs and that your program may raise arbitrary errors when given invalid inputs.

In the file `Interp.hs`, there are several test cases that you can use to test your implementation.

Also note that the parser in the bundle is unable to parse expressions with multiple bindings. If you are adventurous enough, you can go modify the Happy grammar file to add support for multiple bindings.

4 Implementing Interpreter with Environments

There is another way to implement our interpreter and support local variables. We need something called *environments* to help track all the variables and their associated values in an expression. Recall that in the lecture, we have learned that a *binding* is an association of a variable with its value. We can represent bindings in Haskell as a pair. For example, `("x", 5)` means that variable x is associated with value 5. We can use the `type` keyword in Haskell to introduce a type alias as follows:

```
type Binding = (String, Int)
```

There can be multiple variables in a single expression. For example, our interpreter should be able to evaluate $2 \times x + y$ where $x = 3$ and $y = -2$. A collection of bindings is called an environment. In Haskell, we can represent an environment as a list of pairs:

```
type Env = [Binding]
```

In terms of the operational semantics, we use the following grammar to describe environments:

$$\Delta ::= \cdot \mid \Delta, x = n$$

It reads that an environment Δ is either empty (represented by $\cdot$), or non-empty (contains at least one binding). The form of inference rules now becomes $\Delta \vdash e \rightarrow n$. Our extended interpreter now should work as follows:

1. It starts with an empty environment.
2. When encountering a number, it should just return the number as it is.

$$\frac{}{\Delta \vdash n \rightarrow n} \text{EN}$$

3. When encountering arithmetic operations, it recursively evaluates each operand, as we did last time. Note that since arithmetic operations don't change environments, we keep the same environment for both premises and conclusions.

$$\frac{\Delta \vdash e_1 \rightarrow n_1 \quad \Delta \vdash e_2 \rightarrow n_2}{\Delta \vdash e_1 + e_2 \rightarrow n_1 + n_2} \text{E+} \qquad \frac{\Delta \vdash e_1 \rightarrow n_1 \quad \Delta \vdash e_2 \rightarrow n_2}{\Delta \vdash e_1 - e_2 \rightarrow n_1 - n_2} \text{E-}$$

$$\frac{\Delta \vdash e_1 \rightarrow n_1 \quad \Delta \vdash e_2 \rightarrow n_2}{\Delta \vdash e_1 \times e_2 \rightarrow n_1 \times n_2} \text{E}\times \qquad \frac{\Delta \vdash e_1 \rightarrow n_1 \quad \Delta \vdash e_2 \rightarrow n_2}{\Delta \vdash e_1 \div e_2 \rightarrow n_1 \div n_2} \text{E}\div$$

4. When encountering a variable, it should look up the variable in the environment. If the variable is found, return the value associated with it; if not, signal an error.

$$\frac{\Delta(x) = n}{\Delta \vdash x \rightarrow n} \text{EVAR}$$

Note that $\Delta(x) = n$ means “lookup x in Δ and bind the return value to n ”.

5. The most interesting part is variable declarations. To evaluate that, we first evaluate the expression associated with the variable using the old environment. Then we bind the variable with the value that results from the previous evaluation to form a binding, and augment the old environment with this binding. Finally we evaluate the body of the variable declaration in the new environment.

$$\frac{\Delta \vdash e_1 \rightarrow n_1 \quad \Delta, x = n_1 \vdash e_2 \rightarrow n_2}{\Delta \vdash (\mathbf{var} \ x = e_1; e_2) \rightarrow n_2} \text{EDECL}$$

To get yourself familiar with the new inference rules, here is an example of the derivation tree for **e1**:

$$\frac{\frac{\frac{\cdot \vdash 3 \rightarrow 3}{x = 3 \vdash x \rightarrow 3} \text{EVAR} \quad x = 3 \vdash 3 \rightarrow 3}{x = 3 \vdash x \times 3 \rightarrow 9} \text{E}\times}{\cdot \vdash (\mathbf{var} \ x = 3; x \times 3) \rightarrow 9} \text{EDECL}$$

Question 6. Create a derivation tree for the expression:

```
var a = 3; var b = 8; a + b
```

You may create a text file `derivation.txt` where you can write the answer to this question.

Question 7. In the `Interp.hs` file, fill in the definition of `evaluate2` as shown below.

```
evaluate2 :: Exp -> Int
evaluate2 e = eval e []  -- starts with an empty environment
  where
    eval :: Exp -> Env -> Int
    eval = error "TODO: Question 4"
```

To test it out, we need to extend our parser to support local variables. Fortunately, I have done that for you. The bundle contains a modified Happy grammar file `Parser.y`. You should be able to use our old friend `calc` to test your implementation of `evaluate2`. For example:

```
*Interp> calc "var x = 3; x + 4"
7
```

Or, if we have an unbound variable:

```
*Interp> calc "var x = 3; x + y"
*** Exception: Not found
```

Of course, the exception message could be more informative, for example, printing out the unbound variable name.

4.1 Nested Declarations and Variable Name Conflicts

The rules we defined above does not handle variable name conflicts for nested declarations. For example, deriving this expression:

```
var a = 3; var a = 4; a + 3
```

Our context Δ would contain two bindings for variable a , i.e. $a = 3$ and $a = 4$, which confuses the variable lookup process.

We now further clarify the behavior of our interpreter: the inner bindings will *shadow* the outer ones if they happen to have the same name. (You may want to check the behavior of the `lookup` function and properly encode your context.)

Here are some test cases:

```
var a = 3; var b = 8; a + b  -- evaluates to 11
var a = 3; var a = 4; a + 3  -- evaluates to 7
```

4.2 Multiple Bindings in Variable Declarations

Question 8. Please first write the inference rule for multiple-variable declarations (you may use any notation for lists as you like), and then modify the definition of `evaluate2` to cover the case of `DeclareMulti`.

THE UNIVERSITY OF HONG KONG

COMP3259: PRINCIPLES OF PROGRAMMING LANGUAGES

Tutorial 2

Tutor

Wan Qianrong `qywan@cs.hku.hk`

Instructor

Bruno Oliveira `bruno@cs.hku.hk`

18 February 2025

Table of Contents

1	Introduction	3
2	A Language of Arithmetic	4
2.1	Abstract Syntax Tree	4
2.2	Interpreter and Parser	5
2.3	Pretty Printing and Type Classes	7
2.4	Error Handling	9
2.5	Inference Rules	11

1 Introduction

The goal of this tutorial is to build a simple interpreter that supports basic arithmetic calculations with error handling. We will use the concepts learned so far in the lectures and put them in good use.

2 A Language of Arithmetic

First, please open `Declare.hs` under the `src` folder.

Recall that in the lecture, we were presented with the following definition for the abstract syntax of the arithmetic language we will be using (and later extending), similar to the following:

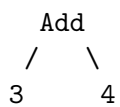
```
data Exp = Num Int
         | Add Exp Exp
         | Sub Exp Exp
         | Mult Exp Exp
         | Div Exp Exp
         | Power Exp Exp
         | Neg Exp
deriving Show
```

2.1 Abstract Syntax Tree

For any abstract syntax expression of type `Exp`, we can draw a corresponding abstract syntax tree. For example, the expression:

```
e1 :: Exp
e1 = Add (Num 3) (Num 4)
```

has the abstract syntax tree as follows:

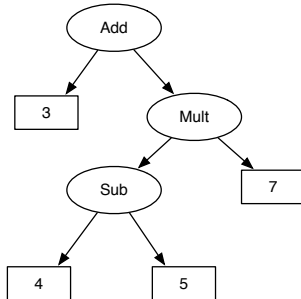


For non-recursive constructors (such as `Num`), we omit the constructor name. For recursive constructors, we use the name of the constructor to create a tree node and represent each argument of the constructor as a sub-tree.

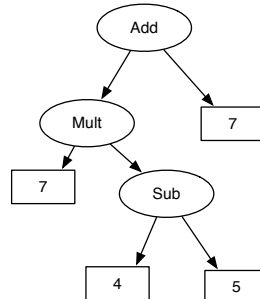
Question 1. Which one is the correct abstract syntax tree for the following expression?

```
e2 :: Exp
```

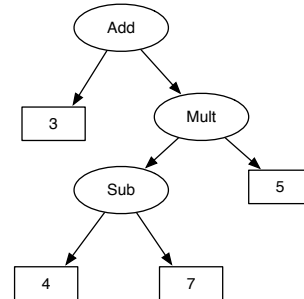
```
e2 = Add (Num 3) (Mult (Sub (Num 4) (Num 5)) (Num 7))
```



Option A



Option B



Option C

2.2 Interpreter and Parser

Now, please open `Interp.hs` under the `src` folder.

In the lecture, we have been shown the definition of a interpreter for arithmetic expressions as follows:

```
evaluate :: Exp -> Int
evaluate (Num n)      = n
evaluate (Add a b)    = evaluate a + evaluate b
evaluate (Sub a b)    = evaluate a - evaluate b
evaluate (Mult a b)   = evaluate a * evaluate b
evaluate (Div a b)    = evaluate a `div` evaluate b
```

Question 2. Following the logic of the above `evaluate` function, extend it to support negation and exponentiation operations in the file `Interp.hs`. (You can use the `^` operator like `2 ^ 3` to calculate “2 to the power of 3” and the `negate` function like `negate 3` in Haskell. Have a try in GHCi!)

It is cumbersome to have to write long Exps like `Add (Num 3) (Num 4)` and then feed it to `evaluate`. We humans like to write the string `"3 + 4"`, and then let the interpreter tell us the answer. To achieve this, we need to find a way to tell the computer what those symbols mean.

This is where parsing comes onto stage. Parsing is a process that converts between *concrete syntax* and *abstract syntax*. For example, the expression $1 + 8 \times 2$ would be parsed to:

```
Add (Num 1) (Mult (Num 8) (Num 2))
```

Writing parsers can sometimes be unbearably tedious. Good news is that there are lots of *parser generator* tools that can help automate the work. In this tutorial, we will use [Happy](#), a parser generator for Haskell. Let's install [Alex](#) (the Lexer) and Happy by running the following command:

```
stack install alex happy
```

A parser needs a set of rules called *grammar* that specify valid combinations of tokens to form expressions of a language. Fortunately, for our tiny language, the grammar is as simple as the following:

```
AExpr    ::= AExpr '+' Term
           | AExpr '-' Term
           | Term

Term      ::= Term '*' Factor
           | Term '/' Factor
           | Factor

Factor    ::= Expon '^' Factor
           | Expon

Expon     ::= '-' Expon
           | Primary

Primary   ::= digits
           | '(' AExpr ')'
```

Note that it's not the only way to write the grammar. You may merge some definitions and get the same result.

In the code bundle, we have included a Happy grammar file named `Parser.y` in `src` directory. Happy will take this grammar file and generate a Haskell module that provides a parser called `parseExpr`. This can be done as follows:

```
> happy Parser.y
```

Now you should see a Haskell file named `Parser.hs` in the `src` directory. To use it, just load the file on GHCi:

```
> ghci Parser.hs
```

You can type in any arithmetic expression and feed it to `parseExpr`. For example:

```
*Parser> parseExpr "(1 + 2) * 3 / 4"
Div (Mult (Add (Num 1) (Num 2)) (Num 3)) (Num 4)
```

It is recommended that you should try some expressions involving negation and exponentiation operations to see if your definition of `Exp` is working correctly.

Putting all together, now you should have a working interpreter that could do simple arithmetic calculations. Let's do it!

Question 3. In the file `Interp.hs`, write a function called `calc`, which, given an arithmetic expression, evaluates it and returns the result. The type signature of `calc` is as follows:

```
calc :: String -> Int
```

(Hint: Think about what helper functions are at your service and how to compose them.)

2.3 Pretty Printing and Type Classes

Congratulations! At this point, you have created a fully working calculator all by yourself. If you like, you can extend it to have more operations like finding the square root of a given number or calculating factorials.

One nice feature is to have the interpreter printing the textual representation of the abstract syntax. The existing interpreter we have now does support a function called `show` that can be used to print the abstract syntax of an expression. This function is automatically generated by Haskell using `deriving Show` in the declaration of the datatype `Exp`. For example, under the `src` folder, trying:

```
> ghci Declare.hs
.....
*Declare> e2
Add (Num 3) (Mult (Sub (Num 4) (Num 5)) (Num 7))
```

on the console will automatically invoke that `show` function and print the textual representation of `e2`.

However, as you also notice, the textual representation generated is not very pretty because it is based on the abstract syntax.

Now you are asked to define a variant of the `show` function that is prettier and prints output based on the concrete syntax instead. For example, the expression `e2` would be printed as follows:

```
*Declare> e2
3 + (4 - 5) * 7
```

Question 4. Implement the variant of the `show` function.

To define this function, do the following:

1. Go to the file `Declare.hs` and comment out deriving `Show` from the datatype definition of `Exp`.
2. Create an instance of the type class `Show` for `Exp` by using the following code:

```
instance Show Exp where
    show = showExp
```

This code allows you to define your own version of the `show` function to print values of type `Exp`.

3. In the same file, look for the following definition:

```
showExp :: Exp -> String
showExp = error "TODO: Question 4"
```

You need to replace this definition of `showExp` by a definition that creates a string representation of the input expression. Once this definition is done you should be able to get the same result (not exactly) as shown before.

For simplicity, put brackets around every expression using `Add`, `Sub`, `Mult`, `Div`, and other operators, and also add spaces around each operator. The result will not be as pretty as it could be, but we will avoid having to figure out when to omit brackets based on the precedence and associativity of the operators.

Another example for you to try on:

```
e3 :: Exp
e3 = Sub (Div (Add (Num 1) (Num 2)) (Num 3)) (Mult (Sub (Num 5) (Num 6)) (Num 8))
```

Here is the expected output in GHCi:

```
*Declare> e3
(((1 + 2) / 3) - ((5 - 6) * 8))
```

2.4 Error Handling

Let's go to the `Interp.hs` again.

Though our tiny interpreter is good at doing arithmetic calculations, it fails to provide us useful information when it's malfunctioning. For example, when typing in:

```
*Interp> calc "(1 + 2) / (3 - 3)"
*** Exception: divide by zero
```

Oops, a *divide-by-zero* exception! This kind of error message is fine when the expression is small enough. We can immediately spot that, in the divisor, $3 - 3$ evaluates to 0. But when the expression itself becomes larger and larger, we would like to know more than just that.

If you remember, we've seen one way of dealing with errors: calling `error` function, which terminates the program immediately. In functional programming, if a function is not defined for all values of arguments (e.g., raises an exception, loops forever), we say that this function is *partial*. You can guess the opposite ones are called *total* functions: functions defined for all values of their arguments. In Haskell, we would like to have all functions being *total*. How to do that? The trick is to have functions returning a different type, a type that encodes both failure and success of a computation!

Let us define a datatype that represents values of two possibilities:

```
data Either a b = Left a | Right b
```

That is to say, a value of type `Either a b` is either `Left a` or `Right b`. By convention, the `Left` constructor is used to hold an error value, and the `Right` constructor is used to hold a correct ("right" also means "correct") value.

Here is how we can use `Either` to make a safe version of `head` function (compared with the one implemented by `Maybe` datatype). Remember that when `head` is applied to an empty list, it throws an exception.

```
safeHead :: [a] -> Either String a
safeHead []      = Left "can't access the head of an empty list"
safeHead (x:_)   = Right x
```

Now let's try to apply `safeHead` to an empty list and see what happens:

```
*Interp> safeHead []
Left "can't access the head of an empty list"
```

To incorporate `Either` into our interpreter, we need to change the return type of `evaluate` as follows:

```
evaluate2 :: Exp -> Either String Int
```

Question 5. In the file `Interp.hs`, complete the definition of `evaluate2` with error handling. Also create a function `calc2` that is similar to `calc` but uses `evaluate2`.

Here is an example of evaluating `Add a b` (you should follow the example and complete others):

```
evaluate2 (Add a b) =
  case evaluate2 a of
    Left msg -> Left msg
    Right a' ->
      case evaluate2 b of
        Left msg -> Left msg
        Right b' -> Right (a' + b')
```

Notice that the burden now is on the caller to pattern match the result of any such call. It will either continue with the successful result or handle the failure (in our case, just propagate the error message).

Specifically, there are two possible errors when doing integer calculations:

1. When divided by zero (e.g., $2 \div 0$).
2. To the power of a negative number (e.g., 2^{-3}).

You can issue whatever error messages you like, just make sure the above two situations have different error messages, and provide some information about where the error arises. For simplicity, just print out the problematic expression using the pretty printer you just made.

Here are examples of possible error messages:

```
*Interp> calc "(1 + 2) / (3 - 3)"
Left "Divided by zero: (3 - 3)"
```

```
*Interp> calc "(2 - 3) ^ (2 - 4)"
Left "To the power of a negative number: (2 - 4)"
```

2.5 Inference Rules

In the lecture, we have also seen how to define operational semantics through inference rules, in which we write $e \rightarrow n$ to indicate “e evaluates to n”. A judgement like this is thought to be true only if the pair (e, n) is in the relation defined by our inference rules, which means there exists a derivation with $e \rightarrow n$ at the root.

$$\begin{array}{c}
 \frac{}{3 \rightarrow 3} \text{EN} \quad \frac{\frac{}{4 \rightarrow 4} \text{EN} \quad \frac{}{5 \rightarrow 5} \text{EN}}{4 - 5 \rightarrow -1} \text{E-} \quad \frac{}{7 \rightarrow 7} \text{EN} \\
 \hline
 \frac{}{(4 - 5) \times 7 \rightarrow -7} \text{E}\times \\
 \hline
 \frac{}{3 + (4 - 5) \times 7 \rightarrow -4} \text{E+}
 \end{array}$$

An example of a derivation is given above. The derivation shows how we can assign a meaning (i.e. result) to e_2 . Typically, we start writing each derivation bottom-up (i.e. starting with the original expression at the root) and leave the right-hand side of the arrow blank. The right-hand side will be the semantic result for that expression, which will be filled in a top-down manner. At this point, there is only one rule we can apply, by looking at its conclusion: $E+$. Thus, we can write the line above the expression, and split the derivation into two, corresponding to both sub-expressions. Again, we do not fill the right-hand sides, yet. Informally, when applying these rules, we are saying: *assuming that I can derive the semantics for some e_1 and e_2 , then I can also derive that for $e_1 + e_2$.*

We repeat this process until we reach the leaves of the derivation, that is, no more hypothesis is needed. Now, we can easily fill in the missing right-hand sides, by starting at the leaves, and going down the derivation tree. By looking at each rule, we can see how to combine each result in the premises in order to fill the result in the conclusions. For instance, the top-most $E-$ combines both results of its premises 4 and 5 by subtraction, denoted as the result -1 . We can keep applying this process, until we reach our original expression. Notice that, when executing our `evaluate` functions, Haskell will perform this same derivation for us, even though we only get to see the final result.

Question 6. Write the derivation for the expression e_3 .

Question 7. Based on our inference rules, for an expression, can there be more than one derivation? Try to explain why.

As we can see, there is a correspondence between the inference rules and our evaluator. When writing a derivation, we are going through the recursive calls of our `evaluate` function.

Recall that in the lecture, we have the rule $E+$:

$$\frac{e_1 \rightarrow n_1 \quad e_2 \rightarrow n_2}{e_1 + e_2 \rightarrow n_1 + n_2} E+$$

The following rule is another equivalent version. We assume the knowledge of arithmetic equality, so you don't need to write any reasoning for a correct arithmetic equation in the form of $n = n_1 + n_2$ (e.g. $5 = 2 + 3$).

$$\frac{e_1 \rightarrow n_1 \quad e_2 \rightarrow n_2 \quad n = n_1 + n_2}{e_1 + e_2 \rightarrow n} E+',$$

Question 8. Assuming we replace the inference rule $E+$ by $E+'$, please write the derivation for the e_2 .

Question 9. If both $E+$ and $E+'$ are included in our operational semantics, is the set of inference rules syntax-directed still?

THE UNIVERSITY OF HONG KONG

COMP3259: PRINCIPLES OF PROGRAMMING LANGUAGES

Tutorial 1

Tutor

Wan Qianrong `qywan@cs.hku.hk`

Instructor

Bruno Oliveira `bruno@cs.hku.hk`

24 January & 11 February 2025

Table of Contents

1	Introduction	3
2	Setting up the Development Environment	4
2.1	Install the Haskell Toolchain	4
2.2	Setting up editors	5
3	Basic Steps in Haskell	6
3.1	Preamble	6
3.2	First Definitions in Haskell	7
3.3	Pattern Matching	7
3.4	Recursion	9
3.5	User-Defined Datatypes	11
3.6	Parametric Polymorphism and Type Inference	13

1 Introduction

The goal of this tutorial is to get the students familiar with Haskell programming. Students are encouraged to bring their own laptops to go through the installation process of Haskell and corresponding editors, especially if they haven't tried to install Haskell before or if they had problems with the installation.

2 Setting up the Development Environment

2.1 Install the Haskell Toolchain

If you have not installed Haskell yet, you can try to install it now. If you have problems we can try to help you.

Note: For Windows users, please setup [WSL](#) first.

The easiest way to install Haskell is via [GHCup](#). e.g. For Ubuntu / Ubuntu-based WSL:

```
sudo apt update \  
&& sudo apt install build-essential curl libffi-dev libffi8 libgmp-dev \  
libgmp10 libncurses-dev pkg-config -y \  
curl --proto '=https' --tlsv1.2 -sSf https://get-ghcup.haskell.org | sh
```

And follow the installation steps. If you are seeing something like:

```
Detected bash shell on your system... Do you want ghcup to automatically  
add the required PATH variable to "<bashrc_location>"?
```

```
[P] Yes, prepend [A] Yes, append [N] No [?] Help (default is "P").
```

Select “Yes, prepend” by keep the default setting. Otherwise you may need to add it to your shell configuration manually.

The actual installation instructions may vary depending on your system environment. Check <https://www.haskell.org/ghcup/install/#system-requirements> for more information.

You can now run `ghcup tui`. With GHCup you get a comprehensive development environment for Haskell:

1. **GHC**: the *de facto* standard compiler for Haskell.
2. **Cabal**: a system for building and packaging Haskell libraries and programs.
3. **Stack**: an alternative to cabal-install. Stack uses the Cabal library but with a curated version of the Hackage repository called Stackage. Stack even downloads GHC for you and keeps it in an isolated location.

4. **HLS**: the official Haskell IDE support via the Language Server Protocol.

Here we set the default GHC version to **9.6.6** to match stackage snapshot lts-22.43 used in subsequent code (or you can skip it for now and let stack handle it later.)

In our tutorials and assignments, we mainly use **Stack** to build a Haskell project.

2.2 Setting up editors

- Using VSCode (recommended):

Note: For WSL users, please follow [this guide](#) to setup VSCode with WSL2.

Install the [Haskell](#) extension and your development environment should be set by now. It's also recommended to set `haskell.manageHLS` to `GHCup` in preferences. You can save the [workspace](#) for future use.

- Using other editors:
 - **Emacs**: use [Haskell mode](#)
 - **Helix**: out-of-the-box LSP support (but you need to install *HLS* to get Haskell support)
 - **(neo)vim**: install *HLS* and follow [this guide](#)

You may search online for detailed configuration steps for your chosen editor.

3 Basic Steps in Haskell

In this tutorial, you are going to write your first (maybe not?) Haskell code.

3.1 Preamble

Download the project `tutorial1.zip` from Moodle and unzip it. Inside it is the standard structure of a Haskell project. We don't care about `tutorial1.cabal`, `stack.yaml`, `package.yaml`, and so on for the moment. What we are really interested in is all inside the `src` directory.

Before continuing, let's try to build this project first (yes, we can build it even we haven't written a single bit!). Now open the terminal, go to the project root and run `stack build`. If everything is OK as it should be, you will see the following message, though not exactly the same:

- tutorial1-0.1.0.0: unregistering ...
- tutorial1-0.1.0.0: configure
- Configuring tutorial1-0.1.0.0...
- tutorial1-0.1.0.0: build
- ...
- Registering tutorial1-0.1.0.0...

If you run `stack build` again, you will see nothing showing up. This is because the project hasn't changed from the last build. Stack is smart enough to detect it and not actually build it again.

`stack build` is one of the most important Stack commands that we use throughout the development of Haskell projects. We will see another important command shortly when we talk about testing.

Now we are ready to write some code! Go inside the `src` directory, and open the file `Tutorial1.hs` using your favourite editor. In the first line, you will see the module header as follows:

```
module Tutorial1 where
```

3.2 First Definitions in Haskell

In the lecture we have seen some basic Haskell definitions. For starters, let's take a look at a function that computes the absolute value of a number:

```
absolute :: Int -> Int
absolute x = if x < 0 then -x else x
```

Everything is an Expression in Haskell. Generally speaking, Haskell definitions have the following basic form:

- $name\ arg_1 \ \cdots \ arg_n = expression$

Note that in the right side (the body of the definition) what you have is an expression. This is different from a conventional imperative language, where the body of a definition is usually a statement. In fact, there are no statements in Haskell, and in particular, the `if` construct in `absolute` is also an expression.

Question 1. Are both of the following Haskell programs valid?

```
nested_if x = if absolute x <= 10
               then x
               else error "Only numbers between [-10,10] allowed"

nested_if' x = if (if x < 0 then -x else x) <= 10
                  then x
                  else error "Only numbers between [-10,10] allowed"
```

Once you have thought about it, you can try these definitions on your Haskell file and see if they are accepted or not.

Question 2. Can you have nested `if` statements in Java, C or C++? For example, would this be valid in Java?

```
int f(int x) {
    if ((if (x < 0) -x; else x) > 10) return x; else return 0;
}
```

3.3 Pattern Matching

One feature that many functional languages support is pattern matching. Pattern matching plays a role similar to conditional expressions, allowing us to create definitions de-

pending on whether the input matches a certain pattern. For example, the function `hd` (to be read as “head”) given a list returns the first element of the list:

```
hd :: [a] -> a
hd [] = error "cannot take the head of an empty list!"
hd (x:xs) = x
```

In this definition, the pattern `[]` denotes an empty list, whereas the pattern `(x:xs)` denotes a list where the first element (or the head) is `x` and the remainder of the list is `xs`. Note that instead of a single clause in the definition, there are now two clauses for two cases.

Detour: Test Test Test!

The comments in the source file are a bit, shall I say, verbose? For example, in the following code snippet, the comments tell the users how `tl` should work for some sample inputs. The best thing is, they also serve as unit tests for the function so that the compiler will help run those tests for us, and should some function fail the tests, it would let us know!

```
-- | The tail of a list
--
-- Examples:
--
-- >>> tl [1,2,3]
-- [2,3]
--
-- >>> tl [1]
-- []
--
tl :: [a] -> [a]
tl = error "TODO: question 4"
```

Now go to the terminal, run `stack test`. You should see the following

```
...
### Failure in .../Tutorial1.hs:22: expression `tl [1,2,3]'
expected: [2,3]
but got: *** Exception: TODO: question 4
...
Examples: 14 Tried: 10 Errors: 0 Failures: 10
```


It reads that we have 14 examples, the compiler tried 10 of them, and they *all* failed to pass the tests! (No surprise, we haven't started yet.) So here we go, let's make it right!

Question 3. Define a `tl` function that given a list, drops the first element and returns the remainder of the list. That is, the function should behave as follows for the sample inputs:

```
Tutorial1> tl [1,2,3]
[2,3]
Tutorial1> tl ['a','b']
['b']
```

Now if you run `stack test` again, chances are that you should see some improvements.

More Pattern Matching. Pattern matching can be used with different types. For example, here are two definitions with pattern matching on tuples and integers:

```
first :: (a,b) -> a
first (x,y) = x

isZero :: Int -> Bool
isZero 0 = True
isZero n = False
```

3.4 Recursion

In functional languages, mutable state is generally avoided and in the case of Haskell (which is purely functional), it is actually forbidden. So, how can we write many of the programs we are used to? In particular, how can we write programs that in a language like C would normally be written with some mutable state and some type of loop? For example:

```
int sum_array(int a[], int num_elements) {
    int sum = 0;
    for (int i = 0; i < num_elements; i++) {
        sum = sum + a[i];
    }
    return sum;
}
```

The answer is to use **recursive** functions. For example here is how to write a function that sums a list of integers:

```
sumList :: [Int] -> Int
sumList [] = 0
sumList (x:xs) = x + sumList xs
```

Question 4. The factorial sequence is (starting from 1):

1, 2, 6, 24, 120, ...

Implement a recursive function `factorial :: Int -> Int` that given a number returns the corresponding number in the factorial sequence.

Question 5. The Fibonacci sequence (index starting from 0) is:

0, 1, 1, 2, 3, 5, 8, 13, 21, ...

Implement a recursive function `fibonacci :: Int -> Int` that given a number returns the corresponding number in the sequence.

Question 6. Write a function:

```
mapList :: (a -> b) -> [a] -> [b]
```

that applies the function of type `a -> b` to every element of a list. For example:

```
Tutorial1> mapList absolute [4,-5,9,-7]
[4,5,9,7]
```

Question 7. Write a function that given a list of characters returns a list with the corresponding ASCII number of the character. Note that in Haskell, the function `ord`:

```
ord :: Char -> Int
```

gives you the ASCII number of a character. To use it we need to add the following just after the module declaration (which is already been added for you):

```
import Data.Char (ord)
```

Question 8. Write a function `filterList` that given a predicate and a list returns another list with only the elements that satisfy the predicate.

```
filterList :: (a -> Bool) -> [a] -> [a]
```

For example, the following filters all the even numbers in a list (`even` is a built-in Haskell function):

```
Tutorial1> filterList even [1,2,3,4,5]
[2,4]
```

Question 9. In the `Prelude` library, there is a function `zip`:

```
zip :: [a] -> [b] -> [(a,b)]
```

that given two lists pairs together the elements in the same positions. For example:

```
Tutorial1 > zip [1,2,3] ['a','b','c']
[(1,'a'),(2,'b'),(3,'c')]
```

Write a definition `zipList` that implements the `zip` function.

Question 10. Define a function `zipSum`:

```
zipSum :: [Int] -> [Int] -> [Int]
```

that sums the elements of two lists at the same positions.

(Suggestion: You can define this function recursively, but a simpler solution can be found by combining some of the previous functions.)

3.5 User-Defined Datatypes

You may be wondering how we can make our own types out of existing ones in Haskell. The short answer is: use datatype declaration. In fact, many built-in types in Haskell like `Boolean` and lists are actually defined using datatypes. For example, the built-in type `Bool` is defined in the standard library as follows:

```
data Bool = True | False
```

The keyword `data` introduces a new datatype. The part before `=` defines a new type, which in our case is `Bool`. The parts after `=` are data constructors. They tell us what values this type can have. The `|` symbol can be read as “or”. So in the above example, `Bool` type can only have two values, i.e. either `True` or `False`. Note that both the type name and the data constructors must begin with capital letters.

A data constructor can take some parameters and produce a new value. In the `Bool` example, this is not the case since `True` and `False` take no parameters. Let us define a list datatype to illustrate the use of type parameters:

```
data MyList a = Nil | Cons a (MyList a)
```

Here `a` is a type parameter, which can be used in the data constructors. Depending on what kind of values we want our own version of list type to hold, we can have different types of lists. Once the type parameter `a` is fixed to some concrete type, we can only have elements of that type in the list. For example, if we pass `Int` as the type parameter to `MyList`, we end up having a type `MyList Int`, and the two data constructors `Nil` and `Cons` have the following types correspondingly:

```
Nil :: MyList Int
Cons :: Int -> MyList Int
```

Note that the above types tell us that we can only make a list of integers, not a list of characters or some other type. So

```
Cons 1 (Cons 2 (Cons 3 Nil))
```

creates a list of integers, whose elements are 1, 2 and 3.

A `Maybe` datatype is defined as

```
data Maybe a = Nothing | Just a
```

Quite look like our `MyList` example. You can think of this datatype as representing computations which might “go wrong” by not returning a value, or “succeeded” in producing some value.

The `Maybe` datatype provides a way to make a safe version of a partial function. A partial function is simply a function that is undefined for some argument. For example, the built-in function `hd` is partial, because if we pass an empty list `[]` to it, it raises an exception, and makes our program crash. A safe version of `hd` won’t do that:

```
safeHead :: [a] -> Maybe a
```

As can be seen from the type, it returns a value of `Maybe a`. So if we pass an empty list to `safeHead`, it return `Nothing`, otherwise, it wraps the head element of the list inside `Just`. Some examples below:

```
Tutorial1> safeHead []
Nothing
```

```
Tutorial1> safeHead [1,2,3]
Just 1
```

Question 11. Define the function `safeHead`.

Question 12. Define a `catMaybes` function, which takes a list of `Maybes` and returns a list of all the `Just` values. For example:

```
Tutorial1> catMaybes [Just 1, Nothing, Just 2]
[1,2]
```

3.6 Parametric Polymorphism and Type Inference

We have seen that Haskell supports definitions with a type signature or without. When a definition does not have a signature, Haskell infers one and it is still able to check whether some type errors exist or not. For example, for the definition:

```
newline s = s ++ "\n"
```

Haskell is able to infer the type: `String -> String`.

(**Note:** In Haskell strings are represented as lists of characters, i.e. `[Char]`. The operator `++` is a built-in function in Haskell that concatenates two lists.) For certain definitions, it appears as if there was not enough information to infer a type. For example, consider the definition:

```
id x = x
```

This is the definition of the identity function: the function that given some argument returns that argument unmodified. This is a perfectly valid definition, but what type should it have?

The answer is:

```
id :: a -> a
```

The function `id` is a **(parametrically) polymorphic** function. **Polymorphism** means that the definition works for multiple types; and this type of polymorphism is called **parametric** because it results from parametrizing over a type. In the type signature, `a` is a type parameter. In other words, it is a variable that can be replaced by some valid

type (e.g. `Int`, `Char`, or `String`). Indeed, the `id` function can be applied to any type of values. Try the following in GHCi (run `stack ghci` in the terminal), and see what is the resulting type:

```
Tutorial1> id 3
Tutorial1> id 'c'
Tutorial1> id id
-- you may try instead ":t id id"
```

Question 13. Have you seen this form of polymorphism in other languages? Perhaps under a different name?

Monads and Imperative Programming

COMP3259
Principles of Programming Languages

Imperative Programming

Resources

Lecture covers:

- Chapter 6 of “Anatomy of Programming Languages”

<http://www.cs.utexas.edu/~wcook/anatomy/anatomy.htm>

Interpreter so far

- Our current JavaScript-like interpreter already supports a number of features:
- basic expressions (arithmetic & conditionals)
- variable declarations
- function definitions & first-class functions
- recursion
- Error Handling

Imperative Programming

- Imperative Programming is characterized by
 - mutable variables
 - a programming style that uses mutation and iteration (loops) to implement algorithms

```
x = 1;  
for (i = 2; i <= 5; i = i + 1) {  
    x = x * i;  
}
```

Imperative Languages

- Most languages support Imperative Programming
 - Classic Languages: Algol, C, Pascal
 - Imperative OO Languages: C++, Java, C#
 - Scripting Languages: Javascript, Python, Ruby
 - Functional Languages: ML, OCaml, Scala

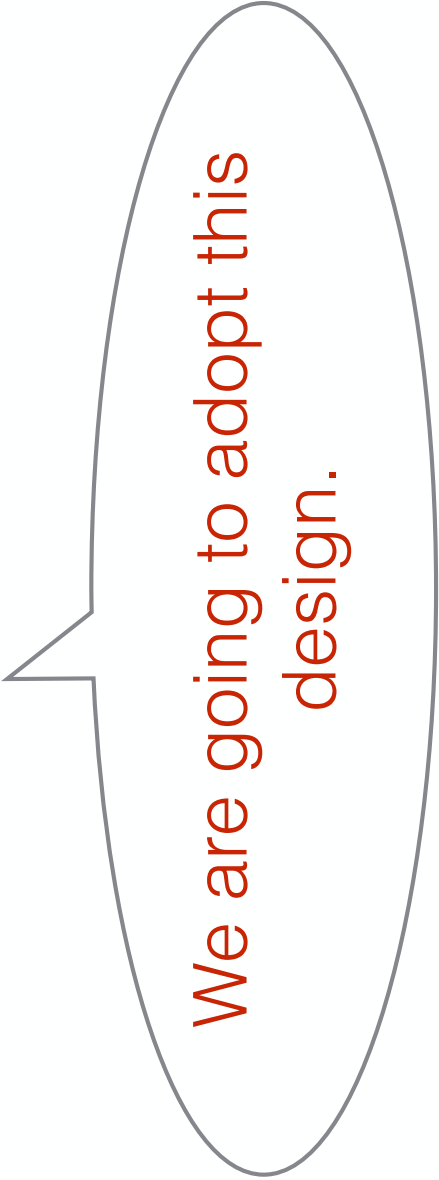
Mutable State

- Different designs are possible:
- **Mutation by default**: Make everything mutable by default
 - Adopted by most languages: Java, C, C#, Ruby
 - ...
- **Explicit Mutation**: Make mutation explicit
 - Some languages, such as ML, adopt this approach

Explicit Mutation

- In a design with explicit mutation, variables need to be declared to be mutable:

```
x = Mutable(1);  
for (i = Mutable(2); @i <= 5; i := @i + 1) {  
    x := @x * @i;  
}
```



We are going to adopt this design.

Addresses

- We are going to use addresses to model mutation:

Operation	Meaning
<code>Mutable (e)</code>	Creates a mutable cell with an initial value given by <code>e</code>
<code>@a</code>	Accesses the contents stored at address <code>a</code>
<code>a := e</code>	Updates the contents at address <code>a</code> to be the value of expression <code>e</code>

Implementing Mutable State

Addresses

- Addresses are a new kind of value

```
data Value = IntV  Int
           | BoolV Bool
           | ClosureV String Exp Env
           | AddressV Int      -- new
deriving (Eq, Show)
```

Memory

- The current value of all mutable cells used in a program is called memory.
- Logically, a memory is a map or association of addresses to values.
- Since addresses are integers, one natural representation is as a list or array of values

type **Memory** = [**Value**]

Memory Operations

- Two important operations on Memory:
- Access: accesses the value stored in the memory address.

```
access i mem = mem !! i
```

- Update: updates the value stored in the memory address

```
update :: Int -> Value -> Memory -> Memory
update addr val mem =
  let (before, _after) = splitAt addr mem in
  before ++ [val] ++ after
```

Monads Again!

Stateful Monad

Before state

- Evaluating binary operators before state:

*evaluate (Binary op a b) env =
binary op (evaluate a env) (evaluate b env)*

After State

- Evaluating binary operators after state:

```
evaluate (Binary op a b) env mem =  
  let (av, mem') = evaluate a env mem in  
    let (bv, mem'') = evaluate b env mem' in  
      (binary op av bv, mem'')
```



Threading
the state around
makes code messy!
Can we do better?

Spotting the pattern

- Evaluating binary operators after errors:

```
evaluate (Binary op a b) env mem =  
  let (av, mem') = evaluate a env mem in  
  let (bv, mem'') = evaluate b env mem' in  
  (binary op av bv, mem'')
```

There seems to be a
repeating pattern here.

Monads to the Rescue!

We can capture **the essence of threading the state** around using a Monad!

```
data Stateful t =  
  ST (Memory -> (t, Memory))  
  
instance Monad Stateful where  
  return val = ST (\m -> (val, m))  
  (ST c) >>= f = ST (\m ->  
    let (val, m') = c m  
    ST f' = f val  
    in f' m')
```

Rewriting code

Using the do-notation for Monads we can rewrite the code as follows:

```
evaluate (Binary op a b) env = do  
  av <- evaluate a env  
  bv <- evaluate b env  
  return (binary op av bv)
```

Error Checking and Monads

COMP3259
Principles of Programming Languages

Error Checking

Resources

Lecture covers:

- Chapter 6 of “Anatomy of Programming Languages”

<http://www.cs.utexas.edu/~wcook/anatomy/anatomy.htm>

Interpreter so far

- Our current JavaScript-like interpreter already supports a number of features:
 - basic expressions (arithmetic & conditionals)
 - variable declarations
 - function definitions & first-class functions
 - recursion

Interpreter so far

- This allows us to write some programs, such as:

```
var fact = function(n) {  
    if (n == 0) 1; else n * fact(n-1)  
};  
var x = 10;  
fact(x)
```

JavaScript Interpreter so far

- However, what happens if we write?

```
var fact = function(n) {  
    if (n == 0) 1; else n * fact(n-1)  
};  
fact(10)
```


JavaScript Interpreter so far

- However, what happens if we write?

```
var fact = function(n) {  
    if (n == 0) 1; else n *  
    fatn(n-1)  
};  
fact(10)
```

If we try in ghci:

```
*Main> execute fact2  
*** Exception: Maybe.fromJust: Nothing
```

JavaScript Interpreter so far

```
*Main> execute fact2  
*** Exception: Maybe.fromJust: Nothing
```



Not very helpful or
informative as an error
message!

Errors

- Errors are an important aspect of computation.
- They are typically a pervasive feature of a language, because they affect the way every expression is evaluated. For example, consider the expression:

$a + b$

- If a or b raise errors then we need to deal with this possibility.

Errors

- Because errors are so pervasive they are a notorious problem in programming and programming languages.
- When coding in C the convention is to check the return codes of all system calls.
- However this is often not done.
- Java's exception handling mechanism provides a more robust way to deal with errors.

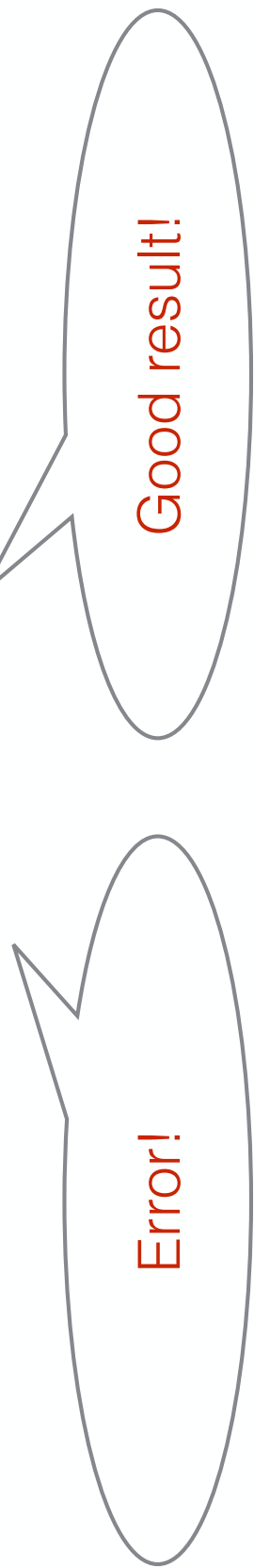
Errors

- During the course so far we have already encountered **some ways to deal with errors**:
- Our type-checkers have been developed with **Maybe to deal with type-errors**
- We have implemented variants of the JavaScript interpreter with a simple **Exception mechanism**.

Maybe

- The Maybe datatype provides a useful mechanism to deal with errors:

data Maybe a = Nothing | Just a



Error!

The diagram consists of two speech bubbles. The first speech bubble on the left contains the text 'Error!'. The second speech bubble on the right contains the text 'Good result!'. Both bubbles are connected to the 'data Maybe a' line of code above them, illustrating the two possible values of the Maybe datatype.

Good result!

Maybe

- However, sometimes we would like to track some more information about what went wrong.
- For example, perhaps we would like to **report an error message**.
- The Maybe datatype is limiting in this case, because **Nothing** does not track any information.
- How to improve the **Maybe datatype** to allow us to track more information?

Representing Errors

- We can create a datatype **Checked**, provides a constructor **Error** to be used instead of **Nothing**

data *Checked* *a* = *Good* *a* | *Error* *String*

A good value!

Error with an error message!

Interpreter that deals with Errors

- Using **Checked**, we can reimplement the evaluate function to deal with errors.

$evaluate :: Exp \rightarrow Env \rightarrow Checked\ Value$



What kind of errors can we have in the current interpreter?

Kinds of Errors

- Three kinds of errors:
 - Division by zero
 $3/0$
 - Undefined variable errors
`var x = 3; x + y`
 - Type-errors
`3 + True`

Implementing Error Handling

Undefined Variables

- Dealing with undefined variables:

evaluate (Variable x) env =
case lookup x env of

Nothing $\rightarrow$ *Error* ("Variable " ++ x ++ " undefined")

Just v $\rightarrow$ *Good v*



Now we can
provide an error
message!

Type Errors

- Dealing with type errors:

checked_binary — — — — — = *Error* "Type error"



We could be more
informative if we wanted!

Division by zero

- Division by zero:

checked_binary Div (IntV a) (IntV 0) = Error "Divide by zero"

Propagating errors

- Propagating errors:

evaluate (Binary op a b) env =

case evaluate a env of

Error msg $\rightarrow$ *Error msg*

Good av $\rightarrow$

case evaluate b env of

Error msg $\rightarrow$ *Error msg*

Good bv $\rightarrow$

checked_binary op av bv

A bit longwinded
though!

A Problem: Tracking Errors
is Painful

Before errors

- Evaluating binary operators before errors:

*evaluate (Binary op a b) env =
binary op (evaluate a env) (evaluate b env)*

After errors

- Evaluating binary operators after errors:

evaluate (Binary op a b) env =
case evaluate a env of

Error msg $\rightarrow$ *Error msg*

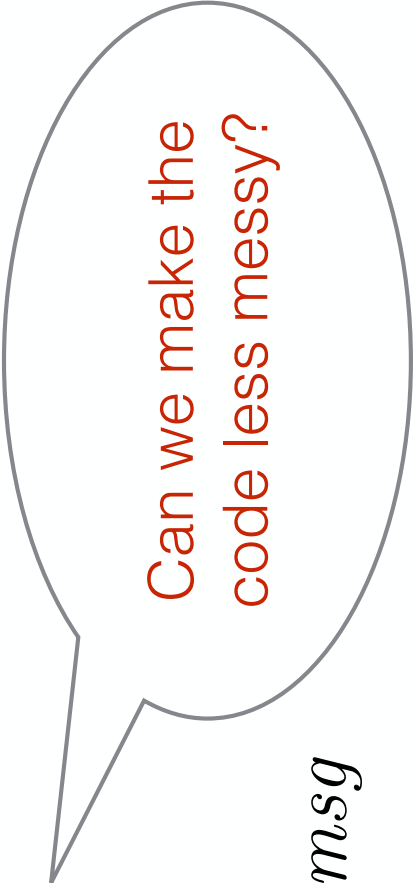
Good av $\rightarrow$

case evaluate b env of

Error msg $\rightarrow$ *Error msg*

Good bv $\rightarrow$

checked_binary op av bv



Can we make the
code less messy?

Spotting the pattern

- Evaluating binary operators after errors:

evaluate (Binary op a b) env =

case evaluate a env of

Error msg $\rightarrow$ *Error msg*

Good av $\rightarrow$

case evaluate b env of

Error msg $\rightarrow$ *Error msg*

Good bv $\rightarrow$

checked_binary op av bv

There seems to be a repeating pattern here.

Spotting the pattern

- We seem to have something like this:

case *first-part* of

Error msg $\rightarrow$ *Error msg*

Good v $\rightarrow$ *next-part v*



How to capture this
pattern as reusable code?

Spotting the pattern

- Use a higher-order function!

$A \triangleright_C F =$

case A of

$\text{Error } msg \rightarrow \text{Error } msg$

$\text{Good } v \rightarrow F \ v$

Revising the Implementation

Creating auxiliary definitions

- The higher-order function capturing error propagation:

```
(>>=) :: Checked a -> (a -> Checked b) -> Checked b
x >>= f =
  case x of
    Error msg -> Error msg
    Good v -> f v
```

We will call this
function **bind** (since it
binds a value *v*)

- A function that creates checked values:

```
return :: a -> Checked a
return v = Good v
```

Rewriting Evaluation

- Here is the new version (4 cases) of evaluation:

```
evaluateM (Literal v) env = return v
evaluateM (Variable x) env =
  case lookup x env of
    Nothing -> Error ("Variable " ++ x ++ "
undefined")
    Just v -> return v
evaluateM (Unary op a) env =
  evaluateM a env >>= checked_unary op
evaluateM (Binary op a b) env =
  evaluateM a env >>=
    \v1 -> evaluateM b env >>=
      \v2 -> checked_binary op v1 v2
```


Propagating errors

- Compare:

```
evaluateM (Binary op a b) env =  
  evaluateM a env >>=  
    \v1 -> evaluateM b env >>=  
      \v2 -> checked_binary op v1 v2
```

Propagating errors

- with

evaluate (*Binary op a b*) *env* =
case evaluate a env of

Error msg $\rightarrow$ *Error msg*

Good av $\rightarrow$

case evaluate b env of

Error msg $\rightarrow$ *Error msg*

Good bv $\rightarrow$

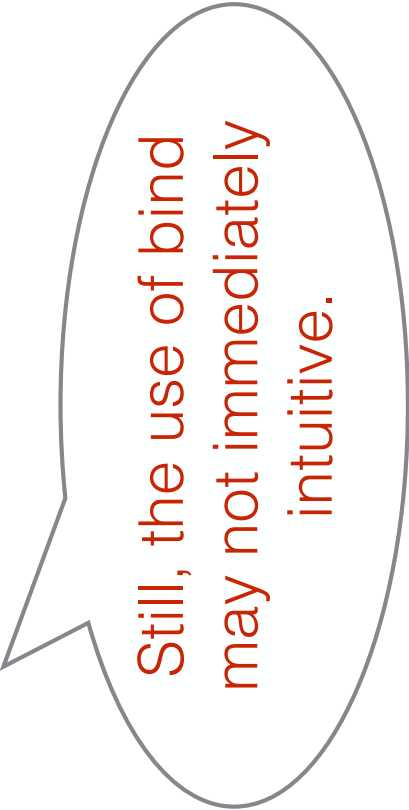
checked_binary op av bv

This code is definitely
longer.

Propagating errors

- Compare:

```
evaluateM (Binary op a b) env =  
  evaluateM a env >>=  
    \v1 -> evaluateM b env >>=  
      \v2 -> checked_binary op v1 v2
```



Still, the use of bind
may not immediately
intuitive.

Monads

Monads in Haskell

- Monads are a structure composed of two basic operations (**bind** and **return**), which capture a common pattern that occurs in many types.
- In Haskell Monads are implemented using type classes:

```
class Monad m where
    (>>=)    :: m a -> (a -> m b) -> m b
    return  :: a -> m a
```

Checked as a Monad

Because **Checked** can implement return and bind it can be made an instance of Monad

```
instance Monad Checked where
  return v = Good v
  x >>= f =
    case x of
      Error msg -> Error msg
      Good v    -> f v
```

Rewriting Code Again

- Using **bind** and **return** from the Monad class does not affect the code:

```
evaluateM (Binary op a b) env =  
  evaluateM a env >>=  
    \v1 -> evaluateM b env >>=  
      \v2 -> checked_binary op v1 v2
```

Rewriting Code Again

- However, because monads are so pervasive, Haskell supports a special notation for monads (called the **do-notation**).
- With the do-notation we can re-write the program as follows:

```
evaluateM (Binary op a b) env =  
  do v1 <- evaluate a env  
    v2 <- evaluate b env  
    checked_binary op v1 v2
```


Do-notation

- In Haskell, code using the do-notation, such as:

```
do  pattern <- exp  
    morelines
```

Is converted to code using bind:

```
exp >>= (\pattern -> do morelines)
```

Recursion

COMP3259
Principles of Programming Languages

Resources

Lecture covers:

- Chapter 5 of “Anatomy of Programming Languages”

<http://www.cs.utexas.edu/~wcook/anatomy/anatomy.htm>

Top-Level Functions

- The language with top-level functions allowed us to define recursive functions, such as:

```
function power(n,m) {  
    if (m = 0) 1; else n * power(n, m-1)  
}
```

- The function environment is built before any evaluation takes place.
- Thus when evaluating an expression all top-level functions are available in the environment.

First-class Functions and Recursion

- With the current implementation first-class functions we can model top-level functions
- However we lose the ability to define recursive functions.
- Consider, for example:

```
let fact = λn → if n ≡ 0 then 1 else n * fact (n - 1)
in fact (10)
```



What happens here?

First-class Functions and Recursion

- Recall the definition of evaluation for first class functions for declarations:

```
evaluate (Declare x exp body) env =  
  evaluate body newEnv  
  where newEnv = (x, evaluate exp env) : env
```

- The declared variable **x** is on scope in **body**, but not on **exp**!

Recalling Scope

Scope

- The **scope** of a variable is the portion of text of a program in which a variable is defined. For example:

let y = 7 in Scope of y

```
let x = 3 in
  5 + (let x = 2 in x + y) * x
```


Scope

- The **scope** of a variable is the portion of text of a program in which a variable is defined. For example:

let y = 7 in Scope of y
let x = 3 in
5 + (let x = 2 in x + y) * x

Scope when y is not a
recursive declaration!

Scoping

- The story about scoping was a little simplified as it does not account for the possibility of recursive definitions. That is, **we assumed that the following was not allowed:**

var **x** = e; body // where e uses the **x** being defined

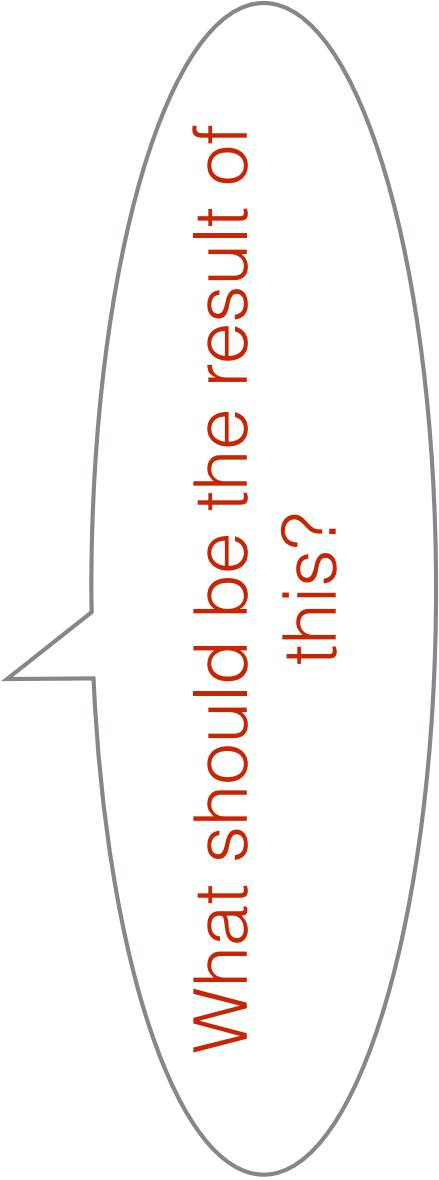
let x = e in body // where e uses the **x** being defined

We will now talk about definitions that allow this.

Recursive Declarations

- To allow recursive functions we need to allow declarations to be recursive.
- However this can cause problems. Consider the following example:

let $x = x + 1$ **in** x



What should be the result of this?

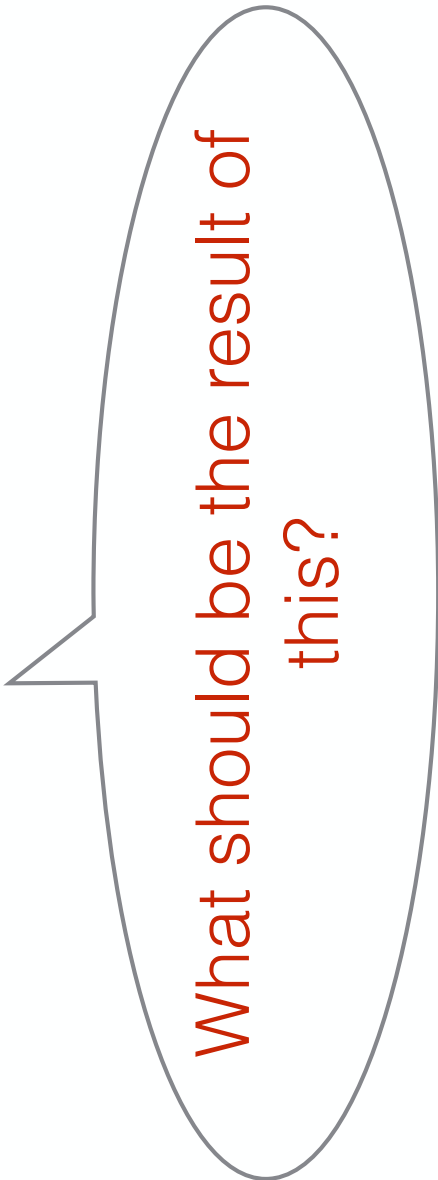
Recursive Declarations

- evaluate $\text{let } x = x + 1 \text{ in } x$ with $[]$
- evaluate $x + 1$ with $[x \rightarrow x + 1]$
- evaluate $(x + 1) + 1$ with $[x \rightarrow x + 1]$
- evaluate $((x + 1) + 1) + 1$ with $[x \rightarrow x + 1]$
- ...

Recursive Definitions in Haskell

- Consider the following Haskell definition:

```
ones = 1 : ones
```



What should be the result of this?

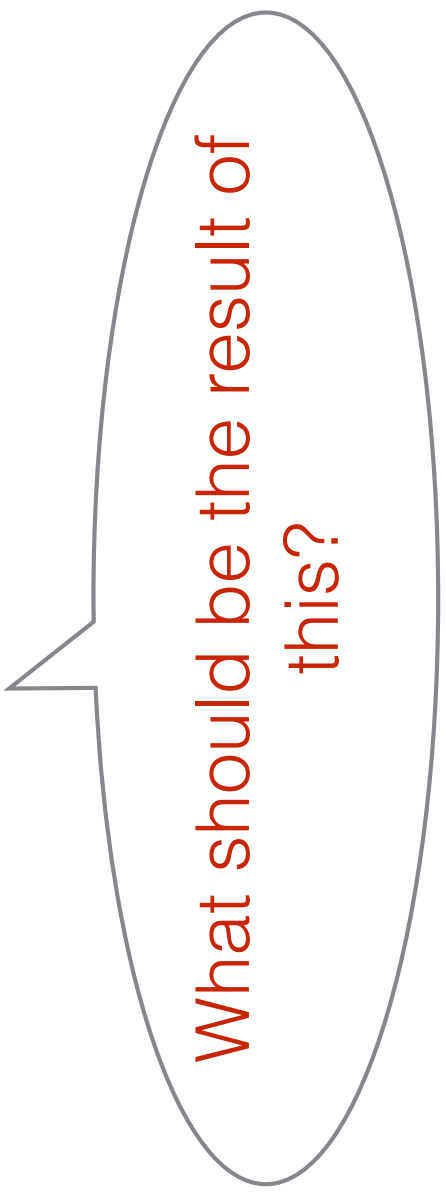
Recursive Definitions in Haskell

- evaluate `ones = 1 : ones` with `[]`
- evaluate `1 : ones` with `[ones -> 1 : ones]`
- evaluate `1 : (1 : ones)` with `[ones -> 1 : ones]`
- evaluate `1 : (1 : (1 : ones))` with `[ones -> 1 : ones]`
- ...

Recursive Definitions in Haskell

- Consider the following Haskell definition:

```
> numbers = 1 : map (\n -> n + 1) numbers
```



What should be the result of this?

- 
- What is the difference between:

```
> loop = 1 + loop
```

```
> numbers = 1 : map (\n -> n + 1) numbers
```

Are the two definitions useless?

Question

- Define a function that gives you the first 100 natural numbers using `numbers` and `take`:

`take :: Int -> [a] -> [a]`

When is a recursive definition good?

- It is not always easy to determine if a value will loop infinitely or not.
- Rule of thumb: if the **recursive variable** is used **within a data constructor** (e. g. :) or **inside a function** then it will probably produce some results.

Cyclic Definitions

Using Results of Functions as Arguments

- An interesting use of recursion and laziness is the ability to use the result of a function as an argument to the function itself.

Trees of Integers

- Consider the following definition of trees:

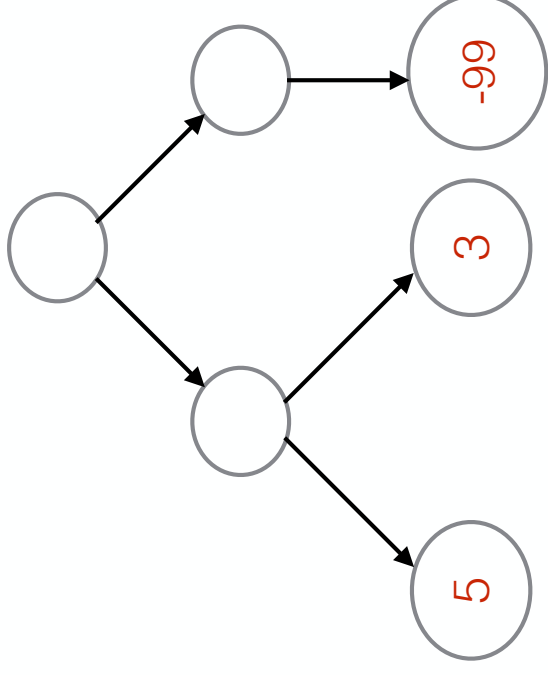
data *Tree* = *Leaf Int* | *Branch Tree Tree*

- A tree example:

Branch (Branch (Leaf 5) (Leaf 3))
(Leaf (-99))

Trees of Integers

- A tree example:



Trees of Integers

- Computing the minimum and maximum of a tree:

$$\mathit{minTree} (\mathit{Leaf} \ n) = n$$

$$\mathit{minTree} (\mathit{Branch} \ a \ b) = \mathit{min} \ (\mathit{minTree} \ a) \ (\mathit{minTree} \ b)$$

$$\mathit{maxTree} (\mathit{Leaf} \ n) = n$$

$$\mathit{maxTree} (\mathit{Branch} \ a \ b) = \mathit{max} \ (\mathit{maxTree} \ a) \ (\mathit{maxTree} \ b)$$

Trees of Integers

- Two traversals are required if we want both the minimum and maximum values!

$$\text{minTree}(\text{Leaf } n) = n$$

$$\text{minTree}(\text{Branch } a \ b) = \text{min}(\text{minTree } a) (\text{minTree } b)$$

$$\text{maxTree}(\text{Leaf } n) = n$$

$$\text{maxTree}(\text{Branch } a \ b) = \text{max}(\text{maxTree } a) (\text{maxTree } b)$$



Can we compute both in a single traversal?

Another operation

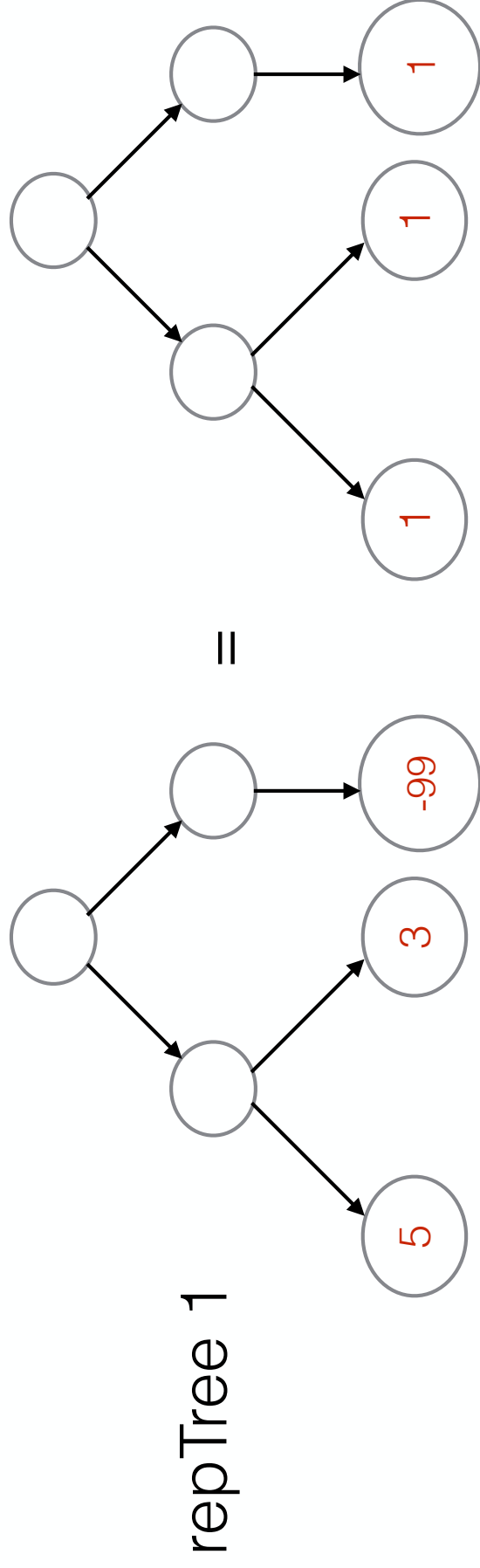
- Consider now an operation that replaces all the leaves with a specific integer value

$$\text{repTree } x (\text{Leaf } n) = \text{Leaf } x$$

$$\text{repTree } x (\text{Branch } a \ b) = \text{Branch } (\text{repTree } x \ a) \ (\text{repTree } x \ b)$$

Another operation

- replacing all values by 1



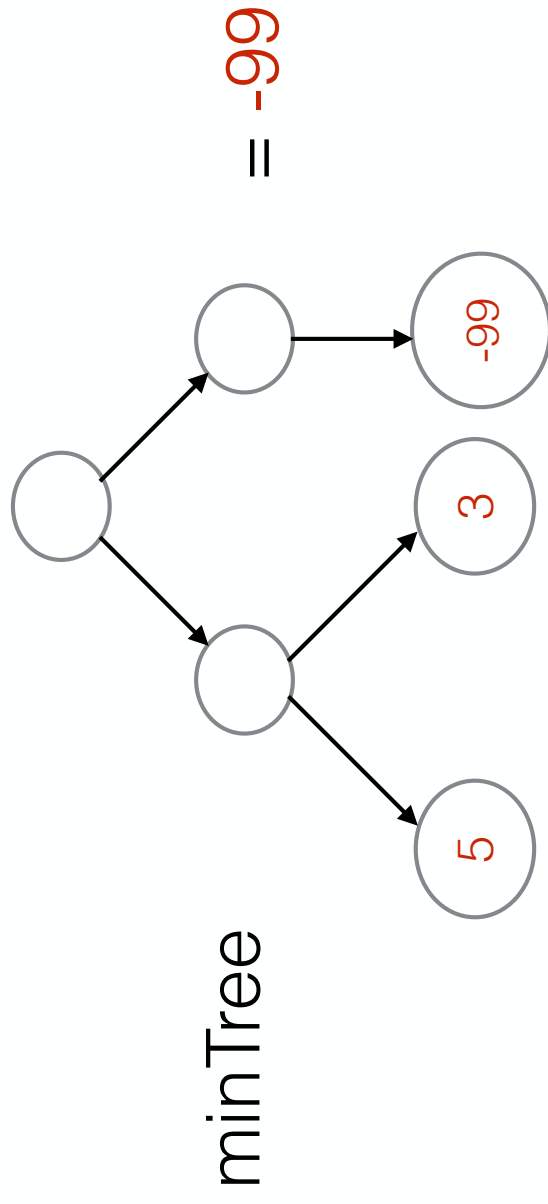
RepMin

- Now consider the problem of computing the minimum value of a tree and then replacing all values in the tree by the minimum.
- We can achieve this easily as follows:

$$\textit{repMinA tree} = \textit{repTree} (\textit{minTree tree}) \textit{ tree}$$

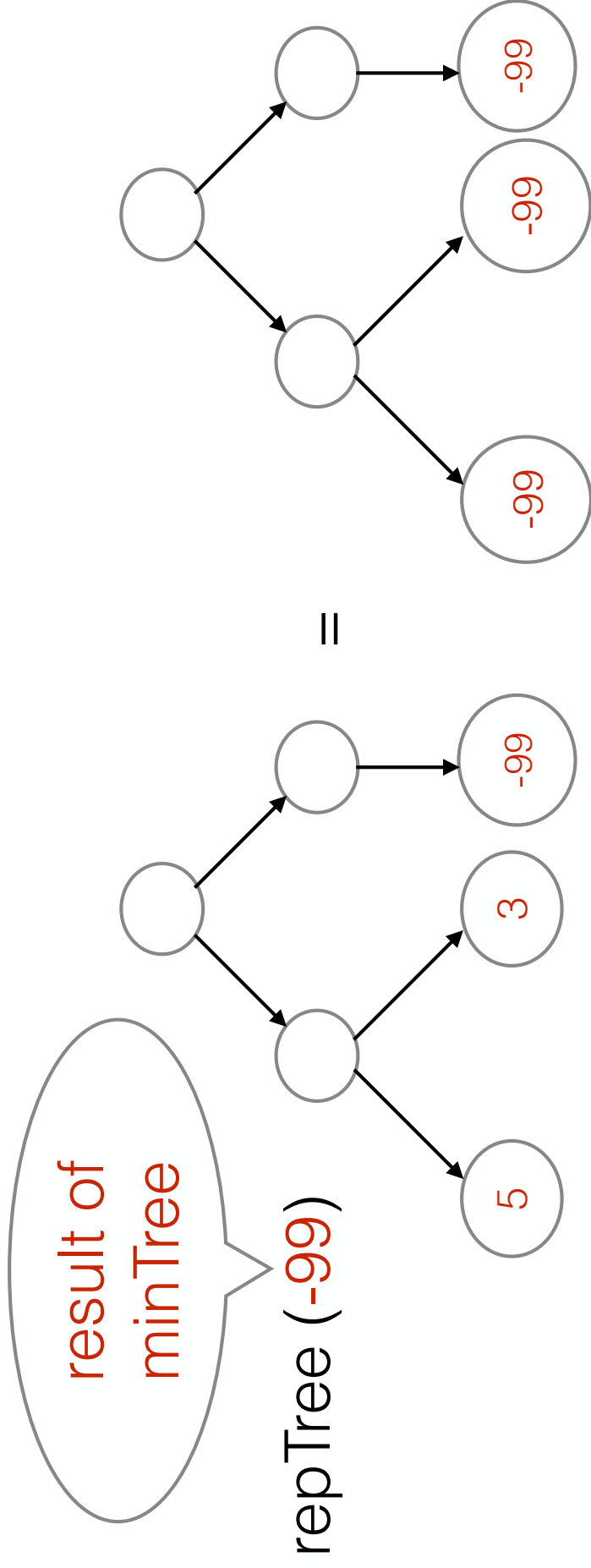
RepMin

- First traversal



RepMin

- Second traversal

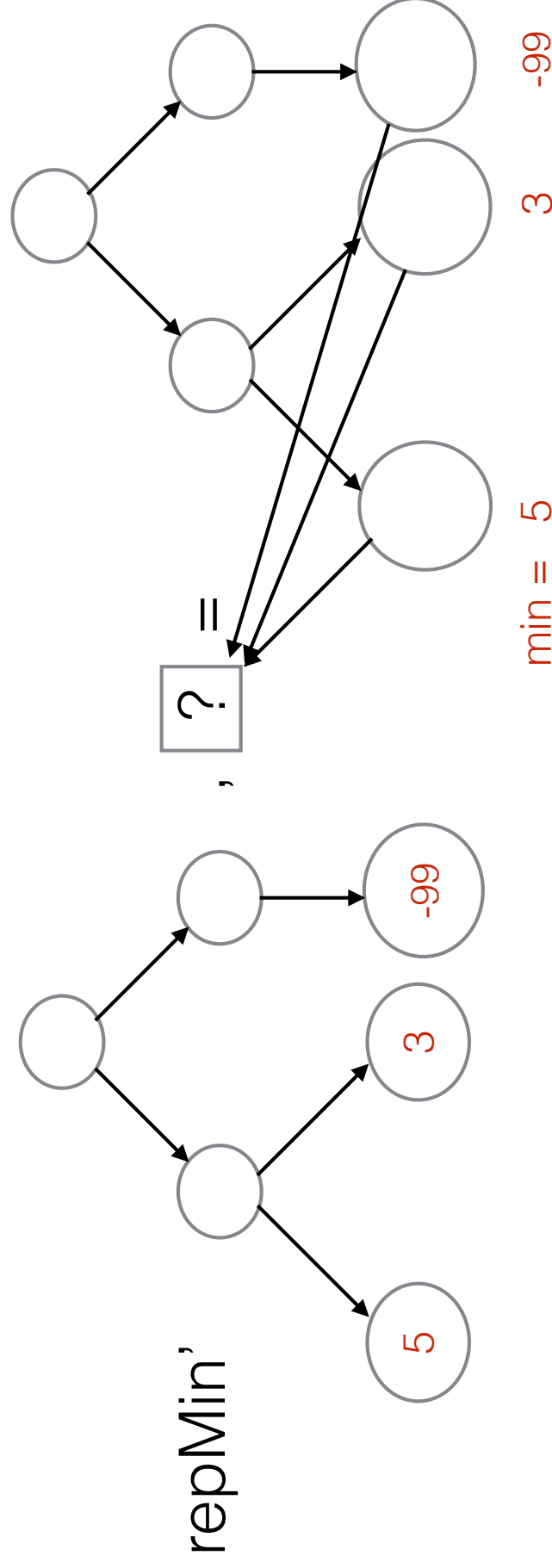


Question

- Can we compute RepMin with a single traversal?
(Similarly to what we have done with maxTree and minTree?)

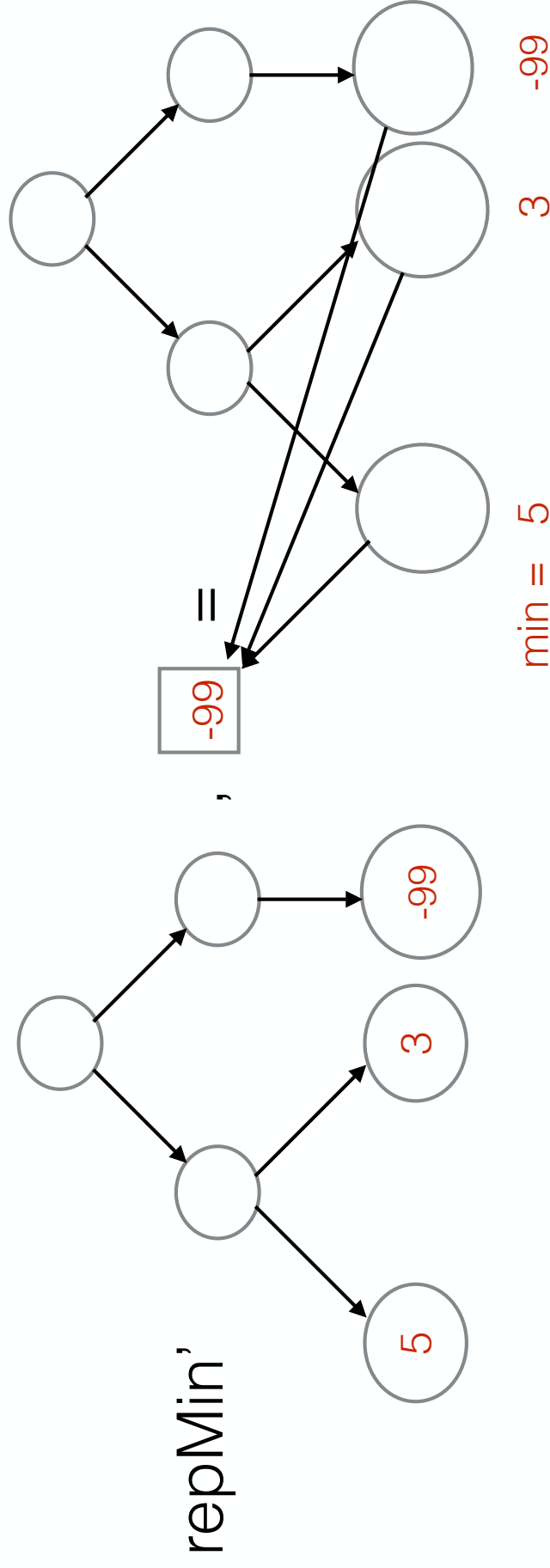
A Different Way to Understand RepMin

When repMin' reaches a leaf, it records the minimum value so far and it builds a tree where leafs point to an uninitialised memory cell.



A Different Way to Understand RepMin

At the end of the execution of repMin' the value of the memory cell is set.



Circular programs

- Repmin is an example of a **circular program**.
- A circular program is a program where some inputs are also outputs.

Implementing Recursive Variable Declarations

Implementing Recursive Variable Declarations

- The code for non-recursive variable declarations is:

evaluate (*Declare* *x exp body*) *env* = *evaluate body newEnv*
where *newEnv* = (*x, evaluate exp env*) : *env*

- Using a circular program the change to recursive variable declarations is minimal:

evaluate (*Declare* *x exp body*) *env* = *evaluate body newEnv*
where *newEnv* = (*x, evaluate exp newEnv*) : *env*

First Class Functions: Design Choices

Principles of Programming Languages

Implementing First-Class Functions

Implementation

Updating expressions:

data *Exp* =
| *Function String Exp* — — *new*

- The function constructor represents a first-class function:

$\lambda x \rightarrow \text{exp}$

Implementation

Updating values:

```
data Value = IntV Int
          | BoolV Bool
          | ClosureV String Exp Env -- new
deriving (Eq, Show)
```

- The ClosureV constructor represents a closure:

(\x -> exp, env)

function

environment at
function definition

Implementation

- Evaluating functions:

evaluate (Function x body) env = Closure V x body env — — new

- Evaluating a function simply creates a closure with the function and the current environment

Implementation

- Evaluating function calls/application:

evaluate (Call fun arg) env = evaluate body newEnv — — changed
where *ClosureV x body closeEnv = evaluate fun env*
newEnv = (x, evaluate arg env) : closeEnv

- We first evaluate the **fun** expression, which should return a closure
- Then we create a suitable new environment (**newEnv**) from the environment in the closure
- Finally we evaluate the body of the function in the closure using **newEnv**

Design Choices

Scoping Strategies

- Historically there have been two different scoping strategies for first-class functions:
 - **Static (or Lexical) Scoping:** The **free variables** in a function definition are bound to the environment at the point of the definition.
 - **Dynamic Scoping:** The **free variables** in a function definition are bound at the function call point.

Scoping Strategies

- Consider the program:

```
let x = 2 in  
let f = \y -> x + y in  
let x = 3 in  
f 5
```

- Free variables of $\lambda y \rightarrow x + y$:

$\{x\}$

Dynamic Scoping

- Example:

```
[]  
let x = 2 in  
[x ↦ 2]  
let f = \y -> x + y in  
?  
let x = 3 in  
?  
f 5
```

Dynamic Scoping

- Example:

```
[]  
let x = 2 in  
[x ↦ 2]  
let f = \y -> x + y in  
[f ↦ \y -> x + y, x ↦ 2]  
let x = 3 in  
[x ↦ 3, f ↦ \y -> x + y, x ↦ 2]  
f 5
```

Just store the
lambda expression

Evaluation

- How to evaluate the following expression?

$[x \mapsto 3, f \mapsto \lambda y \rightarrow x + y, x \mapsto 2]$
 $f\ 5$

- Lookup f in the environment
- evaluate $(\lambda y \rightarrow x + y)\ 5$ with $[x \mapsto 3, f \mapsto \lambda y \rightarrow x + y, x \mapsto 2]$
- evaluate $x + y$ with $[y \mapsto 5, x \mapsto 3, f \mapsto \lambda y \rightarrow x + y, x \mapsto 2]$

Evaluation

- evaluate $(\lambda y \rightarrow x + y) \ 5$ with $[x \mapsto 3, f \mapsto \lambda y \rightarrow x + y, x \mapsto 2]$
- evaluate $x + y$ with $[y \mapsto 5, x \mapsto 3, f \mapsto \lambda y \rightarrow x + y, x \mapsto 2]$
- result is 8
- free variables of $\lambda y \rightarrow x + y$ are bound to the environment at the call point

Dynamic Scoping

- Pros
 - Easy to implement
- Cons
 - Problematic for reasoning about programs.
Binding the free variables at the call point makes it very difficult to reason about programs

Static Scoping

- Example:

```
[]  
let x = 2 in  
[x ↦ 2]  
let f = \y -> x + y in  
[f ↦ (\y -> x + y, [x ↦ 2]), x ↦ 2]  
let x = 3 in  
[x ↦ 3, f ↦ (\y -> x + y, [x ↦ 2]), x ↦ 2]  
f 5
```

We create a
Closure!

Evaluation

- How to evaluate the following expression?

$[x \mapsto 3, f \mapsto (\lambda y \rightarrow x + y, [x \mapsto 2]), x \mapsto 2]$
 $f \ 5$

Environment
from the closure!

- Lookup f in the environment
- evaluate $(\lambda y \rightarrow x + y) \ 5$ with $[x \mapsto 2]$
- evaluate $x + y$ with $[y \mapsto 5, x \mapsto 2]$

Evaluation

- result is 7
- free variables of $\lambda y \rightarrow x + y$ are bound to the environment at the point of the function definition

Static Scoping

- Pros
 - Easy to reason about programs.
- Cons
 - We need closures for the implementation

Scoping Strategies

- Generally speaking it has been accepted that **static scoping** is a superior strategy.
- Essentially all modern programming languages use static scoping.

Evaluation Strategies

- In languages with declarations, functions or first-class functions there are a few different design options when it comes to evaluation.
- **Call-by-value**: In call-by-value expressions, such as parameters of functions or variable initialisers, are always evaluated before being added to the environment.
- **Call-by-name**: In call-by-name an expression is not evaluated until it is needed.

Evaluation Strategies

- Consider the programs:

```
var x = longcomputation; 3
```

```
(\x -> 3) longcomputation
```


Call-by-value

- Consider the programs:

```
var x = longcomputation; 3
```

```
(\x -> 3) longcomputation
```



expression is
evaluated!

- In call-by-value parameters or initializers are always evaluated.
- For some programs this can be wasteful

Call-by-name

- Consider the programs:

```
var x = longcomputation; 3
```

```
(\x -> 3) longcomputation
```

expression is
not evaluated!

- In call-by-name the expressions are added to the environment unevaluated.
- This can avoid some computations that are not needed

Call-by-name

- Consider the programs:

```
var x = longcomputation; x + x
```

```
(\x -> x + x) longcomputation
```



expression is
evaluated twice!

- In call-by-name the expressions are added to the environment unevaluated.
- However call-by-name, if implemented naively, can lead to redundant computation

Exceptions

- In a language with exceptions the result of a program may depend on the evaluation strategy.

For example:

```
var x = 3 / 0; 7
```

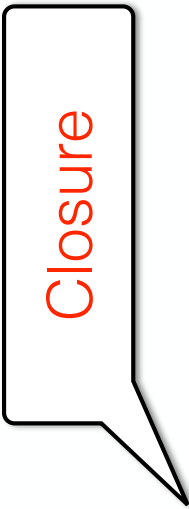
- Results in an **exception** in a call-by-value language;
- Results in **7** in a call-by-name language.

Operational Semantics (First-class functions)

Operational Semantics

- We can extend the operational semantics to account for first-class functions
- The main novelty are closure values, since now our language can compute functions as well as other types of values:

$v ::= n \mid b \mid \langle \text{fun}(x)\{e\}, \Delta \rangle$



Closure

- Closures are represented as a pair of a lambda expression $(\text{fun}(x)\{e\})$ and an environment (Δ) .

New rules

The addition of functions introduces new rules
function definitions (lambda's) and function
application:

$$\frac{}{\Delta \vdash \text{fun}(x)\{e\} \longrightarrow \textcolor{red}{\langle \text{fun}(x)\{e\}, \Delta \rangle}}$$

$$\frac{\begin{array}{c} \Delta \vdash \textcolor{red}{e_1} \longrightarrow \textcolor{red}{\langle \text{fun}(x)\{e\}, \Delta_1 \rangle} \\ \Delta \vdash e_2 \longrightarrow v_1 \quad \Delta_1, x = v_1 \vdash e \longrightarrow v_2 \end{array}}{\Delta \vdash e_1 e_2 \longrightarrow v_2}$$

Full syntax and semantics

$e ::= \dots \mid \text{fun}(x)\{e\} \mid e \ e$	Syntax
$v ::= b \mid n \mid \text{<fun}(x)\{e\}, \Delta \text{>}$	
$\Delta \vdash b \longrightarrow b$	E_b
...	
$\frac{}{\Delta \vdash \text{fun}(x)\{e\} \longrightarrow \text{<fun}(x)\{e\}, \Delta \text{>}}$	E_{lam}
$\Delta \vdash e_1 \longrightarrow \text{<fun}(x)\{e\}, \Delta_1 \text{>}$	
$\Delta \vdash e_2 \longrightarrow v_1 \quad \Delta_1, x = v_1 \vdash e \longrightarrow v_2$	E_{app}
$\frac{}{\Delta \vdash e_1 \ e_2 \longrightarrow v_2}$	

Full syntax and semantics

$$\Delta \vdash n \longrightarrow n$$

E_n

$$\frac{\Delta \vdash e_1 \longrightarrow n_1 \quad \Delta \vdash e_2 \longrightarrow n_2}{\Delta \vdash e_1 + e_2 \longrightarrow n_1 + n_2}$$

E₊

$$\frac{\Delta \vdash e_1 \longrightarrow \text{true} \quad \Delta \vdash e_2 \longrightarrow v_2}{\Delta \vdash \text{if } e_1 \text{ then } e_2 \text{ else } e_3 \longrightarrow v_2}$$

E_{if1}

$$\frac{\Delta \vdash e_1 \longrightarrow \text{false} \quad \Delta \vdash e_3 \longrightarrow v_3}{\Delta \vdash \text{if } e_1 \text{ then } e_2 \text{ else } e_3 \longrightarrow v_3}$$

E_{if2}

$$\frac{\Delta \vdash e_1 \longrightarrow v_1 \quad \Delta, x = v_1 \vdash e_2 \longrightarrow v_2}{\Delta \vdash \text{var } x = e_1; e_2 \longrightarrow v_2}$$

E_{var}

More Semantics

Sample Derivation

Suppose that we wish to find the value of:

var x = 2; var f = fun(y){y+x}; var x = 3; f(5)

Derivation D_1 (used below). Let $\Delta_1 = (x = 2, f = (\text{fun}(y)\{y+x\}, x = 2))$

$$\frac{\frac{(\Delta_1, x = 3)(f) = (\text{fun}(y)\{y+x\}, x = 2)}{\Delta_1, x = 3 \mid - f \longrightarrow (\text{fun}(y)\{y+x\}, x = 2)} \frac{E_x}{\Delta_1, x = 3 \mid - 5 \longrightarrow 5} \frac{D_2}{\Delta_1 \mid - 3 \longrightarrow 3} \frac{E_{\text{app}}}{\Delta_1 \mid - \text{var } x = 3; f \ 5 \longrightarrow 7} \frac{E_{\text{var}}}{\Delta_1 \mid - \text{var } x = 3; f \ 5 \longrightarrow 7}$$

Main Derivation:

$$\frac{\frac{\frac{x = 2 \mid - \text{fun}(y)\{y+x\} \longrightarrow (\text{fun}(y)\{y+x\}, x = 2)}{\mid - 2 \longrightarrow 2} \frac{E_n}{\mid - \text{var } x = 2; \text{var } f = \text{fun}(y)\{y+x\}; \text{var } x = 3; f(5) \longrightarrow 7} \frac{E_{\text{var}}}{\mid - \text{var } x = 2; \text{var } f = \text{fun}(y)\{y+x\}; \text{var } x = 3; f(5) \longrightarrow 7} \frac{D_1}{\mid - \text{var } x = 2; \text{var } f = \text{fun}(y)\{y+x\}; \text{var } x = 3; f(5) \longrightarrow 7} \frac{E_{\text{var}}}{\mid - \text{var } x = 2; \text{var } f = \text{fun}(y)\{y+x\}; \text{var } x = 3; f(5) \longrightarrow 7}$$

Sample Derivation

Suppose that we wish to find the value of:

var x = 2; var f = fun(y){y+x}; var x = 3; f(5)

Derivation D₂

$$\frac{(x = 2; y = 5)(y) = 5}{x = 2; y = 5 \mid - y \longrightarrow 5} E_x \quad \frac{(x = 2; y = 5)(x) = 2}{x = 2; y = 5 \mid - x \longrightarrow 2} E_x}{x = 2; y = 5 \mid - y + x \longrightarrow 7} E_+$$

First-class Functions

COMP3259
Principles of Programming Languages

Resources

Lecture covers:

- Chapter 4 of “Anatomy of Programming Languages”

<http://www.cs.utexas.edu/~wcook/anatomy/anatomy.htm>

First-Class Functions

First-Class Functions

- When a language supports functions as values, then we say that the language has **first-class functions**.
- Being first class, means that **not only we can create functions**, but also to **pass functions as arguments and return functions**.
- The language with top-level functions does not have first class-functions.

IMPORTANT: Home Reading

- Especially for those students not taking the Functional Programming class, please read the following chapter:

<http://learnyouahaskell.com/higher-order-functions>

This chapter will improve your understanding of first-class functions (and Haskell)!

Languages with first-class functions

- Today essentially all major programming languages support first-class functions:
- All functional languages support it of course:
 - Haskell, ML, OCaml, Scala, Scheme, Racket ...
- Most mainstream languages recently added support for it:
 - Java 8, C#, C++11, Swift
- Scripting languages have had this feature for a while
 - Python, Ruby, Javascript

Why First-Class Functions?

Why First-Class Functions?

- First class functions are great for **reuse**.
- First class functions offer a compact notation to **pass code around as arguments**.
- See, for example, the following use-case from the Java 8 tutorial:

<http://docs.oracle.com/javase/tutorial/java/javaOO/lambdaexpressions.html#use-case>

Language with First-Class Functions

- In a language with first class functions we can have functional (or lambda) expressions:

```
(\x -> x + 1), (\x y -> max x y), map (\x -> ord x) l
```

A light blue oval with a grey border. Inside the oval, the word "Haskell" is written in red text. A small grey arrow points from the top of the oval towards the Haskell code example above it.

Haskell

```
function(x) {return x+1;}, function(x,y) {return max(x,y);}
```

A light blue oval with a grey border. Inside the oval, the word "Javascript" is written in red text. A small grey arrow points from the top of the oval towards the Javascript code example above it.

Javascript

Lambda Calculus

- The Lambda calculus provides an extremely elegant foundation for first-class functions



Alonzo Church

Only 3 kinds of expressions in the lambda calculus:

var	variable
exp exp	application
\var -> exp	function

Example:

$(\lambda x \rightarrow x) y$

Lambda Calculus and Programming Languages

- The lambda calculus provides the foundations of modern programming languages
- Haskell is directly based on the lambda calculus

Lambda Expression

- A lambda expression allows us to declare an **anonymous function**: a function with no name. For example:

`\x -> if (x > 0) then x else -x`

function
argument
(called "x")

body
expression

Lambda Expression

- A lambda expression allows us to declare an **anonymous function**: a function with no name. For example:

```
\x -> if (x > 0) then x else -x
```

- Here is an equivalent named function:

```
absolute x = if (x > 0) then x else -x
```


Lambda Expression

- A lambda expression allow us to easily create **first-class functions**: functions that can be **passed as arguments** or returned. For example

```
map (\x -> x + 1) [1..10]
```

Lambda Expression

- A lambda expression allow us to easily create **first-class functions**: functions that can be **passed as arguments** or returned. For example

```
map (\x -> x + 1) [1..10]
```

- Here is an equivalent expression using a named function:

```
let add x = x + 1 in map add [1..10]
```

Lambda Expression

- A lambda expression allow us to easily create **first-class functions**: functions that can be passed as arguments or **returned**. For example

```
add x = \y -> x + y
```



returns a function

```
map (add 5) [1..10]
```

Declarations and Lambdas

- Functions in Haskell are implemented as lambda terms
- Consider the following Haskell definition:

$$f\ x = x * 2$$

- This is equivalent to:

$$f\ x = x * 2$$

$$f = \lambda x \rightarrow x * 2$$

Lambda Calculus in Haskell

- Rule of function arguments:

$$name\ var = body \quad \equiv \quad name = \lambda var \rightarrow body$$

We can transform a
function with 1 argument into a
function value!

Functions with 2 arguments or more

- Consider the following Haskell definition:

$\text{max } x \ y = \text{if } (x > y) \text{ then } x \ \text{else } y$

- The same transformation applies here

$\text{max } x = \backslash y \rightarrow \text{if } (x > y) \text{ then } x \ \text{else } y$

$\text{max} = \backslash x \rightarrow \backslash y \rightarrow \text{if } (x > y) \text{ then } x \ \text{else } y$



How to interpret
these definitions?

Functions with 2 arguments or more

- Consider the following Haskell definition:

`max x y = if (x > y) then x else y`

two argument function

- The same transformation applies here

`max x = \y -> if (x > y) then x else y`

one argument
function, returning a
function expression

`max = \x -> \y -> if (x > y) then x else y`

function expression

Types: Functions with 2 arguments or more

```
max :: Int -> Int -> Int  
max x y = if (x > y) then x else y
```

```
max :: Int -> (Int -> Int)  
max x = \y -> if (x>y) then x else y
```

```
max :: (Int -> (Int -> Int))  
max = \x -> \y -> if (x>y) then x else y
```


Types: Functions with 2 arguments or more

Function types are **right-associative**. Therefore all definitions have the same type:

```
max :: Int -> Int -> Int
```

```
max x y = if (x > y) then x else y
```

```
max :: Int -> Int -> Int
```

```
max x = \y -> if (x>y) then x else y
```

```
max :: Int -> Int -> Int
```

```
max = \x -> \y -> if (x>y) then x else y
```

Question

- Which of the following types are equivalent:

Type 1	Type 2
$(\text{Int} \rightarrow \text{Int}) \rightarrow \text{Int} \rightarrow \text{Int}$	$(\text{Int} \rightarrow \text{Int}) \rightarrow (\text{Int} \rightarrow \text{Int})$
$\text{Int} \rightarrow (\text{Int} \rightarrow \text{Int}) \rightarrow \text{Int}$	$\text{Int} \rightarrow \text{Int} \rightarrow \text{Int} \rightarrow \text{Int}$
$(a \rightarrow (a \rightarrow a)) \rightarrow a \rightarrow a$	$(a \rightarrow a \rightarrow a) \rightarrow (a \rightarrow a)$

Answer

- Which of the following types are equivalent:

Type 1	Type 2
$(\text{Int} \rightarrow \text{Int}) \rightarrow \text{Int} \rightarrow \text{Int}$	$(\text{Int} \rightarrow \text{Int}) \rightarrow (\text{Int} \rightarrow \text{Int})$
$\text{Int} \rightarrow (\text{Int} \rightarrow \text{Int}) \rightarrow \text{Int}$	$\text{Int} \rightarrow \text{Int} \rightarrow \text{Int} \rightarrow \text{Int}$
$(a \rightarrow (a \rightarrow a)) \rightarrow a \rightarrow a$	$(a \rightarrow a \rightarrow a) \rightarrow (a \rightarrow a)$

The types in the first and last rows are equivalent

Sections

- In Haskell, there is **syntactic sugar** to make some lambda functions even shorter. Instead of:

`map (\x -> x + 1) [1..10]`

- We can write the code with a section:

`map (+1) [1..10]`

- Here, the compiler does the following expansion:

`(+1) ==> (\x -> x + 1)`

Function Equality

- Another useful equality to know is:

$f\ x = g\ x \iff f = g$

so, instead of writing:

```
add1 xs = map (\x -> x + 1) xs
```

we can write

```
add1 = map (\x -> x + 1)
```

Question

- What do the following expressions return? Can you explain their result?

`zipWith (+) [1..10] [1..10] ==> ?`

`map (/2) [1..10] ==> ?`

`map (2/) [1..10] ==> ?`

Answers

- Answers:

`zipWith (+) [1..10] [1..10] ==> [2,4,6,...]`

`map (/2) [1..10] ==> [0.5,1.0,1.5, ...]`

`map (2/) [1..10] ==> [2.0,1.0,0.666, ...]`

Curried vs Uncurried functions

- Consider the following two Haskell definitions

$\text{max } x \ y = \text{if } (x > y) \text{ then } x \text{ else } y$

$\text{max } (x,y) = \text{if } (x > y) \text{ then } x \text{ else } y$



Are these definitions equivalent?

Curried vs Uncurried functions

- A **curried function** is a function where arguments can be partially applied

$\max x y = \text{if } (x > y) \text{ then } x \text{ else } y$

- An **uncurried function** is a function where arguments must be passed all at once

$\max (x,y) = \text{if } (x > y) \text{ then } x \text{ else } y$

Functions as Data

Another Application of First-Class Functions

Functions as Data

- When functions are values, they can also be used to define data
- For example, we can create an alternative representation of environments based on functions:
type EnvF = String -> Maybe Value
- With EnvF we can implement operations such as

```
empty :: EnvF  
lookup :: String -> EnvF -> Maybe Value  
insert :: String -> Value -> EnvF -> EnvF
```

Functions as data

The key idea is that a value of type EnvF represents the lookup function of an environment. For example:

```
env :: EnvF  
env = insert "x" 5 empty
```

To lookup a value in env all is needed is to apply it:

```
env "x"    ==> Just 5  
env "y"    ==> Nothing
```

Implementing Functions as Data

Functions as data

- The lookup operation is very simple:

```
lookup :: String -> EnvF -> Maybe Value  
lookup s f = f s
```

Functions as data

- The empty operation defines what happens when we lookup an empty environment:

```
empty :: EnvF  
empty s = Nothing
```

In other words, looking up an empty environment always returns **Nothing**.

Functions as data

- The insert operation defines what happens when we lookup a variable `s2` in an environment with one entry (`s1` $\mapsto$ `v1`) and a remaining environment `e`:

```
insert :: String -> Value -> EnvF -> EnvF
insert s1 v1 f =
  \s2 -> if (s1 == s2) then Just v1 else f s2
```



`s2` is what to
lookup

Revisiting Functions and Declarations

Revisiting Functions and Declarations

- The language with declarations allows us to write expressions such as

`var x = 5; x` (JavaScript)

`let x = 5 in x` (Haskell)

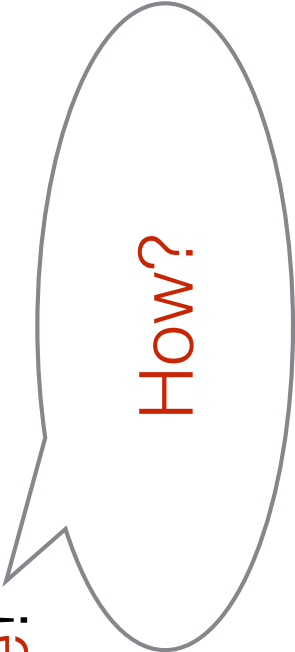
A declaration is quite similar to a function (except that it does not allow arguments).

How to add functions to a language with declarations?

Revisiting Functions and Declarations

How to add functions to a language with declarations?

1. One option is to add **functions independently of variable declarations** (we did this before for **top-level functions**).
2. Another option is to add first class functions and **get top-level functions for free!**



How?

Revisiting Functions and Declarations

Named functions with declarations and lambda expressions in JavaScript:

```
var absolute = function(x) {  
  if (x > 0)  
    return x;  
  else  
    return -x;  
};  
absolute(-3)
```

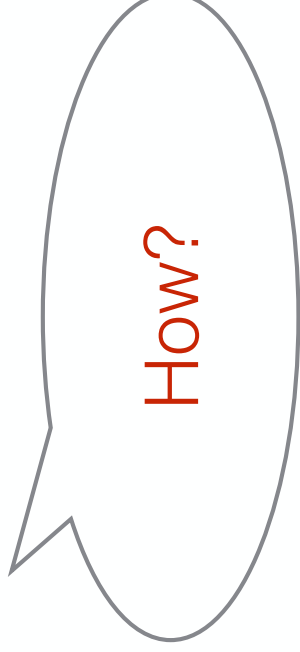
Revisiting Functions and Declarations

Named functions with declarations and lambda expressions in Haskell:

```
let absolute = \x -> if (x > 0) then x else -x  
in absolute(-3)
```

Declarations & Lambdas

- It turns out that even declarations are not needed with lambdas: **we can implement declarations in terms of lambdas and function application**



Declarations & Lambdas

- It turns out that even declarations are not needed with lambda's: we can implement declarations in terms of lambdas and function application

```
let x = exp in body
====>
(\x -> body) exp
```

Declarations & Lambdas

- For example

let x = 5 in x + 8

==>

(\x -> x + 8) 5



Both expressions
return 13

Scoping Again

- Consider the (Haskell) program:

```
let x = 2 in  
let f y = x + y in  
let x = 3 in  
f 5
```



What's the
result?

Scoping Again (Java)

```
public class Test {  
    int x = 2;  
}
```

```
public class A {  
    int y = x;  
}
```

```
int f(int y) {  
    return x + y;  
}
```

```
public static void main(String args[]) {  
    int x = 3;  
    Test t = new Test();  
    A o = t.new A();  
    System.out.println(o.y);  
    System.out.println(o.f(5));  
}
```

What is the result?

What is the result?

Scoping Again

- Consider the (Haskell) program:

```
let x = 2 in  
let f y = x + y in
```

```
let x = 3 in  
f 5
```



In Haskell result
is 7

Scoping Again (Java)

```
public class Test {  
    int x = 2;  
}
```

```
public class A {  
    int y = x;  
}
```

```
int f(int y) {  
    return x + y;  
};  
}
```

```
public static void main(String args[]) {  
    int x = 3;  
    Test t = new Test();  
    A o = t.new A();  
    System.out.println(o.y);  
    System.out.println(o.f(5));  
}
```

Result is 2

Result is 7, like
Haskell

Scoping Again

- What's going on?

```
[]  
let x = 2 in  
[x |-> 2]  
let f = \y -> x + y in  
[(f |-> \y -> x + y, [x |-> 2]) , x |-> 2]  
let x = 3 in  
[x |-> 3, (f |-> \y -> x + y, [x |-> 2]), x |-> 2]  
f 5
```

Continued

$[x] \rightarrow 3, (f \rightarrow [y \rightarrow x + y, [x \rightarrow 2]], x \rightarrow 2]$
f 5

$[x] \rightarrow 2]$

$([y \rightarrow x + y) 5$

$[x] \rightarrow 2]$

$x + 5$

$[x] \rightarrow 2]$

$2 + 5$

Scoping Again

- What's going on?

```
[]  
let x = 2 in  
[x ↦ 2]  
let f = \y -> x + y in  
[f ↦ (\y -> x + y, [x ↦ 2]), x ↦ 2]  
let x = 3 in  
[x ↦ 3, f ↦ (\y -> x + y, [x ↦ 2]), x ↦ 2]  
f 5
```

We create a
Closure!

Closure

- **Closure**: A closure is a combination of a function expression and an environment.
- Closures preserve the **bindings that existed at the point when the function was defined**.

Evaluation

- What's going on?

```
[]  
let x = 2 in  
[x ↦ 2]  
let f = \y -> x + y in  
[f ↦ (\y -> x + y, [x ↦ 2]), x ↦ 2]  
let x = 3 in  
[x ↦ 3, f ↦ (\y -> x + y, [x ↦ 2]), x ↦ 2]  
f 5
```



How to evaluate?

Evaluation

- How to evaluate the following expression?

$[x \mapsto 3, f \mapsto (\lambda y \rightarrow x + y, [x \mapsto 2]), x \mapsto 2]$
 $f \ 5$

Environment
from the closure!

- Lookup f in the environment
- evaluate $(\lambda y \rightarrow x + y) \ 5$ with $[x \mapsto 2]$
- evaluate $x + y$ with $[y \mapsto 5, x \mapsto 2]$

Closures in Other Languages

- The idea of closures is not only important for functional languages and first-class functions.
- OOP languages, for example, also need a form of closures.

Scoping Again (Java)

```
public class Test {  
    int x = 2;  
}
```

```
public class A {  
    int y = x;  
}
```

```
int f(int y) {  
    return x + y;  
};  
}
```

```
public static void main(String args[]) {  
    int x = 3;  
    Test t = new Test();  
    A o = t.new A();  
    System.out.println(o.y);  
    System.out.println(o.f(5));  
}
```

Result is 2

Result is 7, like
Haskell

Evaluation of functions

- The rule to evaluate lambda expressions is:

$(\backslash \text{var} \rightarrow \text{body}) \text{ exp}$

$====>$

substitute $\text{var} \mapsto (\text{evaluate exp})$ in body

Top-Level Functions

Resources

Lecture covers:

- Chapter 4 of “Anatomy of Programming Languages”

<http://www.cs.utexas.edu/~wcook/anatomy/anatomy.htm>

Top-level Functions

- One more extension to the language is to allow top-level functions:

```
function absolute(x) {  
  if (x > 0) then x else -x  
}
```

```
absolute(-3)
```

Top-level functions are defined **outside expressions**.

Many languages familiar to you support top-level functions only (example **C**).

Examples of top-level functions

- Top-level functions in C

```
int fact(int n)
{
    if (n >= 1)
        return n*fact(n-1);
    else
        return 1;
}
```

Examples of top-level functions

- Top-level functions in JavaScript

```
function fact(n) {  
    if(n >= 1) {  
        return 1  
    } else {  
        return n * fact(n - 1);  
    }  
}
```

Top-level Functions

- Syntax for function definitions:

function *function-name* (*parameter-name*, ..., *parameter-name*) { *body-expression* }

- Syntax for function calls:

function-name (*expression*, ..., *expression*)

- Complete Program:

function1 function2 ... exp

Complete Program Example

function **absolute**(x) {if (x > 0) then x else -x}

function **max**(x,y) {if (x > y) then x else y}

max(**absolute**(-5),4)

function definitions

final expression

Implementing Top-Level Functions

Programs

- The datatype of programs is:

data *Program* = *Program FunEnv Exp*

- Programs have a function environment:

type *FunEnv* = [(*String*, *Function*)]

- where functions are defined as:

data *Function* = *Function [String] Exp*

body of the
function

argument names

Function Calls

- To be able to call functions we need one extension to expressions:

data $Exp = \dots$

| *Call String* [Exp]

name of function
to call

list of arguments

Evaluating Programs

- We use a top-level execution function:

$execute :: Program \rightarrow Value$

$execute (Program\ funEnv\ main) = evaluate\ main\ []\ funEnv$

- The evaluator takes one additional argument: the function environment

$evaluate :: Exp \rightarrow Env \rightarrow FunEnv \rightarrow Value$

Evaluating Programs

- Finally we need to extend the evaluator with the case for function calls:

```
evaluate (Call fun args) env funEnv = evaluate body newEnv funEnv  
where Function xs body = fromJust (lookup fun funEnv)  
newEnv = zip xs [evaluate a env funEnv | a ← args]
```

More Kinds of Data

COMP3259
Principles of Programming Languages

More Kinds of Data (Booleans)

Data

- So far our language has a single type of data: **integers**
- Programming languages usually support multiple types of data (such as **booleans**, **floating point numbers**, **arrays**, **functions**...)
- In this lecture we will extend our language with **booleans** and (start with) **functions**.

Boolean Data

- A simple extension is to allow booleans and boolean operations in the language

`true, false, if (x > 0) then x+y else z`

- Adding this extension requires a little refactoring, but it is not difficult

Implementing Boolean Data

Values

- We now need a datatype of values instead of integers

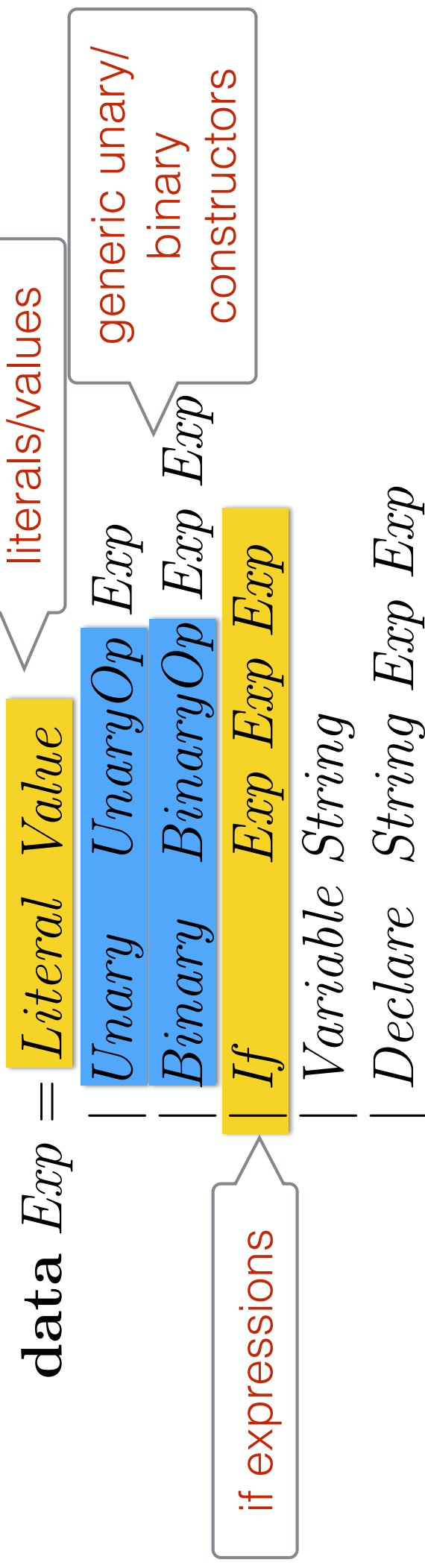
data *Value* = *IntV Int*
 | *BoolV Bool*

integer values

boolean values

Abstract Syntax

- The **Exp** datatype needs to be extended to account for boolean data and operations
- It is also useful to refactor abstract syntax a little bit



Operator Tags

- To implement generic operators, we use two datatype **UnaryOp** and **BinaryOp**

```
data BinaryOp = Add | Sub | Mul | Div | And | Or  
              | GT | LT | LE | GE | EQ
```

```
data UnaryOp = Neg | Not
```

- These datatypes are just **enumerations**: the constructors are used as tag names.

Evaluation

- Here is how to do evaluation:

type *Env* = [(*String*, *Value*)]

evaluate :: *Exp* → *Env* → *Value*

evaluate (*Literal* *v*) *env* = *v*

evaluate (*Unary* *op* *a*) *env* = *unary* *op* (*evaluate* *a* *env*)

evaluate (*Binary* *op* *a* *b*) *env* = *binary* *op* (*evaluate* *a* *env*) (*evaluate* *b* *env*)

evaluate (*Variable* *x*) *env* = *fromJust* (*lookup* *x* *env*)

evaluate (*Declare* *x* *exp* *body*) *env* = *evaluate* *body* *newEnv*

where *newEnv* = (*x*, *evaluate* *exp* *env*) : *env*

evaluate (*If* *a* *b* *c*) *env* =

let *BoolV* *test* = *evaluate* *a* *env* **in**

if *test* **then** *evaluate* *b* *env*

else *evaluate* *c* *env*

main change:
new case for if

Evaluation

- The small, but notable difference is the use of functions **unary** and **binary** to interpret the unary and binary operations

unary *Not* (*BoolV* *b*) = *BoolV* ($\neg b$)

unary *Neg* (*IntV* *i*) = *IntV* ($-i$)

binary *Add* (*IntV* *a*) (*IntV* *b*) = *IntV* ($a + b$)

binary *Sub* (*IntV* *a*) (*IntV* *b*) = *IntV* ($a - b$)

binary *Mul* (*IntV* *a*) (*IntV* *b*) = *IntV* ($a * b$)

binary *Div* (*IntV* *a*) (*IntV* *b*) = *IntV* ($a \text{ ‘div’ } b$)

binary *And* (*BoolV* *a*) (*BoolV* *b*) = *BoolV* ($a \wedge b$)

binary *Or* (*BoolV* *a*) (*BoolV* *b*) = *BoolV* ($a \vee b$)

binary *LT* (*IntV* *a*) (*IntV* *b*) = *BoolV* ($a < b$)

binary *LE* (*IntV* *a*) (*IntV* *b*) = *BoolV* ($a \leq b$)

binary *GE* (*IntV* *a*) (*IntV* *b*) = *BoolV* ($a \geq b$)

binary *GT* (*IntV* *a*) (*IntV* *b*) = *BoolV* ($a > b$)

binary *EQ* *a* *b* = *BoolV* ($a \equiv b$)

Operational Semantics

Operational Semantics

- We can extend the operational semantics to account for other types of data
- The main novelty is the introduction of values, since now our language can compute booleans as well as integers:

$$v ::= n \mid b$$

- For this language, values (v) can either be numbers (n) or booleans (b).

New rules

The addition of values does not introduce many changes, but there are new rules

The new rules are the rules conditional expressions:

$$\frac{\Delta \vdash e_1 \longrightarrow \text{true} \quad \Delta \vdash e_2 \longrightarrow v_2}{\Delta \vdash \text{if } e_1 \text{ then } e_2 \text{ else } e_3 \longrightarrow v_2}$$

$$\frac{\Delta \vdash e_1 \longrightarrow \text{false} \quad \Delta \vdash e_3 \longrightarrow v_3}{\Delta \vdash \text{if } e_1 \text{ then } e_2 \text{ else } e_3 \longrightarrow v_3}$$

Full syntax and semantics

$e ::= \dots \mid \text{true} \mid \text{false} \mid \text{if } e \text{ then } e \text{ else } e$	Syntax
$v ::= b \mid n$	
$\Delta \vdash b \longrightarrow b$	E_b Semantics

...

$$\frac{\Delta \vdash e_1 \longrightarrow \text{true} \quad \Delta \vdash e_2 \longrightarrow v_2}{\Delta \vdash \text{if } e_1 \text{ then } e_2 \text{ else } e_3 \longrightarrow v_2} E_{\text{if1}}$$
$$\frac{\Delta \vdash e_1 \longrightarrow \text{false} \quad \Delta \vdash e_3 \longrightarrow v_3}{\Delta \vdash \text{if } e_1 \text{ then } e_2 \text{ else } e_3 \longrightarrow v_3} E_{\text{if2}}$$

Local Variables

COMP3259
Principles of Programming Languages

Resources

Lecture covers:

- Chapter 3 of “Anatomy of Programming Languages”

<http://www.cs.utexas.edu/~wcook/anatomy/anatomy.htm>

Local Variables

- So far variables are defined outside the language.
- **Local variables** allow the definition of variables inside the language.

```
int x = 3;  
return 2 * x + 5;
```

C or Java code

```
var x = 3;  
return 2 * x + 5;
```

JavaScript

```
let x = 3 in 2 * x + 5
```

Haskell

Multiple Local Variables

- There can be multiple local variables.

```
int x = 3;  
int y = x * 2;  
return x + y;
```

C or Java code

```
let x = 3 in let y = x * 2 in x + y
```

Haskell

Shadowing

- Multiple variables introduce an interesting issue: there can be **multiple local variables with the same name!**
- What should happen in this situation?

Shadowing in C

What is the output of the following C programs? Is the output the same?

```
#include <stdio.h>

int main(void)
{
    int x = 0;

    if (x == 0) {
        int x = 1;
        printf("Inside if x is: %d\n", x);
    }

    printf("Here x is: %d\n", x);
    return 0;
}
```

```
#include <stdio.h>

int main(void)
{
    int x = 0;

    if (x == 0) {
        x = 1;
        printf("Inside if x is: %d\n", x);
    }

    printf("Here x is: %d\n", x);
    return 0;
}
```

Shadowing in C

This program declares **two variables** called **x**. Each having different values.

```
#include <stdio.h>

int main(void)
{
    int x = 0;

    if (x == 0) {
        int x = 1;
        printf("Inside if x is: %d\n", x);
    }

    printf("Here x is: %d\n", x);
    return 0;
}
```

This program declares **one variable** called **x** and it **mutates** its value.

```
#include <stdio.h>

int main(void)
{
    int x = 0;

    if (x == 0) {
        x = 1;
        printf("Inside if x is: %d\n", x);
    }

    printf("Here x is: %d\n", x);
    return 0;
}
```

Shadowing in C

```
#include <stdio.h>
```

```
int main(void)  
{
```

```
    int x = 0;
```

```
    if (x == 0) {
```

```
        int x = 1;
```

```
        printf("Inside if x is: %d\n", x);
```

```
    }
```

```
    printf("Here x is: %d\n", x);
```

```
    return 0;
```

```
}
```

this variable
declaration **shadows** the
previous definition of **x**
inside the if block

here the value of the
most local variable called x
is used

Shadowing in Haskell

- We can create a similar program in Haskell to illustrate variable shadowing

```
Prelude> let x = 0 in (let x = 1 in x, x)  
(1,0)
```

In the past students asked me whether **let expressions in Haskell are not the same as mutation**. The answer is no. What is happening here is **shadowing**, not mutation!

Shadowing in General

- Nearly every language has some form of shadowing
- It is important for programmers to be aware of shadowing and its semantics as it can often give rise to subtle bugs
- Some languages forbid certain types of shadowing (though usually not all)
- Generally speaking it is better to avoid shadowing (although there are some use cases for it)

Scope

- The **scope** of a variable is the portion of text of a program in which a variable is defined. For example:

let y = 7 in Scope of y

```
let x = 3 in  
  5 + (let x = 2 in x + y) * x
```

Scope

- When a variable is redefined this creates a hole in the scope of the outer definition:

```
let y = 7 in
  let x = 3 in
    5 + (let x = 2 in x + y) * x
```

Scope of first x

```
let y = 7 in
  let x = 3 in
    5 + (let x = 2 in x + y) * x
```

Scope of second x

Scoping in C

```
#include <stdio.h>
```

```
int main(void)  
{
```

```
    int x = 0;
```

```
    if (x == 0) {
```

```
        int x = 1;
```

```
        printf("Inside if x is: %d\n", x);
```

```
    }
```

```
    printf("Here x is: %d\n", x);
```

```
    return 0;
```

```
}
```

scope of first x

scope of second x

In a block-structured language like C, blocks delimit scopes of variables

Scoping

- The story about scoping is a little simplified as it does not account for the possibility of recursive definitions. That is, **we assume that the following is not allowed:**

int **x** = e; // where e uses the **x** being defined

let x = e // where e uses the **x** being defined

Some languages do allow such definitions. We will talk about this later in the course.

Implementing Local Variables

Local Variables

- We will see how to implement an interpreter with local variables in two different ways:
 - Using **substitution**
 - Using **environments**
- Substitution is **helpful for reasoning** about the behaviour of the interpreter, but costly. Environments enable **better performance**.

Substitution in PLS

- Conceptually substitution happens every time we do a function application. Consider:

`function f(x) {e}` // for some expression e

- In the function call

`f(5)`

- we **substitute** all occurrences of x by 5

Substitution with Declarations

substitute $x \mapsto 5$ in $x + 3$ $\implies 5 + 3$

substitute $x \mapsto 5$ in $\text{var } x = 4; x + 3$ $\implies$ $\text{var } x = 4; x + 3$

substitute $x \mapsto 5$ in $\text{var } y = x; y + 3$ $\implies$ $\text{var } y = 5; y + 3$

substitute $x \mapsto 5$ in $\text{var } y = x; \text{var } x = 4; y + x$
 $\text{var } y = 5; \text{var } x = 4; y + x$

Demo: Implementing Interpreter with Substitution

Local Variables

- We need to extend the abstract syntax:

data $Exp = \dots$
 | *Declare String Exp Exp*

- update the substitution function:

substitute1 (var, val) exp = subst exp

...
subst (Declare x exp body) = Declare x (subst exp) body'
 where *body' = if* $x \equiv var$
 then *body*
 else *subst body*

Local Variables

- and finally update evaluation

evaluate (Declare x exp $body$) = evaluate (substitute1 (x , evaluate exp) $body$)

Environment-based Evaluation

- Evaluation using substitution is intuitive, but not efficient

$\text{evaluate} :: \text{Exp} \rightarrow \text{Exp}$

- Another way to implement evaluation is to use environments:

$\text{evaluate} :: \text{Exp} \rightarrow \text{Env} \rightarrow \text{Exp}$

Demo: Implementing Interpreter with Environments

Environment-based Evaluation

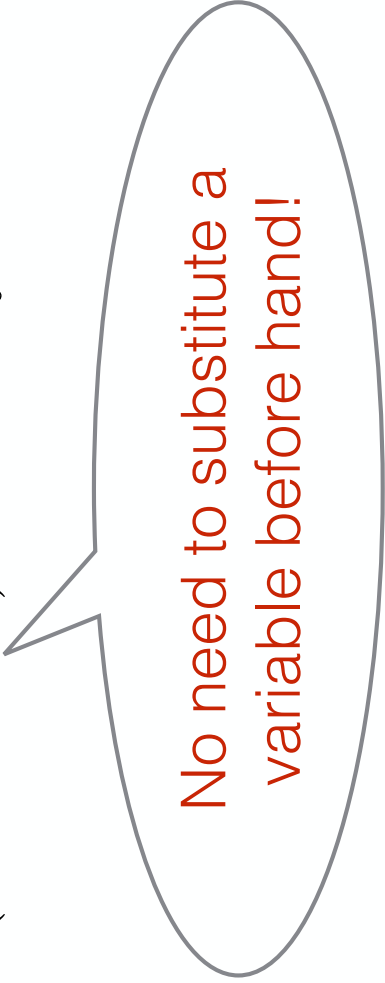
- Here is an implementation of Environment-based substitution

```
evaluate :: Exp → Env → Int
evaluate (Number i) env = i
evaluate (Add a b) env   = evaluate a env + evaluate b env
evaluate (Subtract a b) env = evaluate a env - evaluate b env
evaluate (Multiply a b) env = evaluate a env * evaluate b env
evaluate (Divide a b) env   = evaluate a env `div` evaluate b env
evaluate (Variable x) env   = fromJust (lookup x env)
evaluate (Declare x exp body) env = evaluate body (newEnv
    where newEnv = (x, evaluate exp env) : env
```

Environment-based Evaluation

- Variable case:
- looks up the value of a variable from the environment

evaluate (Variable x) env = fromJust (lookup x env)



No need to substitute a variable before hand!

Environment-based Evaluation

- Declare case:
- Extends the environment for evaluating the body

evaluate (Declare x exp $body$) env = evaluate $body$ $newEnv$
where $newEnv = (x, \text{evaluate } exp \ env) : env$



Extending the
environment with a new binding

Inefficiency of Substitution-based Evaluation

- When doing evaluation in a variable declaration expression substitution creates a **copy the body with variables substituted**.
- With nested variable declarations this becomes inefficient because the **body is copied multiple times**.

Inefficiency of Substitution-based Evaluation

- Consider the following expression:

var $x = 2$;

var $y = x + 1$;

var $z = y + 2$;

$x * y * z$

- What are the **steps** when evaluating this expression?

Step	Result
initial expression	$var\ x = 2;$
	$var\ y = x + 1;$
	$var\ z = y + 2;$
	$x * y * z$

Step	Result
initial expression	$var\ x = 2;$
	$var\ y = x + 1;$
	$var\ z = y + 2;$
	$x * y * z$
evaluate bound expression	$2 \Rightarrow 2$

Step	Result
initial expression	$var\ x = 2;$
	$var\ y = x + 1;$
	$var\ z = y + 2;$
	$x * y * z$
evaluate bound expression	$2 \Rightarrow 2$
substitute $x \mapsto 2$ in body	$var\ y = 2 + 1;$
	$var\ z = y + 2;$
	$2 * y * z$

creating a copy

Step	Result
initial expression	$var\ x = 2;$ $var\ y = x + 1;$ $var\ z = y + 2;$ $x * y * z$
evaluate bound expression	$2 \Rightarrow 2$
substitute $x \mapsto 2$ in body	$var\ y = 2 + 1;$ $var\ z = y + 2;$ $2 * y * z$
	creating a copy
evaluate bound expression	$2 + 1 \Rightarrow 3$
substitute $y \mapsto 3$ in body	$var\ z = 3 + 2;$ $2 * 3 * z$
	creating a copy

Step	Result
initial expression	$var\ x = 2;$ $var\ y = x + 1;$ $var\ z = y + 2;$ $x * y * z$
evaluate bound expression	$2 \Rightarrow 2$
substitute $x \mapsto 2$ in body	$var\ y = 2 + 1;$ $var\ z = y + 2;$ $2 * y * z$
	creating a copy
evaluate bound expression	$2 + 1 \Rightarrow 3$
substitute $y \mapsto 3$ in body	$var\ z = 3 + 2;$ $2 * 3 * z$
	creating a copy
evaluate bound expression	$3 + 2 \Rightarrow 5$
substitute $z \mapsto 5$ in body	$2 * 3 * 5$
evaluate body	$2 * 3 * 5 \Rightarrow 30$

Environment-Based Evaluation

- Rather than fully performing substitution for each variable declaration, we can:
- add a binding to the environment;
- perform substitution when reaching variable case, by looking up that variable in the environment.

Environment	Evaluation
$\emptyset$	$var\ x = 2;$
	$var\ y = x + 1;$
	$var\ z = y + 2;$
	$x * y * z$

Environment	Evaluation
$\emptyset$	$var\ x = 2;$ $var\ y = x + 1;$ $var\ z = y + 2;$ $x * y * z$ $\{ \text{evaluate bound expression } 2 \}$ $2 \Rightarrow 2$
$\emptyset$	

Environment	Evaluation
$\emptyset$	$var\ x = 2;$ $var\ y = x + 1;$ $var\ z = y + 2;$ $x * y * z$ $\{ \text{evaluate bound expression } 2 \}$
$\emptyset$	$2 \Rightarrow 2$ $\{ \text{add new binding for } x \text{ and evaluate body of variable declaration } \}$
$x \mapsto 2$	$var\ y = x + 1;$ $var\ z = y + 2;$ $x * y * z$

Environment	Evaluation
$\emptyset$	$var\ x = 2;$ $var\ y = x + 1;$ $var\ z = y + 2;$ $x * y * z$ $\{ \text{evaluate bound expression } 2 \}$
$\emptyset$	$2 \Rightarrow 2$ $\{ \text{add new binding for } x \text{ and evaluate body of variable declaration } \}$
$x \mapsto 2$	$var\ y = x + 1;$ $var\ z = y + 2;$ $x * y * z$ $\{ \text{evaluate bound expression } x + 1 \}$
$x \mapsto 2$	$x + 1 \Rightarrow 3$

we know $x = 2$ from the environment

Environment	Evaluation
$\emptyset$	$var\ x = 2;$ $var\ y = x + 1;$ $var\ z = y + 2;$ $x * y * z$ $\{ \text{evaluate bound expression } 2 \}$
$\emptyset$	$2 \Rightarrow 2$ $\{ \text{add new binding for } x \text{ and evaluate body of variable declaration } \}$
$x \mapsto 2$	$var\ y = x + 1;$ $var\ z = y + 2;$ $x * y * z$ $\{ \text{evaluate bound expression } x + 1 \}$
$x \mapsto 2$	$x + 1 \Rightarrow 3$ $\{ \text{add new binding for } y \text{ and evaluate body of variable declaration } \}$
$y \mapsto 3, x \mapsto 2$	$var\ z = y + 2;$ $x * y * z$

we know $x = 2$ from the environment

Environment	Evaluation
$\emptyset$	$var\ x = 2;$ $var\ y = x + 1;$ $var\ z = y + 2;$ $x * y * z$ { evaluate bound expression 2 } $2 \Rightarrow 2$ { add new binding for x and evaluate body of variable declaration }
$x \mapsto 2$	$var\ y = x + 1;$ $var\ z = y + 2;$ $x * y * z$ { evaluate bound expression $x + 1$ }
$x \mapsto 2$	{ add new binding for y and evaluate body of variable declaration } $var\ z = y + 2;$ $x * y * z$ { evaluate bound expression $y + 2$ }
$y \mapsto 3, x \mapsto 2$	$y + 2 \Rightarrow 5$ { add new binding for z and evaluate body of variable declaration }
$z \mapsto 5, y \mapsto 3, x \mapsto 2$	$x * y * z \Rightarrow 70$

we know $x = 2$ from the environment

we know $y = 3$ from the environment

Operational Semantics

Operational Semantics

- We can extend the operational semantics for Arithmetic expressions
- The main novelty is the introduction of (evaluation) environments, which we represent using the greek letter Δ (Delta).

Operational Semantics

The grammar of environments is:

$$\Delta ::= . \mid \Delta, x = n$$

Environments can either be:

- **Empty**: represented as a dot (.)
- **Non-empty**: Contain at least a binding ($\Delta, x = n$)

Operational Semantics

The addition of environments **changes** the existing arithmetic rules because environments now have to be propagated. For example, instead of:

$$\frac{e_1 \longrightarrow n_1 \quad e_2 \longrightarrow n_2}{e_1 + e_2 \longrightarrow n_1 + n_2}$$

we have:

$$\frac{\Delta \mid - e_1 \longrightarrow n_1 \quad \Delta \mid - e_2 \longrightarrow n_2}{\Delta \mid - e_1 + e_2 \longrightarrow n_1 + n_2}$$

New rules

The new rules are the rules for variables and declarations:

$\Delta(x) = n$ means the
lookup x in Δ
returns n

$$\frac{\Delta(x) = n}{\Delta \vdash x \longrightarrow n}$$

Environment is
updated here!

$$\frac{\Delta \vdash e_1 \longrightarrow n_1 \quad \Delta, x = n_1 \vdash e_2 \longrightarrow n_2}{\Delta \vdash \text{var } x = e_1; e_2 \longrightarrow n_2}$$

Full syntax and semantics

$e ::= \dots \mid x \mid \text{var } x = e; e$

Syntax

$\Delta \vdash n \longrightarrow n$

E_n

$$\frac{\Delta \vdash e_1 \longrightarrow n_1 \quad \Delta \vdash e_2 \longrightarrow n_2}{\Delta \vdash e_1 + e_2 \longrightarrow n_1 + n_2}$$

E_+

...

Semantics

$$\frac{\Delta(x) = n}{\Delta \vdash x \longrightarrow n}$$

E_x

$$\frac{\Delta \vdash e_1 \longrightarrow n_1 \quad \Delta, x = n_1 \vdash e_2 \longrightarrow n_2}{\Delta \vdash \text{var } x = e_1; e_2 \longrightarrow n_2} E_{\text{var}}$$

Example derivation

Suppose that we want to find the result of evaluating:

var $x = 3$; $x + 5$

The following derivation illustrates the process:

$$\frac{\frac{\frac{\vdash 3 \longrightarrow 3 \ (E_n)}{\vdash \text{var } x = 3; x + 5 \longrightarrow 8} \ (E_{\text{var}})}{\frac{\frac{(x=3)(x) = 3}{x = 3 \mid - x \longrightarrow 3} \ (E_x)}{x = 3 \mid - 5 \longrightarrow 5 \ (E_n)} \ (E_+)} \ (E_{\text{var}})$$

Variables and Substitution

COMP3259

Principles of Programming Languages

Resources

Lecture covers:

- Chapter 3 of “Anatomy of Programming Languages”

<http://www.cs.utexas.edu/~wcook/anatomy/anatomy.htm>

Variable Discussion

What is meant by a Variable?

- Discussing variables in programming languages can be confusing. **What's the difference between?**

π

math (or Haskell)

$f(x) = x * x$

`int x = 0; x = x+1;`

Java or C

What is meant by a Variable?

- π is a constant

Math/FP

- $f(x) = x * x$ here x is a (immutable) variable

Java/C

- `int x = 0; x = x+1;` here x is a (mutable) variable

What is meant by a Variable?

- **constant**: The value of a constant is always the same in any **context**.
- **(immutable) variable**: In a particular **definition** the value of the variable never changes. However the value of the variable can be vary in different **contexts**.
- Example: different calls **f(2)**, **f(3)** use different values for the argument **x** of **f**, but **x** never changes in the definition of **f**.
- **(mutable) variable**: The value of a variable can change even in a particular **definition**.

Immutable variables in Java

- Although many languages have mutable variables by default, some languages allow immutable variables as well:

```
int f(final int x) {  
    return x * x;  
}
```

```
int g(final int x) {  
    x = x * x;  
    return x;  
}
```



This is a
compile-time error!

Meaning of variable the course

- By default, when referring to **variable** we mean **(immutable) variable**
- We will use the term **mutable variable** later in the course, when we talk about imperative programming.

Arithmetic Expressions with Variables

Arithmetic Expressions with Variables

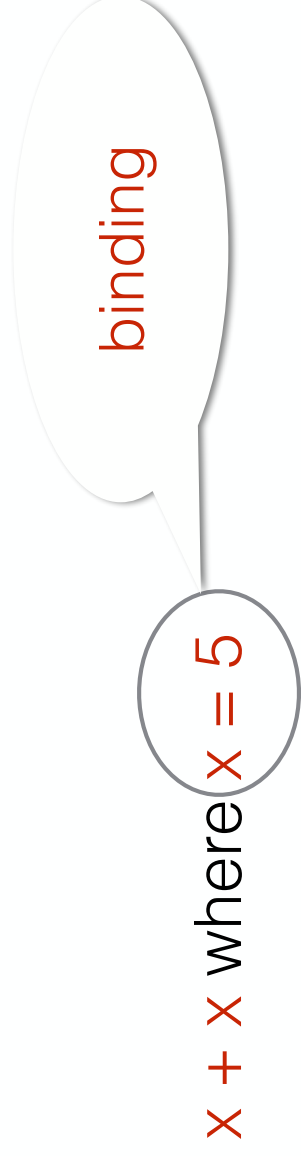
- A simple extension to the language of arithmetic is to allow variables:

$$x + x \text{ where } x = 5$$

- This can be useful to avoid repetition and reuse expressions

Bindings

- We call the pair constituted of a variable and the associated value a **binding**.



- We will use the following notation to denote binding:
$$\text{variable} \mapsto \text{value} \quad (\text{Example: } x \mapsto 5)$$
- Bindings can be represented in Haskell as a pair. For example **(“x”,5)**.

Substitution

- **Substitution** replaces a variable in with a value in an expression. For example:
 - substitute $x \mapsto 5$ in $x + 2 \longrightarrow 5 + 2$
 - substitute $x \mapsto 5$ in $2 \longrightarrow 2$
 - substitute $x \mapsto 5$ in $x \longrightarrow 5$
 - substitute $x \mapsto 5$ in $x * x + x \longrightarrow 5 * 5 + 5$
 - substitute $x \mapsto 5$ in $x + y \longrightarrow 5 + y$

Implementing Substitution

Arithmetic Expressions with Variables

- To allow this extension we first need to change the abstract syntax:

```
data Exp = Number Int
      | Add Exp Exp
      | Subtract Exp Exp
      | Multiply Exp Exp
      | Divide Exp Exp
      | Variable String – – added
deriving (Eq)
```

Substitution

- The substitution operation can be defined in Haskell as:

```
substitute1 :: (String, Int) → Exp → Exp  
substitute1 (var, val) exp = subst exp where  
  subst (Number i)      = Number i  
  subst (Add a b)       = Add (subst a) (subst b)  
  subst (Subtract a b)  = Subtract (subst a) (subst b)  
  subst (Multiply a b)  = Multiply (subst a) (subst b)  
  subst (Divide a b)    = Divide (subst a) (subst b)  
  subst (Variable name) = if var ≡ name  
    then Number val  
    else Variable name
```

Using Substitution

- The substitution function can be used as follows

substitute ("x", 5) $[x + 2]$
 $\implies [5 + 2]$

pseudo-code. Real code is:
Add (Variable "x") (Number 2)

substitute ("x", 5) $[32]$
 $\implies [32]$

substitute ("x", 5) $[x]$
 $\implies [5]$

substitute ("x", 5) $[x * x + x]$
 $\implies [5 * 5 + 5]$

substitute ("x", 5) $[x + 2 * y + z]$
 $\implies [5 + 2 * y + z]$

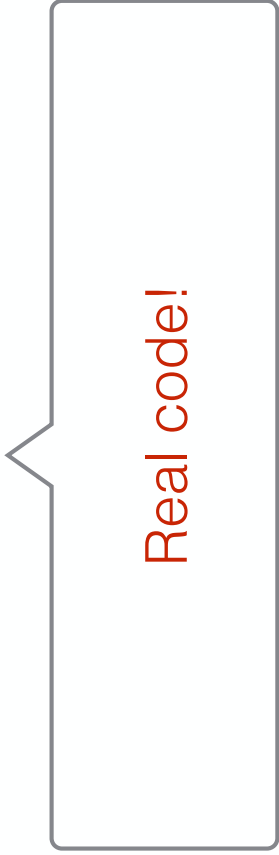
Using Substitution

The substitution function can be used as follows

```
substitute ("x", 4) (parseExp "x+5")
```

or even

```
substituteExp ("x", 4) "x+5"
```



Real code!

Multiple Substitution

Environments

- Arithmetic expressions can have multiple variables.
For example:

$2 * x + y$ where $x = 3$ and $y = -2$

- A collection of bindings is called an **environment**.
- In Haskell we can represent an **environment** as a **list of bindings**.

type *Env* = [(*String*, *Int*)]

Some Preliminaries

An important operation on environments is **variable lookup**:

lookup “x” [(“y” ,5), (“x” ,6)] == => 6

lookup is an operation that given a variable name and an environment returns the value bound to that variable in the environment.

Some Preliminaries

lookup can fail if the variable being looked up does not exist in the environment:

lookup “x” [(“y” ,5), (“z” ,6)] == => ????

Thus **lookup** needs to account for this possibility of failure.

**It is important that we know when lookup fails.
How can we do that?**

Multiple Substitution

- **Multiple substitution** allows us to simultaneously replace many variables at once using an environment.

$substitute :: Env \rightarrow Exp \rightarrow Exp$

$substitute\ env\ exp = subst\ exp\ where$

$subst\ (Number\ i) \quad = Number\ i$

$subst\ (Add\ a\ b) \quad = Add\ (subst\ a)\ (subst\ b)$

$subst\ (Subtract\ a\ b) = Subtract\ (subst\ a)\ (subst\ b)$

$subst\ (Multiply\ a\ b) = Multiply\ (subst\ a)\ (subst\ b)$

$subst\ (Divide\ a\ b) \quad = Divide\ (subst\ a)\ (subst\ b)$

$subst\ (Variable\ name) =$

case $lookup\ name\ env$ **of**

$Just\ val \rightarrow Number\ val$

$Nothing \rightarrow Variable\ name$

lookup the variable called **name**
in the environment

Multiple Substitution

- In **multiple substitution** the interesting case happens when we deal with variables.
- If the variable being looked up exists in the environment then we should substitute it by the corresponding value.

substitute [(“y”,6), (“x”,5)] (Variable “x”) ==> Number 5

- Otherwise we should just return the same expression

substitute [(“y”,6), (“x”,5)] (Variable “z”) ==> Variable “z”

Multiple Substitution

- Using multiple substitution

$e1 = [(\text{"x"}, 3), (\text{"y"}, -1)]$

$substitute\ e1\ [x + 2]$

$\Rightarrow [3 + 2]$

$substitute\ e1\ [32]$

$\Rightarrow [32]$

$substitute\ e1\ [x]$

$\Rightarrow [3]$

$substitute\ e1\ [x * x + x]$

$\Rightarrow [3 * 3 + 3]$

$substitute\ e1\ [x + 2 * y + z]$

$\Rightarrow [3 + 2 * -1 + z]$

Evaluation using Environments

Evaluation

- To deal with variables, evaluation can be modified to take an environment.

$\text{eval} :: \text{Env} \rightarrow \text{Exp} \rightarrow \text{Int}$

- This way it is possible to lookup the value of variables and deal with the new Variable case.

Implementing Evaluation with Environments

$evaluate :: Exp \rightarrow Env \rightarrow Int$
 $evaluate (Number\ i)\ env = i$
 $evaluate (Add\ a\ b)\ env = evaluate\ a\ env + evaluate\ b\ env$
 $evaluate (Subtract\ a\ b)\ env = evaluate\ a\ env - evaluate\ b\ env$
 $evaluate (Multiply\ a\ b)\ env = evaluate\ a\ env * evaluate\ b\ env$
 $evaluate (Divide\ a\ b)\ env = evaluate\ a\ env \text{ 'div' } evaluate\ b\ env$
 $evaluate (Variable\ x)\ env = fromJust\ (lookup\ x\ env)$

Arithmetic Expressions

COMP3259

Principles of Programming Languages

Expressions, Syntax and Evaluation

Resources

- Chapter 2 of “Anatomy of Programming Languages”

<http://www.cs.utexas.edu/~wcook/anatomy/anatomy.htm>

Programming Languages

- Wikipedia definition:
- A **programming language** is an artificial language designed to communicate instructions to a machine, particularly a computer.
- A more abstract definition:
- A (programming) **language** is a mechanism for expressing manipulations over certain types of values.

Language of Arithmetic

- **Values:** numbers
 - 1, 2, 3, 10, 100, 6893, ...
- **Arithmetic expressions:** using the language of arithmetic to denote some value
 - $2+3$, $3 * 4 - 10$, $4 - (-3)$
- **Values are not expressions!**
- **Multiple expressions can have the same value!**

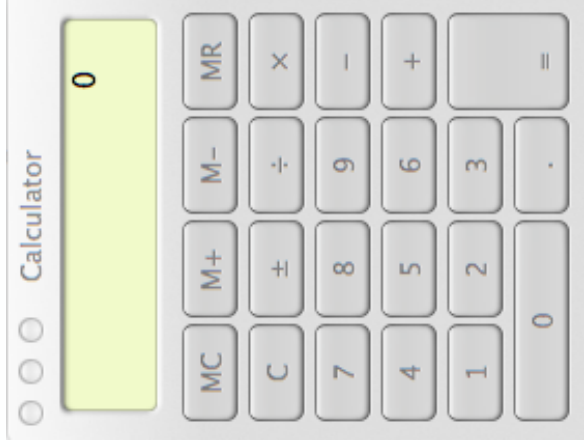
Expressions, Syntax and Evaluation

Three fundamental concepts:

- **Expression**: A combination of atomic components such as variables, values and operations over these values.
- **Syntax**: the syntax of an expression prescribes how the various components of the rules can be combined.
- **Evaluation**: gives the meaning of an expression in terms of a value.

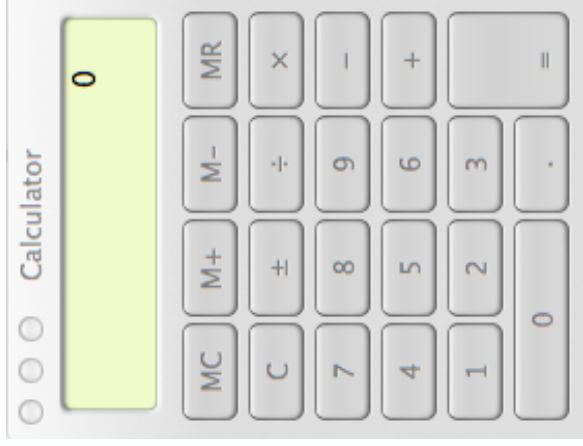
Representing Arithmetic Expressions

- Suppose that you are asked to implement a calculator in Haskell, what would you do?



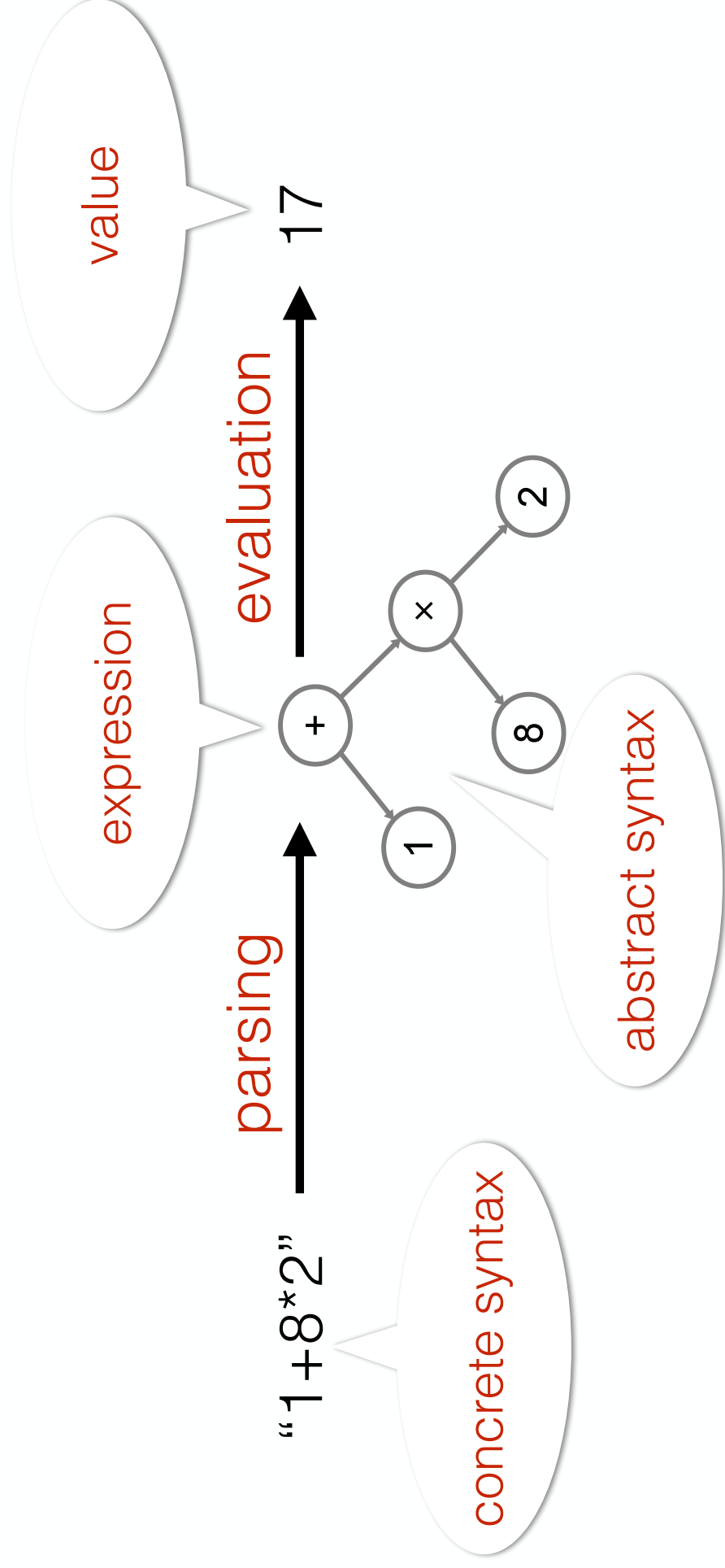
Representing Arithmetic Expressions

- Lets focus on two subproblems:
- What would be a good representation of arithmetic expressions in Haskell?
- How to evaluate arithmetic expressions?



Overview

An overview of the “architecture”:



Syntax and Parsing

- Concrete Syntax of Arithmetic Expressions:

4

-5+6

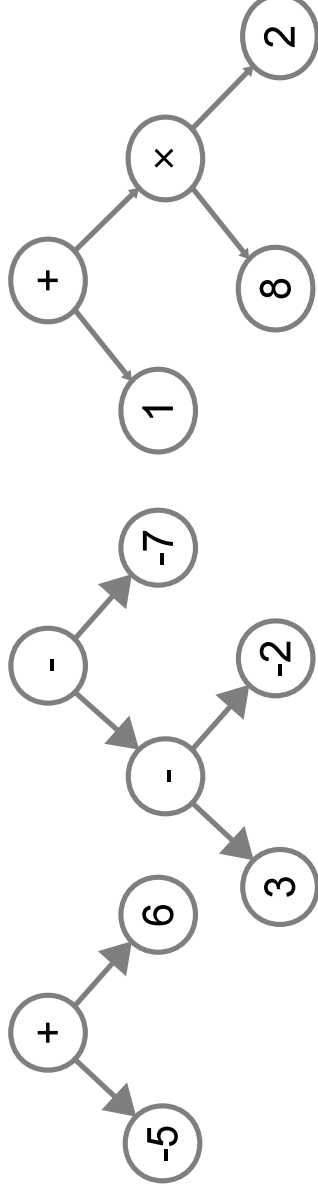
3--2--7

3*(8+5)

1+(8*2)

1+8*2

- Abstract Syntax of Arithmetic Expressions:

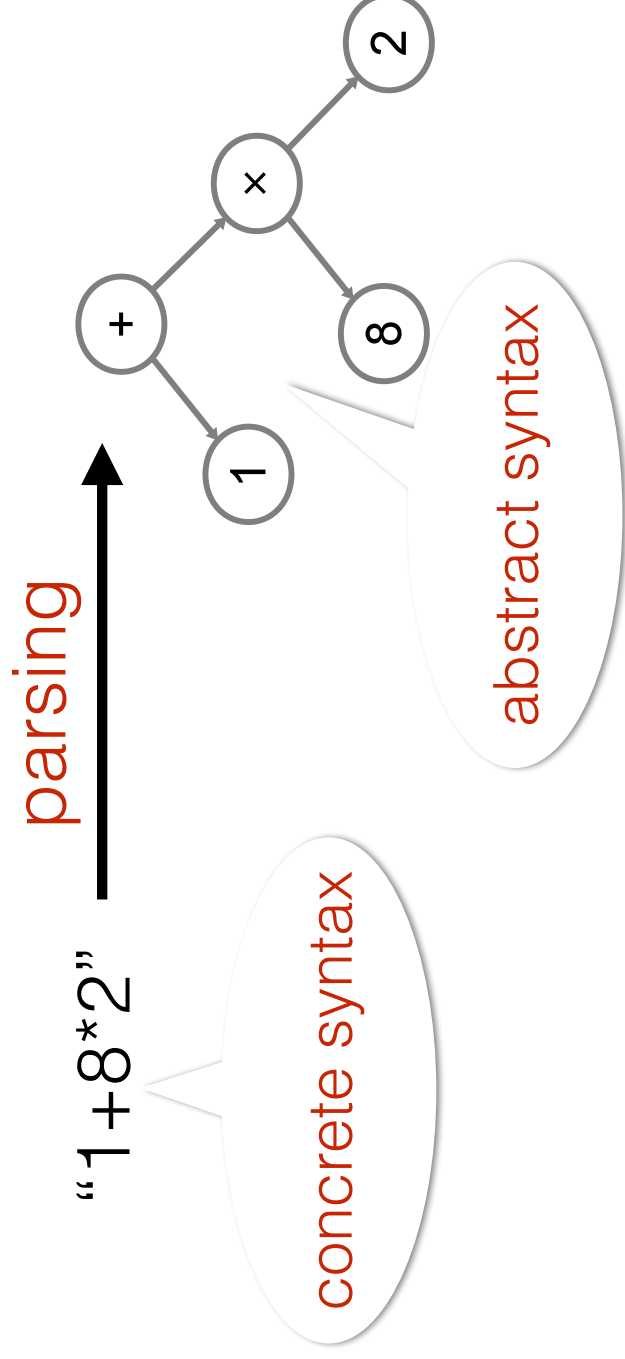


Syntax and Parsing

- **Concrete Syntax**: describes how the abstract concepts (such as expressions) in the language are represented as text.
- Usually **ambiguous**; care needs to be taken to define rules to rule out ambiguity.
- **Abstract Syntax**: A structured representation of the concepts in a language.
- **Unambiguous**, but not as convenient/easy to understand by humans.

Syntax and Parsing

- Parsing: process that converts between **concrete** syntax and **abstract syntax**.



Abstract Syntax

- Abstract Syntax of arithmetic expressions

$e ::= n \mid e + e \mid e - e \mid e * e \mid e / e$

Notation:

e - represents an expression

n - represents a number

- How to represent this in Haskell?

Abstract Syntax

- Abstract Syntax of arithmetic expressions

$e ::= n \mid e + e \mid e - e \mid e * e \mid e / e$

- Abstract syntax can be represented nicely with Haskell datatypes

```
data Exp = Number Int
      | Add      Exp Exp
      | Subtract Exp Exp
      | Multiply  Exp Exp
      | Divide    Exp Exp
```

Lets Build our First
Interpreter!

Code for Interpreter

```
evaluate :: Exp -> Int
evaluate (Number i)      = i
evaluate (Plus e1 e2)    =
    evaluate e1 + evaluate e2
evaluate (Minus e1 e2)   =
    evaluate e1 - evaluate e2
evaluate (Mult e1 e2)    =
    evaluate e1 * evaluate e2
evaluate (Division e1 e2) =
    evaluate e1 `div` evaluate e2
```

Code for Interpreter

Can we have some errors? If so which errors?

Answer: This interpreter is quite simple, but we can still have **division by zero errors**.

Language and Meta-Language

In this course we will be using Haskell to implement other languages. In this lecture we will implement a simple language of arithmetic in Haskell.

- **Be careful not to confuse the two languages!**

Language and Meta-Language

To avoid confusion:

- we refer to the implementation language as the **meta-language**,
- and the language being implemented as the **language**.
- **What are the language and meta-language in this lecture?**

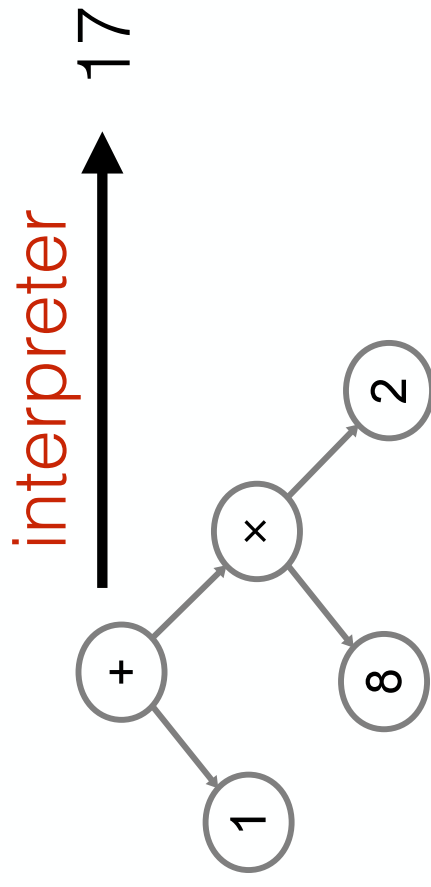
Language and Meta- Language

In this lecture:

- **Haskell** is the meta-language
- **Arithmetic** is the language

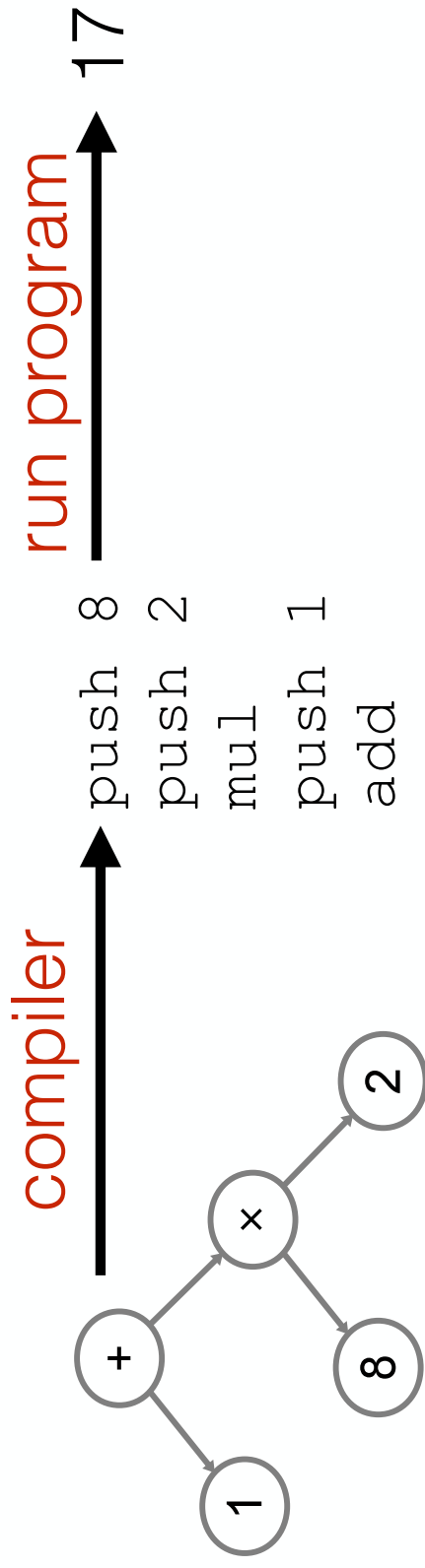
Evaluation: Compilation vs Interpretation

- **Interpreter:** Evaluates abstract syntax directly



Evaluation: Compilation vs Interpretation

- **Compiler:** Transforms abstract syntax into another language (possibly bytecode/assembly), which can be more efficiently executed afterwards.



Operational Semantics

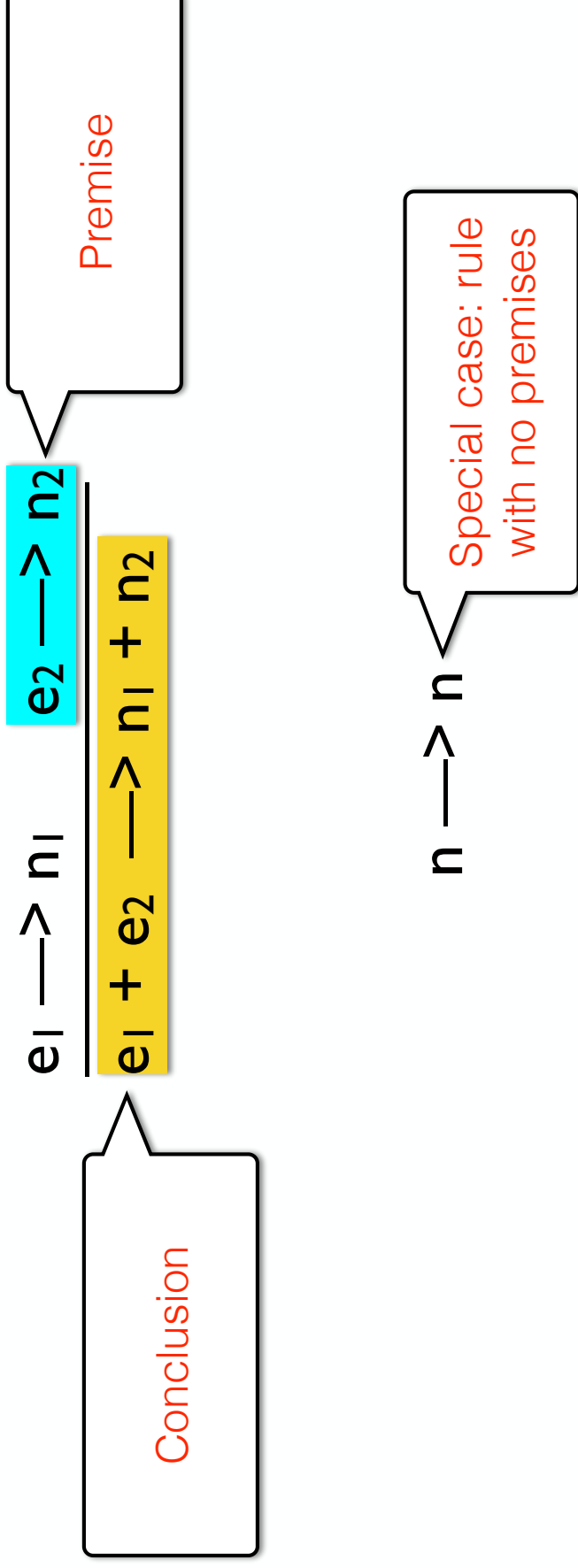
- The small evaluator that we have just written describes the **operational semantics** of the language of arithmetic expressions.
- Generally speaking an operational semantics describes the result (or meaning) of a program.
- We can describe operational semantics using a set of inference rules.

Operational Semantics:

Inference Rules

Inference rule	$n \longrightarrow n$	E_n
	$\frac{e_1 \longrightarrow n_1 \quad e_2 \longrightarrow n_2}{e_1 + e_2 \longrightarrow n_1 + n_2}$	E^+
	$\frac{e_1 \longrightarrow n_1 \quad e_2 \longrightarrow n_2}{e_1 * e_2 \longrightarrow n_1 * n_2}$	E^*
	$\frac{e_1 \longrightarrow n_1 \quad e_2 \longrightarrow n_2}{e_1 - e_2 \longrightarrow n_1 - n_2}$	E^-
	$\frac{e_1 \longrightarrow n_1 \quad e_2 \longrightarrow n_2}{e_1 / e_2 \longrightarrow n_1 / n_2}$	$E/$

Operational Semantics: Inference Rules



Inference Rules and Implementation

Inference rules are often in a one-to-one correspondence with the implementation:

- Inference rule:

$$\frac{e_1 \longrightarrow n_1 \quad e_2 \longrightarrow n_2}{e_1 + e_2 \longrightarrow n_1 + n_2}$$

- Haskell case in the implementation:

`evaluate (Plus e1 e2) = evaluate e1 + evaluate e2`

Full syntax and semantics

$e ::= n \mid e + e \mid e - e \mid e * e \mid e / e$

Syntax

$n \longrightarrow n$

E_n

$$\frac{e_1 \longrightarrow n_1 \quad e_2 \longrightarrow n_2}{e_1 + e_2 \longrightarrow n_1 + n_2}$$

E_+

$$\frac{e_1 \longrightarrow n_1 \quad e_2 \longrightarrow n_2}{e_1 * e_2 \longrightarrow n_1 * n_2}$$

E_*

$$\frac{e_1 \longrightarrow n_1 \quad e_2 \longrightarrow n_2}{e_1 - e_2 \longrightarrow n_1 - n_2}$$

E_-

$$\frac{e_1 \longrightarrow n_1 \quad e_2 \longrightarrow n_2}{e_1 / e_2 \longrightarrow n_1 / n_2}$$

$E_/_$

Semantics

Example of a Derivation

$$\frac{\frac{2 \longrightarrow 2 \ E_n \quad 3 \longrightarrow 3 \ E_n \quad E^*}{2 * 3 \longrightarrow 6}}{(2 * 3) + 4 \longrightarrow 10} \frac{4 \longrightarrow 4 \ E_n \quad E^+}{}$$

Haskell and Functional Programming Basics

COMP3259
Principles of Programming Languages

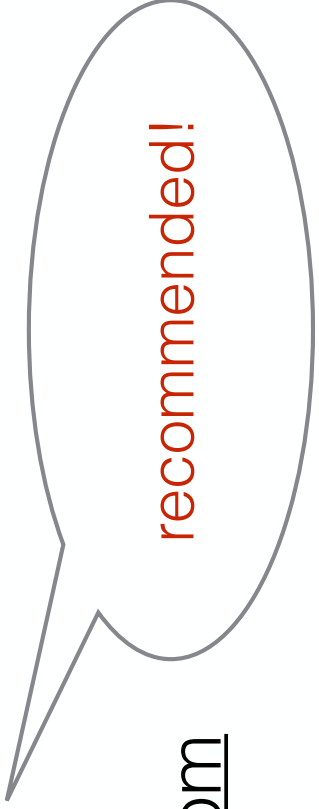
Goal of this Lecture

- To give you a crash course on the basics of Haskell and Functional Programming
- This will be needed for the course, since we will be using Haskell as the implementation Language

Suggested Reading

- Learn You a Haskell for Great Good! (up to Chapter 6)

- <http://learnyouahaskell.com>



recommended!

- Haskell tutorial (up to Section 4)
- <http://www.haskell.org/tutorial>

Functional Programming

- What is functional programming?
- **PS:** I won't be assuming that you are taking the Functional Programming class.

Functional Programming

- What is functional programming? Some possible answers:
 - Programming with first-class functions
 - `map (\x -> x + 1) [1,2,3]` $\sim > [2,3,4]$
 - Programming with **mathematical** functions
 - No side-effects (no global mutable state, no IO)
 - Calling a function with the same arguments, always returns the same output (not true in most languages!)
- The main means of computation is function application



Pure Functional
Programming

Functional Programming Languages



Traditionally focused on
Functional Programming

- Impure Functional Languages
 - Statically Typed: ML, OCaml, Scala ...
 - Dynamically Typed: Scheme, Lisp ...
- Pure Functional Languages
 - Statically Typed: Haskell

Functional Programming Languages

- Impure Functional Languages
 - Statically Typed: ML, OCaml, Scala, Java 8, C#, C++11, Swift ...
 - Dynamically Typed: Scheme, Lisp, Python, Ruby ...
- Pure Functional Languages
 - Statically Typed: Haskell, Agda, Idris



Bleeding Edge!

Haskell in the course

- Haskell is going to be used in the course as the implementation language
- to implement interpreters for programming languages
- to serve as an example of a (state-of-the-art) functional language

Why Haskell?

- Reason for Haskell in a Programming Languages course: Functional Languages have excellent mechanisms to implement programming languages:
- Datatypes & Pattern Matching

Installing Haskell

- Haskell Platform (for Windows, mac and Linux)

<http://www.haskell.org/platform/>

- IDE/Editor
 - Emacs
 - Visual Studio Code (Haskell)

Using Haskell

- The Haskell Platform installs the GHC compiler, which comes with a number of useful tools:
 - `ghc` is a compiler, similar to `gcc`
 - `ghci` is an interactive interpreter
 - great for debugging and testing programs

ghci

- Usage from the command line
 - `ghci`
 - `ghci <filename.hs>`
- Usage from emacs
 - Press **Control - C - L** when in Haskell mode

ghci

```
Brunos-iMac:AoPL bruno$ ghci
GHCi, version 7.6.3: http://www.haskell.org/ghc/  :? for help
Loading package ghc-prim ... linking ... done.
Loading package integer-gmp ... linking ... done.
Loading package base ... linking ... done.
Prelude> 3 + 4
7
c:\cmm.apple.screenscapture type image_format
3 emUIServer
Prelude>
```

Basic commands

- **:?** (help and available commands)
- **:! <file>** (load file)
- **:r** (reload file)
- **:set +t** (show type information)
- **:t e** (show the type of an expression)

Basics of ghci

Demo

Pattern Matching

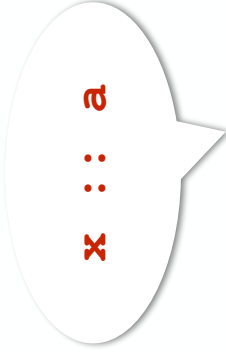
Pattern Matching

Haskell programs are often defined using **pattern matching**:

```
hd :: [a] -> a
hd [] = error "cannot take the head of an empty list!"
hd (x:xs) = x
```

To take the head of a list we need to consider two possibilities:

1. **[]** - The list is empty
2. **(x:xs)** - The list has one element **x** and a tail **xs**.



x :: a



xs :: [a]

Pattern Matching

Pattern matching can be used on many types:

```
first :: (a,b) -> a
first (x,y) = x
```

```
isZero :: Int -> Bool
isZero 0 = True
isZero n = False
```

Here we have used two different types of patterns:

1. **(x,y)** - A pattern on a tuple with 2 elements **x** and **y**.
2. **0** - A pattern on numbers.

Pattern Matching

Demo

Recursion

Recursion

Haskell programs are often defined using **recursion**:

```
countElements :: [a] -> Int
countElements [] = 0
countElements (x:xs) = 1 + countElements xs
```



recursive
call

Good recursive definitions normally have:

- **Base case(s)**: Cases where the program terminates.
 - For countElements the base case is **[]**.
- **Recursive case(s)**: Cases where a step is taken towards the base cases.
 - For countElements the recursive case is (**x:xs**).
 - Recursive calls are normally done on **smaller** values.

Recursion

Demo

Recursion

There may be programs with **bad recursion**:

```
badcountElements1 :: [a] -> Int
badcountElements1 [] = 0
badcountElements1 xs = 1 + badcountElements1 xs
```

why is this bad?

```
badcountElements2 :: [a] -> Int
badcountElements2 (x:xs) = 1 + badcountElements2 xs
```

why is this bad?

Recursion

There may be programs with **bad recursion**:

```
badcountElements1 :: [a] -> Int
badcountElements1 [] = 0
badcountElements1 xs = 1 + badcountElements1 xs
```

recursive call is
not smaller!

```
badcountElements2 :: [a] -> Int
badcountElements2 (x:xs) = 1 + badcountElements2 xs
```

no base case!

Case Analysis

- Haskell also supports case analysis
- Case analysis is an alternative way to use pattern matching
- It is sometimes **useful when we need to do something before we do pattern matching**

Case Analysis

- Example: Defining a `hasPositives` function

```
positives :: [Int] -> [Int] --uses pattern matching
positives [] = []
positives (x:xs) =
    if (x > 0) then x : positives xs
    else positives xs

hasPositives :: [Int] -> Bool -- uses case analysis
hasPositives xs =
    case positives xs of -- we first call positives
        [] -> False
        (x:xs) -> True
```

Datatypes

Datatypes

- How are Haskell lists defined? Conceptually they correspond to the following **datatype**:

data `[a] = [] | a : [a] -- pseudo-code`

- Note: Haskell lists are actually built-in. The above definition is just for illustration purposes.

User-Defined Datatypes

- Haskell supports user-defined datatypes:

```
data ListInt = Nil | Cons Int ListInt
```

recursive
definition

- New datatype definitions support their own pattern matching notation.

```
headListInt :: ListInt -> Int  
headListInt Nil = error "Empty list!"  
headListInt (Cons x xs) = x
```

Showing and Equality

- Some operations are useful for various datatypes
- Examples: **equality** and **conversion to a string**
- Haskell provides an easy mechanism for supporting these operations:

```
data ListInt = Nil | Cons Int ListInt
deriving (Eq, Show)
```

Showing and Equality

- Using equality and show:

```
test :: ListInt -> ListInt -> String
test l1 l2 = if (l1 == l2) then
    show l1
  else
    "Not equal!"
```

Demo

Maybe

- An important concern in this course will be **dealing with failure**: interpreters may fail for some expressions.
- The Maybe type (and also the Either type) will be helpful

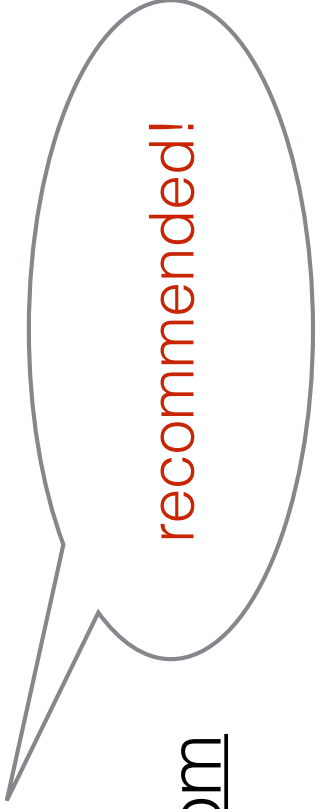
```
data Maybe a = Nothing | Just a
```

Demo

Suggested Reading

- Learn You a Haskell for Great Good! (up to Chapter 6)

- <http://learnyouahaskell.com>



recommended!

- Haskell tutorial (up to Section 4)
- <http://www.haskell.org/tutorial/>

Programming Languages

Course Information

COMP3259

Principles of Programming Languages

Basic Information

- Instructor
 - Dr. Bruno Oliveira (bruno@cs.hku.hk, CYC 420)
- Demonstrator
 - Wan Qianying (qywan@cs.hku.hk, HW1-01 M/F, #21)

About me

- Associate Professor at HKU
 - Language: English (and Portuguese)
- Research Interests
 - Programming Languages
 - Functional Programming (especially Haskell and Scala)
 - Object-Oriented Programming
 - Modularity

More about me

- I come from Portugal
- Best known these days as Cristiano Ronaldo's Land.
- Macau was administered by Portugal until 1999.



What is this course about?

(Short version)

- How do Programming Languages work?

What is this course about?

- Learning the Principles of Programming Languages.
- Basic concepts: Variable scoping, mutable state, closures, ...
- Programming language semantics
- Type systems
- Programming paradigms (and how to program using different paradigms)

What is this course about?

- Programming Paradigms
 - Functional Programming:
 - First-Class Functions
 - No mutable state (Immutable variables)
 - Imperative Programming
 - Mutable State (Mutable variables)
 - Object-Oriented Programming
 - Objects, Inheritance

What is this course good for?

- Learn general concepts used by many languages
 - Java, C, Haskell, ... all have many concepts in common
 - By the end of the course you should be able to pick up new languages faster
- Learn to think differently about programming
 - Functional Programming vs Imperative Programming
 - Apply techniques from different paradigms to any languages
- Learn how to build your own languages
- To make you a better programmer!

Requirements

- This is not a beginners programming course
- Students need to be comfortable programming in at least one Language (Java, C, C++, ...)
- Some basic knowledge about algorithms is desirable
- **Functional Programming** is recommended, but not a requirement.

Comparison to other Courses

Functional Programming vs Programming Languages

- I teach 2 courses:
 - CSIS0259 / COMP3259: Principles of Programming Languages
 - CSIS2258 / COMP3258: Functional Programming
- What is the relation and differences between these 2 courses? Should I take both?

Functional Programming vs Programming Languages

- Functional Programming
 - **Goal: Learn how to do Functional Programming**
 - It will be a full course on Functional Programming and **Haskell**
- Programming Languages:
 - **Goal: Learn how Programming Languages work**
 - Develop interpreters for Programming Languages
 - **Haskell used to develop the interpreters**

What Course should I take?

- Functional Programming first
- Works well
- Programming Languages **first** or **both** at once
- Ok, **if you are comfortable with programming** and maybe you don't want to take Functional Programming.
- First week of PL are spend on a crash course on Haskell (only features needed for implementing interpreters are needed)

Methodology of the course

- Learn by doing
 - You will implement programming languages with the concepts learned in class
- Growing a Language with features
 - Start with a small language (arithmetic expressions) and add more features (conditionals, function definitions, higher-order functions, mutable-state, ...)
- Use Haskell as Implementation Language

<http://www.haskell.org/haskellwiki/Haskell>

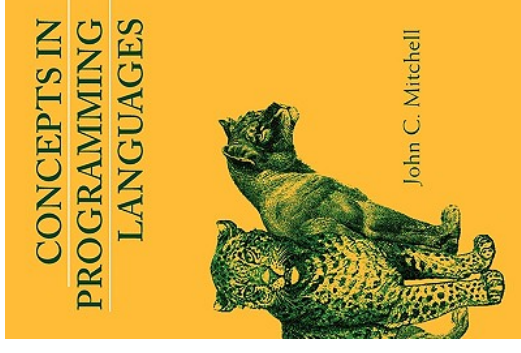
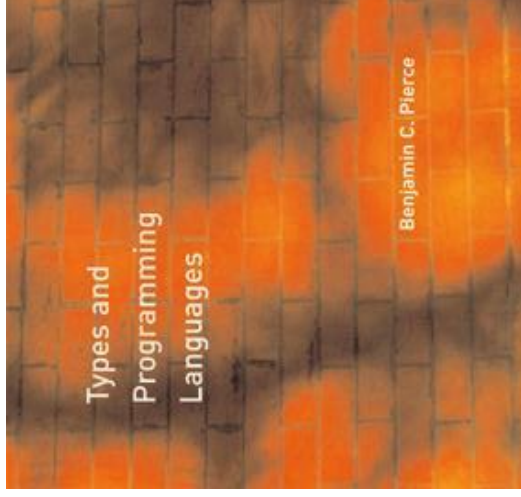
Schedule

Week	Topic
1	<i>Introduction/Haskell</i>
2	<i>Basic Expressions</i>
3 - 4	<i>Closures and first-class functions (Functional Programming)</i>
5 - 6	<i>Recursion and Data Abstraction</i>
7 - 8	<i>Errors and Mutable State (Imperative Programming)</i>
9 - 10	<i>Objects and Inheritance (Object-Oriented Programming)</i>
11 - 12	<i>Mini JavaScript</i>

Warning: Content may change according to progress!

Basic Information

- Reference Books and Materials
 - Anatomy of Programming Languages (book in development by Prof. William Cook, with my own contributions)
 - Types and Programming Languages (Benjamin Pierce, MIT Press)
 - Concepts in Programming Languages (John C. Mitchell, Cambridge Press)



Lectures and Tutorials

- Tuesday (Tutorials)
 - Time: 15:30 ~ 16:20
 - Venue: LE9
- Friday (Lectures and first Tuesday)
 - Time: 15:30 - 17:20
 - Venues: LE9

Tutorials

- Tutorial Participation
- Not compulsory
- But participation recommended!
- Please answer/raise questions during tutorial.

Policy

- Late assignments
 - upto 1 day late submissions (**15% of marks removed**)
 - upto 3 days (**30% of marks removed**)
 - more than 3 days (**not accepted**)
- Collaboration in study groups is encouraged, but you should **write your own program for the assignments**.
- Plagiarism will be taken seriously! (Can be reported to the university).

Assessments

- We will use the following assessment scheme for this course:
 - 3 assignments (40%)
 - Final exam (60%)

Communication Channels

- Please come to us if you have any difficulties in the course
- There are several ways to contact and get in touch with us:
 - email
 - newsgroup
 - consultation hours
 - ...